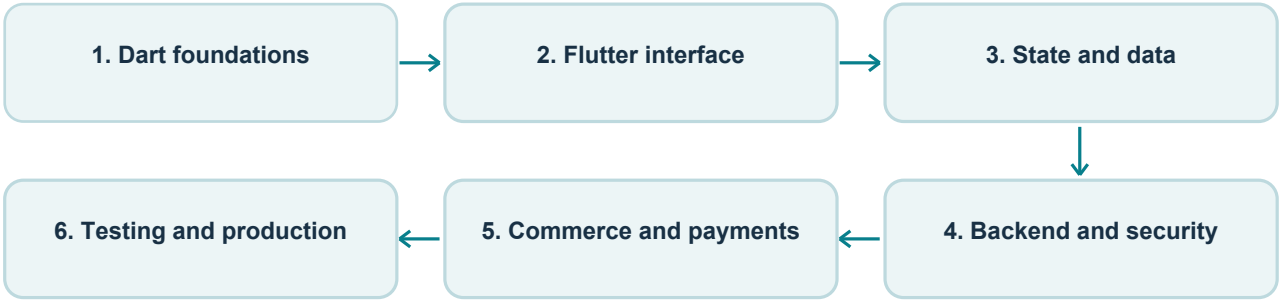
FLUTTER
Complete learning notes

From your first Dart program to a connected shirts shop

Concepts explained in ordinary language. Complete source listings. Line-by-line code notes. Expected screens, diagrams, exercises, and failure cases.



A practical study edition based on your Levels 0-20 learning roadmap.

Prepared 4 October 2026 | Self-contained PDF with an ML integration workshop

Contents

Before you begin 4

Level 0 | Computer and programming foundations 5

Level 1 | Flutter fundamentals 13

Level 2 | Layout, widgets, and professional UI 19

Level 3 | Application fundamentals 31

Level 4 | State management and professional frontend 48

Level 5 | Backend, APIs, and databases 53

Level 6 | Authentication and sign-in 65

Level 7 | Clean Architecture and SOLID 77

Level 8 | Designing the e-commerce application 81

Level 9 | Payments that can be trusted 93

Level 10 | Admin and management systems 106

Level 11 | Database and data architecture 109

Level 12 | Security with concrete examples 112

Level 13 | Testing from small rules to complete journeys 114

Level 14 | Performance based on measurement 123

Level 15 | Git, deployment, and delivery automation 125

Level 16 | Cloud infrastructure without unnecessary services 131

Level 17 | Monitoring and production operation 133

Level 18 | Scalability and practical architecture 135

Level 19 | Advanced architecture in plain language 137

Level 20 | Final shirts platform: run, understand, extend 142

Level 21 | Machine learning and Flutter integration 160

Worked exercises and debugging notebook 183

Reference glossary 184

Sources, versions, and verification 185

Before you begin

This is a working notebook for learning to build a shirts shop. Read in order, type the examples, run them, and change them. Knowing a definition is different from being able to use it. Your test is whether you can predict a result, explain a failure, and make a small change without copying a solution.

The book covers Levels 0 through 20 of your roadmap and an added Level 21 on machine learning and Flutter integration. Small programs teach one idea at a time. The larger projects combine those ideas. Pictures are labelled either as screens rendered from Flutter widget tests or as explanatory illustrations. None is presented as a deployed product. Terminal output is shown separately. A learning implementation is identified wherever it simplifies security, payments, storage, or operations. Do not mistake a successful local demonstration for a completed production review.

This PDF contains the source listings needed for the worked examples. Create each named file by copying its listing without the printed line numbers. Wrapped continuation lines belong to the same source line unless a new number begins. Each listing names its destination file. A Dart console file runs with `dart run filename.dart`. A Flutter main file replaces `lib/main.dart` in a project created with `flutter create`. Python examples use a virtual environment. Commands are written for Windows PowerShell unless stated otherwise. Run commands in the folder named by the lesson.

Read code punctuation as carefully as words. Parentheses `()` contain arguments or a condition. Curly braces `{}` enclose a block or a map. Square brackets `[]` represent a list or indexing. A semicolon ends a Dart statement. Indentation shows nesting to humans; in Python it also determines the block. Comments explain the line immediately before them or the next small block. They are not instructions executed by the computer.

How to study a lesson

1. Read the concept and the problem it solves before looking at the code.

2. Predict the example's result. Write your prediction in a notebook.

3. Create the named file, run it, and compare the real result with the expected result.

4. Explain each line aloud, including the inputs and the value it produces.

5. Complete the exercise. Deliberately cause one error and fix it.

6. Explain where the idea belongs in the final shop.

Use version control after the Git chapter to save each working step. Until then, keep a separate folder for each exercise. Never overwrite your only working copy while experimenting.

Level 0 | Computer and programming foundations

0.1 Programming, languages, and Dart

1. Concept. Programming means writing precise instructions for a computer. A program is the stored set of instructions. A programming language supplies the words and grammar used to express them. Dart is the language used for the Flutter code in this book. Flutter is a toolkit for building applications; Dart is how we describe their behavior.

2. Why it exists. A computer cannot infer what you meant by "calculate the bill properly." You must specify which prices to read, how quantities affect them, how to handle invalid values, and what to display.

3. Why Flutter needs it. Flutter supplies reusable screen-building objects. Dart describes which objects to create and what should happen when someone taps a button.

4. Analogy. A recipe is a program, its written language is the programming language, and the kitchen equipment is the toolkit. A missing recipe step produces the wrong meal even if the equipment works.

5. Small example. Ask the computer to print a greeting. The console is a text window used to show messages; it is not the app's graphical screen.

6. Code - save as hello.dart.

```
void main() {
  print('Hello, shirt shop!');
}
```

7. Every line. Line 1 declares the starting function `main`. `void` means this function does not return a useful value to its caller. The opening brace starts its body. Line 2 calls the built-in `print` function with a string, which is text between quotes. The semicolon ends the instruction. Line 3 closes the function.

8. Visual and expected output.

```
Run file -> enter main -> call print -> console
Hello, shirt shop!
```

9. Internally. Dart checks and translates your source so it can execute. During Flutter development, a running Dart environment supports quick changes. Release builds use platform-appropriate compilation; the web target differs from native machine-code targets. You do not need engine implementation details to write this program.

10. Mistakes. `Main` and `main` are different names. Missing quotes makes Dart look for a variable. A missing semicolon often causes a message on the following line; inspect the line above too.

11. Exercise. Print your name, then a second message saying what the shop sells. Run `dart run hello.dart` from the file's folder.

12. Project usage. Logging and small console experiments help you understand calculations before connecting them to widgets.

0.2 Variables, types, strings, numbers, and Boolean values

1. Concept. A variable is a name for a value. A type tells Dart what kind of value is allowed. `String` holds text, `int` holds whole numbers, `double` holds floating-point numbers, and `bool` holds `true` or `false`. A declaration introduces a name; assignment gives it a value.

2. Why it exists. A program needs to remember information between instructions. Types catch mistakes such as trying to add a customer's name to a price.

3. Why Flutter needs it. Screen titles, quantities, loading indicators, and selected sizes all depend on values.

4. Analogy. Imagine labelled containers in a shop. "Stock count" accepts whole numbers; "customer name" accepts text. The label prevents putting the wrong kind of information into the container.

5. Small example. Two shirts at 799 rupees cost 1,598 rupees before other charges. Store money as an integer number of paise: 799 rupees is 79,900 paise. This avoids floating-point rounding in authoritative money calculations.

6. Code - values.dart.

```
void main() {
  final String name = 'Oxford shirt';
  const int pricePaise = 79900;
  int quantity = 2;
  quantity = quantity + 1;
  final int total = pricePaise * quantity;
  final bool available = quantity <= 5;
  final double displayedRupees = total / 100;
  print('$name: $quantity');
  print(displayedRupees.toStringAsFixed(2));
  print(available);
}
```

7. Every line. Line 1 starts the program. Line 2 creates a text value that cannot later be reassigned because it is `final`. Line 3 defines a compile-time constant price. Line 4 creates a changeable quantity. Line 5 reads the old quantity, adds one, and assigns three. Line 6 multiplies price by quantity once and keeps the result. Line 7 compares quantity with five and stores a Boolean. Line 8 divides by one hundred for display; this is not the stored authoritative money amount. Line 9 inserts variables into text using `$`. Line 10 formats the display number with two decimal places. Line 11 prints the Boolean. Line 12 ends the body.

8. Expected output.

```
Oxford shirt: 3
2397.00
true
```

9. Internally. Dart associates names with values and checks operations against their types. `var quantity = 2` infers `int`; it does not mean the variable accepts every type. `final` prevents reassignment, but a final list can still contain mutable elements. `const` requires a constant expression and creates a deeply constant value where applicable.

10. Mistakes. `=` assigns, while `==` compares. `'2'` is text and `2` is a number. `/` returns a division result; `~/` performs truncating integer division. Never silently truncate money when applying discounts: explicitly

define the rounding rule.

11. Exercise. Change quantity to four and predict the total. Add `bool soldOut = false`. Print an interpolated sentence using both the name and availability.

12. Project usage. Product prices, stock counts, selected quantities, and "loading" flags use these types. Format money at the edge of the system; calculate it with integers on the server.

0.3 Lists and maps

1. Concept. A list stores an ordered sequence. A map associates a key with a value. `List<String>` means a list whose elements are strings. `Map<String, int>` means string keys and integer values. The angle brackets describe the contained types; this is called a generic type.

2. Why it exists. A shop has many products and many stock counts. Separate variable names for every item would make searching, looping, and adding products awkward.

3. Why Flutter needs it. Lists drive scrolling screens. Maps are useful for JSON-shaped data and for quantities indexed by product identifier.

4. Analogy. A list is a numbered shelf. A map is a set of labelled drawers: ask for the drawer labelled `M` and receive its stock count.

5. Small example. List available sizes and look up the number of medium shirts.

6. Code - collections.dart.

```
void main() {
  final sizes = <String>['S', 'M'];
  sizes.add('L');
  final stock = <String, int>{'S': 2, 'M': 5};
  stock['L'] = 3;
  print(sizes[0]);
  print(sizes.length);
  print(stock['M']);
  print(stock['XL'] ?? 0);
}
```

7. Every line. Line 1 starts execution. Line 2 creates the list and fixes its variable binding. Line 3 mutates that list by adding `L`. Line 4 creates two key-value pairs; colons separate keys from values. Line 5 adds a new pair. Line 6 reads index zero, the first element. Line 7 reads the number of elements. Line 8 looks up `M`. Line 9 uses the fallback operator `??` because a missing map key returns `null`. Line 10 closes the function.

8. Expected output.

```
S
3
5
0
```

9. Internally. A list checks that its index is within range. A map lookup may fail to find a key, which is different from finding a key with quantity zero. Dart's type system reflects this uncertainty through a nullable lookup result.

10. Mistakes. The last index is `length - 1`, not `length`. Reusing the same map key replaces its value. Mutating a collection while iterating through it may fail; build a new collection or remove items after the loop.

11. Exercise. Add an `XL` count, print all sizes, and calculate the sum of the map's values after you learn loops.

12. Project usage. The cart maps variant IDs to quantities. A product catalogue is a list. Prefer a typed product class over a raw map once data passes into your application logic.

0.4 Conditions and loops

1. Concept. A condition chooses a path. A loop repeats instructions. `if`, `else if`, and `else` form choices. `for` is convenient when visiting a collection; `while` repeats as long as a condition remains true.

2. Why it exists. Different inputs need different responses. Processing twenty products should not require twenty copies of the same code.

3. Why Flutter needs it. Show an empty-cart message only when the cart is empty. Build one row for each product. Stop allowing additions when stock is exhausted.

4. Analogy. A cashier checks whether a coupon is valid, then scans every item in the basket.

5. Small example. Classify stock counts as empty, low, or available, and count down packing steps.

6. Code - control.dart.

```
void main() {
  for (final stock in <int>[0, 2, 8]) {
    if (stock == 0) {
      print('Sold out');
    } else if (stock < 3) {
      print('Low stock');
    } else {
      print('Available');
    }
  }
  int remaining = 2;
  while (remaining > 0) {
    print('Pack $remaining');
    remaining--;
  }
}
```

7. Every line. Line 1 starts `main`. Line 2 assigns each list value to a fresh loop variable in turn. Line 3 checks equality with zero. Line 4 prints only for zero stock. Line 5 chooses a second test if the first failed. Line 6 prints for positive stock below three. Line 7 selects the remaining case. Line 8 prints availability. Line 9 closes the choice, and line 10 closes the loop. Line 11 creates a counter. Line 12 checks it before each repetition. Line 13 prints the current counter. Line 14 subtracts one. Line 15 ends the while body. Line 16 ends the program.

8. Expected output.

```
Sold out
Low stock
Available
Pack 2
Pack 1
```

9. Internally. Conditions are evaluated in order; the first matching branch runs. A while loop can execute zero times. A loop that never changes its stopping condition can run forever and freeze a user interface.

10. Mistakes. `&&` means both conditions must hold; `||` means at least one must hold. Test boundary values such as zero, one, and the maximum. A negative stock count should be rejected rather than classified as ordinary low stock.

11. Exercise. Add a branch rejecting negative stock and test `-1, 0, 1, 3`. Write a counted loop that prints numbers one through five.

12. Project usage. Validation, filters, order-state decisions, and rendering lists depend on conditions and repetition.

0.5 Functions, parameters, and return values

1. Concept. A function is a named operation. Parameters are the inputs it accepts. A return value is its answer. A caller is the code that requests the operation. A pure function depends only on its inputs and does not change outside state.

2. Why it exists. A calculation should have one clear definition. Copying the same calculation into multiple screens makes bug fixes unreliable.

3. Why Flutter needs it. Event handlers react to taps; validation functions check fields; pricing functions calculate subtotals.

4. Analogy. A calculator accepts two numbers and an operation, then returns an answer. Its display is separate from the calculation itself.

5. Small example. Return a subtotal and reject a negative quantity.

6. Code - functions.dart.

```
int subtotal(int pricePaise, {required int quantity}) {
  if (quantity < 0) throw ArgumentError('Negative quantity');
  return pricePaise * quantity;
}
void main() {
  final result = subtotal(79900, quantity: 2);
  print(result);
}
```

7. Every line. Line 1 declares an integer result, a positional price parameter, and a required named quantity parameter. The braces inside parentheses identify named parameters; the final brace opens the function body. Line 2 rejects an invalid input with an exception. Line 3 returns the calculation and finishes this call. Line 4 closes the function. Line 5 begins the entry point. Line 6 calls the function and saves its result. Line 7 prints that result. Line 8 closes `main`.

8. Expected output. `159800`. The flow is `inputs -> validation -> multiplication -> return -> print`.

9. Internally. Each call has its own parameter values and local variables. A variable declared inside the function is not automatically visible outside it; this boundary is called scope. `=> expression` is shorthand for returning one expression.

10. Mistakes. Printing a value is not the same as returning it. A `void` function is appropriate for an action, but a reusable calculation should return its result. Avoid giving a function unrelated responsibilities such as calculating money, making a network request, and opening a dialog.

11. Exercise. Add a named discount in paise. Reject a discount larger than the subtotal. Write down whether a zero quantity is allowed and make your code follow that rule.

12. Project usage. The backend owns final prices, but the same separation of calculation from display makes the Flutter cart easy to test.

0.6 Classes, objects, and constructors

1. Concept. A class defines the shape and behavior of a kind of thing. An object is one actual value of that class. A constructor creates the object. Fields hold its data; methods are functions attached to it.

2. Why it exists. Separate lists of names, prices, and identifiers can fall out of alignment. A product object keeps related values together.

3. Why Flutter needs it. Flutter widgets themselves are objects. Your application also needs clear models for products, employees, orders, and addresses.

4. Analogy. A blank product card is a class; a filled card for the blue Oxford shirt is an object. Filling in the required fields is the constructor's job.

5. Small example. Create a product with a stable ID and a price.

6. Code - objects.dart.

```
class Product {
  final int id;
  final String name;
  final int pricePaise;
  const Product(this.id, this.name, this.pricePaise);
  int costFor(int quantity) => pricePaise * quantity;
}
void main() {
  const shirt = Product(1, 'Oxford', 79900);
  print(shirt.name);
  print(shirt.costFor(2));
}
```

7. Every line. Line 1 begins the class. Lines 2, 3, and 4 declare immutable ID, name, and price fields. Line 5 is a constructor; `this.id` and its companions assign the arguments to fields. `const` permits constant objects when arguments are constant. Line 6 is a method that returns a multiplication result. Line 7 closes the class. Line 8 starts `main`. Line 9 constructs one product. Line 10 reads its name using a dot. Line 11 calls its method with two. Line 12 closes the program.

8. Expected output. `Oxford`, then `159800` on the next line. Visually: `Product blueprint -> shirt object -> fields and methods`.

9. Internally. Different instances have their own field values. `final` fields support predictable models. Replacing a whole immutable object is often easier to reason about than changing it invisibly in several places.

10. Mistakes. An object ID is not its position in a list. Two separately created objects do not automatically compare equal by all fields. Do not use a widget as your database model.

11. Exercise. Add a `size` field and construct two variants with different IDs. Decide which information belongs to a product and which belongs to a sellable size-colour variant.

12. Project usage. Typed models form the boundary between JSON, business rules, and UI.

0.7 Null safety and error handling

1. Concept. `null` means no value is present. `String?` permits text or null; `String` requires text. An exception signals that an operation could not complete normally. `try` surrounds a risky operation; `catch` handles a failure.

2. Why it exists. A customer may have no middle name, a lookup may find nothing, and user input may not be a valid number. These cases must be explicit.

3. Why Flutter needs it. Screens often have data that has not arrived yet. An app should show a useful error instead of crashing when a network request fails.

4. Analogy. An empty mailbox is not a letter containing an empty sentence. Absence and an empty value have different meanings.

5. Small example. Read an optional name and safely parse a quantity.

6. Code - errors.dart.

```
void main() {
  String? nickname;
  print(nickname ?? 'Guest');
  final quantity = int.tryParse('two');
  print(quantity ?? 0);
  try {
    final value = int.parse('two');
    print(value);
  } on FormatException catch (error) {
    print('Enter digits, such as 2.');
    print(error.runtimeType);
  }
}
```

7. Every line. Line 1 starts execution. Line 2 declares a nullable local without a value. Line 3 selects `Guest` because the value is null. Line 4 tries to parse text and returns null on invalid input. Line 5 displays a fallback zero for this demonstration. Line 6 opens a guarded block. Line 7 uses the throwing parser. Line 8 would print only if parsing succeeded. Line 9 catches this particular exception type. Line 10 shows an understandable instruction. Line 11 prints the error's type for learning. Lines 12 and 13 close the handler and program.

8. Expected output. `Guest`, `0`, `Enter digits, such as 2.`, and `FormatException`, each on a separate line.

9. Internally. Throwing skips normal instructions until a matching handler is found. `finally` runs cleanup whether the operation succeeds or fails. `?.` accesses a member only if the receiver exists. `!` asserts that a value is not null and throws if your assertion is wrong.

10. Mistakes. Adding `!` to silence the analyzer hides the question instead of answering it. Do not convert invalid checkout quantities to zero silently. Show validation feedback. Catch errors at a boundary where you can actually recover or report them.

11. Exercise. Try `''`, `'0'`, `'-1'`, and `'3'`. Accept only positive quantities. Explain why parsing and business validation are separate steps.

12. Project usage. Optional addresses, missing products, failed requests, and expired sessions all require explicit handling.

Level 0 practical: console receipt

Create a folder named `console_receipt`. Save the following as `main.dart`. This project combines a class, a collection, a loop, integer money, and a function. It does not need Flutter widgets.

```
class LineItem {
  final String name;
  final int price;
  final int quantity;
  const LineItem(this.name, this.price, this.quantity);
  int get total => price * quantity;
}
String rupees(int paise) => (paise / 100).toStringAsFixed(2);
void main() {
  const items = [LineItem('Oxford', 79900, 2),
    LineItem('Linen', 99900, 1)];
  int total = 0;
  for (final item in items) {
    total += item.total;
    print('${item.name} x${item.quantity}: ${rupees(item.total)}');
  }
  print('Total INR ${rupees(total)}');
}
```

Line walkthrough. Line 1 declares the receipt-entry type. Lines 2-4 define name, price in paise, and quantity. Line 5 fills those fields. Line 6 declares a getter, which is read like a field but computes the line total. Line 7 closes the class. Line 8 converts an integer amount into display text. Line 9 starts execution. Lines 10-11 create two immutable entries in a constant list. Line 12 initializes the accumulator. Line 13 visits each entry. Line 14 adds that entry's total. Line 15 prints its description. Line 16 finishes the loop. Line 17 prints the combined total. Line 18 closes `main`.

Run `dart run main.dart`. Expect `Oxford x2: 1598.00`, `Linen x1: 999.00`, and `Total INR 2597.00`. If your output is 1798.00, you probably forgot to multiply the Oxford price by two. Modify the program to print a delivery fee and a grand total. Before moving on, explain why a price is an integer while the displayed amount contains a decimal point.

Level 1 | Flutter fundamentals

1.1 What Flutter does

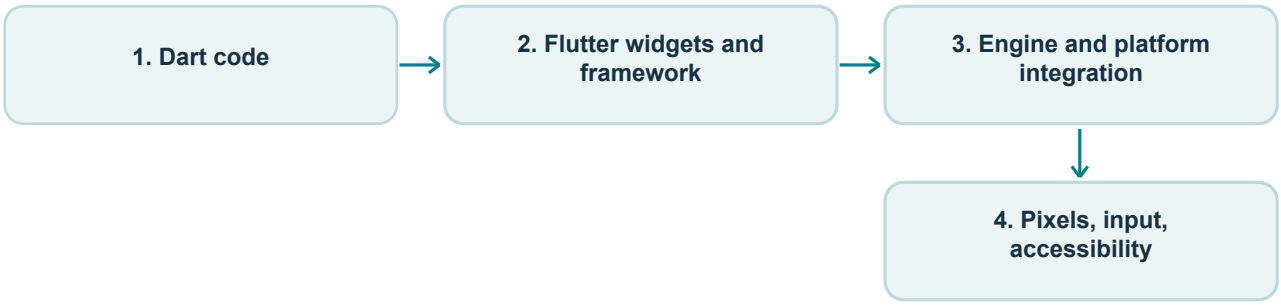
Concept. Flutter is Google's open-source UI toolkit for building applications from a shared Dart codebase. UI means user interface: the text, controls, pictures, and interactions a person sees. A framework gives structure and reusable building blocks, so you do not draw every letter and implement every button yourself.

Problem and purpose. Companies often need similar behavior on several platforms. Sharing a substantial part of the application can reduce duplicated implementation. It does not eliminate platform work. Permissions, sign-in integration, payments, accessibility, keyboard behavior, packaging, and store requirements still need platform-specific attention.

Analogy. A construction system supplies doors, windows, and standard connections. You still design the building and obey local requirements.

Flutter is different from writing an Android app directly with Android-specific UI tools. Native development gives direct access to that platform's conventions and APIs; Flutter gives a shared widget and rendering system. React Native uses JavaScript or TypeScript and a different rendering/component integration approach. Ordinary web development builds browser pages with HTML, CSS, and JavaScript; Flutter web renders a Flutter application inside a browser. A content-heavy public site and an interactive application have different needs. Choose by requirements, team skill, platform integration, and measured behavior, not by a universal ranking.

1.2 A beginner's architecture picture



Your Dart code describes the interface and behavior. Flutter's framework manages widget descriptions, state, layout, painting instructions, and gestures. Lower layers render and communicate with the operating system. A platform is an environment such as Android, iOS, Windows, or the browser. The exact rendering path varies with target and configuration. This picture is a learning map, not a claim that all work happens in one linear call.

A widget is a description of part of an interface. An element is the framework's persistent connection between a widget description and its place in the tree. Some widgets configure render objects, which perform layout and painting. You can build useful apps before mastering these internals; remember only that replacing a description does not require throwing away every underlying object.

1.3 Installing the tools and checking them

The SDK, or software development kit, contains the tools needed to build and run programs. The Flutter SDK includes a compatible Dart SDK. An editor such as VS Code helps you write code. Android Studio supplies Android tooling and an emulator manager even if you prefer VS Code for editing. The Android SDK contains platform tools and build components. An emulator behaves like a phone on your computer. A real device catches performance and hardware issues an emulator may miss.

Follow the platform-specific official installation instructions [S1]. On Windows, place the Flutter SDK in a user-writable location, add its `bin` directory to PATH, then open a new terminal. PATH is the list of folders the shell searches for executable commands. Installing an editor extension alone does not guarantee all target-platform tools are installed.

```
flutter --version
dart --version
flutter doctor -v
flutter devices
```

The first two commands identify the installed tool versions. `doctor` reports missing target dependencies. `devices` lists available run targets. Fix the target you intend to use first; you do not need every platform to be ready. Accept Android SDK licences through `flutter doctor --android-licenses` after installing the required SDK tools. Building and signing iOS applications requires Apple's supported macOS/Xcode toolchain.

If the terminal says a command is unknown, check PATH and restart the terminal. If no Android device appears, start an emulator or enable USB debugging on a connected phone and approve that computer on the device. If downloads fail, inspect the network/proxy error instead of repeatedly changing code.

1.4 First project and folder map

```
flutter create first_shop
cd first_shop
flutter run -d chrome
```

`create` writes a starter project. `cd` changes the current folder. `run` builds and launches it; use an ID from `flutter devices` instead of `chrome` if needed. Save files before rebuilding. Hot reload updates many Dart changes while preserving state; a hot restart starts the Dart application again. Native plugin configuration changes may require stopping and rebuilding.

`lib/` contains application Dart code. `lib/main.dart` is the usual entry file. `test/` holds automated tests. `android/`, `ios/`, and `web/` hold platform wrappers and configuration for enabled targets. Desktop folders have a similar purpose. `pubspec.yaml` declares package metadata, dependency constraints, assets, and fonts. `pubspec.lock` records resolved package versions; commit it for applications. `.dart_tool/` and `build/` are generated working folders, not places for your own business logic.

YAML uses indentation. A misplaced space can change structure. After changing dependencies run `flutter pub get`. To add a package use `flutter pub add package_name`. Do not copy the source of a library into your app just to avoid learning dependency management.

1.5 The smallest screen and its widget tree

1. Concept. A widget describes a piece of UI. Parent and child describe the relationship between widgets. `MaterialApp` supplies application-level Material behavior. `Scaffold` supplies a standard page structure. `AppBar` is its top bar. `Text` displays text, and `Center` positions its child centrally.

2. Why it exists. Nested descriptions let us combine small reusable pieces into complete screens.

3. Why Flutter needs it. The framework reads the tree to determine layout, paint, input, and inherited configuration.

4. Analogy. A shop window contains a display stand, which contains a sign. Each object has one job and a place within a larger structure.

5. Small example. Display a welcome message in the middle of a page.

6. Code - replace lib/main.dart.

```
import 'package:flutter/material.dart';
void main() {
  runApp(const MaterialApp(
    home: Scaffold(
      appBar: null,
      body: Center(child: Text('Welcome to Shirt Studio')),
    ),
  ));
}
```

7. Every line. Line 1 imports Flutter's Material types. Line 2 is the entry function. Line 3 attaches a constant application description to the Flutter runtime. Line 4 selects a scaffold as the first screen. Line 5 explicitly omits a top bar. Line 6 centres a text child. Line 7 closes the scaffold argument. Line 8 closes both the application constructor and the call to `runApp`. Line 9 ends `main`.

8. Visual.



9. Internally. Flutter mounts a tree, receives size constraints, lays out the child, and paints its text. `BuildContext` identifies a location in this tree; it is not the screen itself.

10. Mistakes. An import does not start the app. `runApp` is not a screen navigation command. Widgets can have a single `child` or multiple `children`; use the property supported by that widget.

11. Exercise. Replace the message, add an AppBar, and explain each new parent-child relationship.

12. Project usage. Every shop screen is a composition of small descriptions.

1.6 StatelessWidget, StatefulWidget, and build

A `StatelessWidget` has no separate mutable State object of its own. It can still rebuild when its parent provides new configuration or when inherited dependencies change. "Stateless" does not mean its pixels can never change. A `StatefulWidget` has a separate State object that can survive replacement of the widget description at a compatible tree position.

The `build` method describes what the UI should look like now. Keep it quick and free of side effects. Starting a purchase, fetching data, or changing state inside every build can create duplicate work and feedback loops. Store changing screen data in State or an appropriate external controller, not in a local variable that is reset inside `build`.

`const` describes an object that can be constant. It can reduce unnecessary object creation and improve clarity, but it does not magically stop every descendant from rebuilding. `@override` asks the analyzer to check that you really are implementing an inherited member. A name beginning with _ is private to its Dart library.

Practical 1: a counter you can explain

Objective: tap a button and see the number increase. Required concepts: integers, functions, widgets, and a stateful screen. Create `counter_lab` with `flutter create counter_lab`. Replace `lib/main.dart` with the complete listing below. There are no additional packages.

flutter_examples/counter.dart | complete source listing

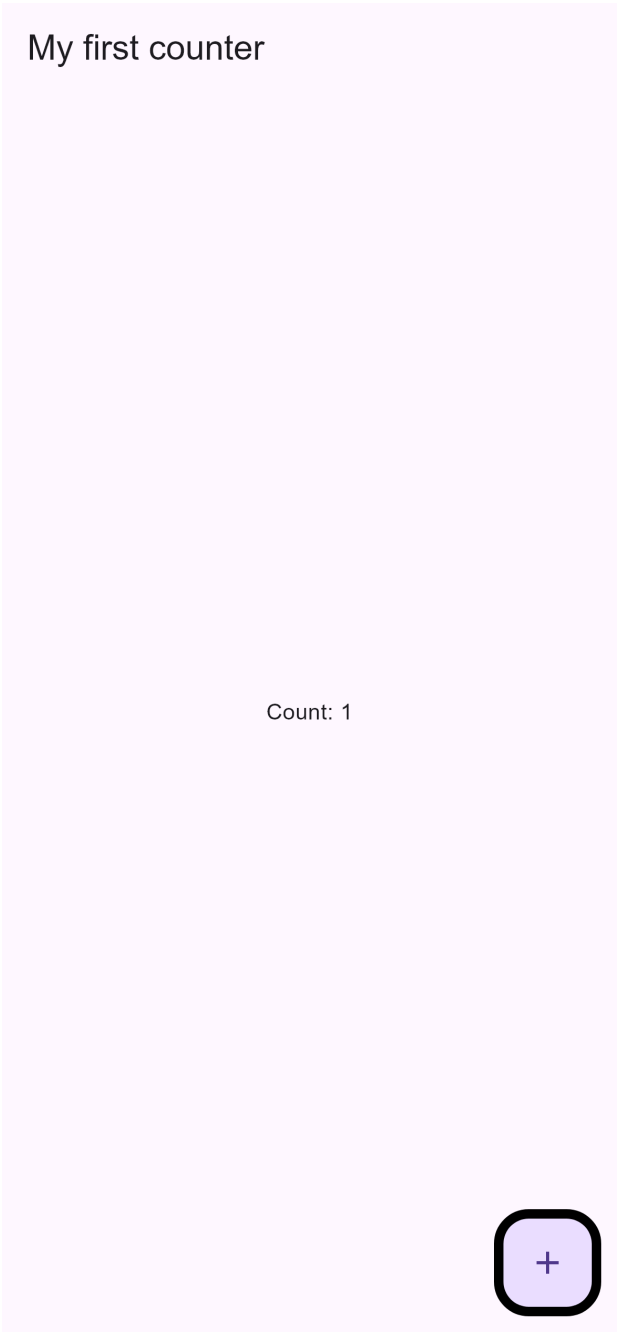
Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

counter.dart - lines 1-21

```
001 import 'package:flutter/material.dart';
002 void main() => runApp(const MaterialApp(home: Counter()));
003 class Counter extends StatefulWidget {
004   const Counter({super.key});
005   @override
006   State<Counter> createState() => _CounterState();
007 }
008 class _CounterState extends State<Counter> {
009   int count = 0;
010   @override
011   Widget build(BuildContext context) {
012     return Scaffold(
013       appBar: AppBar(title: const Text('My first counter')),
014       body: Center(child: Text('Count: $count')),
015       floatingActionButton: FloatingActionButton(
016         onPressed: () => setState(() => count++),
017         child: const Icon(Icons.add),
018       ),
019     );
020   }
021 }
```

Line-by-line explanation

1. Import Material widgets and Flutter's core UI types.
2. Start an app whose home screen is Counter.
3. Define a widget with state that can change over time.
4. Accept an optional identity key from the parent.
5. Check that the following method replaces an inherited method.
6. Create a separate state object for this widget.
7. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.
8. Keep this state implementation private to its Dart library.
9. Remember the number of taps across rebuilds.
10. Tell the Dart analyzer this declaration implements or replaces an inherited member.
11. Describe the current screen using the current count.
12. Provide the usual page structure.
13. Put a title in the top bar.
14. Centre the current count in the available body area.
15. Show a prominent action at the bottom right.
16. Increment the value and request a rebuild.
17. Draw a plus symbol inside the button.
18. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.
19. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.
20. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.
21. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.



Expected screen rendered from the Flutter example in a widget test. Fonts and platform details may differ on your device.

The picture shows the state after one tap. Before tapping, the number is zero. The callback changes `count`, and `setState` tells Flutter that this state needs rebuilding. Flutter later calls `build` again; do not imagine `setState` directly painting a digit onto the screen.

Run `flutter run`. If the number always stays zero, check that `count` is a field on the State object. If it changes in a debugger but not on screen, check the notification. Add a reset button and prevent decrements below zero. This same pattern will later manage a selected quantity on a product page.

Level 2 | Layout, widgets, and professional UI

2.1 The layout rule that prevents many bugs

1. Concept. Layout decides the size and position of each visible object. A constraint is a limit on size. Parents pass constraints down; children select a size within those limits; parents position children. A logical pixel is Flutter's layout unit, not necessarily one physical screen pixel.

2. Why it exists. Phones, windows, keyboards, and text scaling all change the space available to your UI.

3. Why Flutter needs it. A child's requested width is not permission to violate the parent's limits.

4. Analogy. A shop display must fit inside the window allocated to it. Asking for a larger shelf does not enlarge the building.

5. Small example. A Row with a long product title and a price needs a bounded area for the title.

```
import 'package:flutter/material.dart';
void main() => runApp(const MaterialApp(home: Scaffold(
  body: SafeArea(child: Padding(
    padding: EdgeInsets.all(16),
    child: Row(children: [
      Expanded(child: Text('Comfortable Oxford cotton shirt')),
      SizedBox(width: 12),
      Text('INR 799'),
    ]),
  )),
)));
```

7. Every line. Line 1 imports widgets. Line 2 starts an app and page. Line 3 avoids system intrusions and adds a padding wrapper. Line 4 specifies spacing. Line 5 starts a horizontal layout. Line 6 gives the title the remaining width after fixed children are measured. Line 7 inserts a gap. Line 8 draws the price. Line 9 closes the children and row. Line 10 closes padding and the safe area. Line 11 closes the page, app, and run call.

8. Visual. `16px margin | title uses remaining width | 12px gap | price | 16px margin.`

9. Internally. A Row arranges children along a horizontal main axis. The cross axis is vertical. Expanded participates in the Row's flex allocation. It must be in a compatible Flex ancestry, such as a Row or Column, rather than being placed arbitrarily under a ListView.

10. Mistakes. A yellow-and-black overflow warning means a layout exceeded available space. Do not hide it by clipping essential content. A Column inside a scroll view can receive unbounded height; placing Expanded there demands "remaining height" where no finite remainder exists.

11. Exercise. Narrow the window, increase system font size, and use a very long title. Keep the price visible without making the title unreadably small.

12. Project usage. Product cards, cart rows, checkout summaries, and admin tables all need explicit layout decisions.

2.2 Layout widget field guide

Row and Column. Row arranges siblings horizontally; Column arranges them vertically. `mainAxisAlignment` distributes space on the long axis; `crossAxisAlignment` controls alignment across it. They do not scroll. Choose a scrolling parent for content taller than the screen.

Expanded and Flexible. Both participate in flex allocation. Expanded forces its child to fill its allocated share. Flexible can let its child occupy less. In a Row, use Expanded around a long title while leaving a price at its natural width. Flex factors such as 2 and 1 divide available space in proportions.

Padding, SizedBox, and Container. Padding adds space around a child. SizedBox requests dimensions or represents a gap. Container combines conveniences such as padding, alignment, constraints, and decoration. A Container is not always needed: a Text inside Padding is often clearer than several nested Containers. Do not set both `color` and a decoration colour on the same Container.

Center and Align. Center is a simple centering wrapper. Align supports positions such as top-right. Their resulting size still depends on parent constraints. Use alignment for positioning, not a pile of magic-number margins.

Stack and Positioned. Stack layers children on top of each other. Positioned locates a child within that Stack. A sale badge over a product image is a good use. A normal form with vertically arranged fields is usually better expressed as a Column or ListView.

Wrap. Wrap moves children to another run when space is insufficient. Use it for size chips, filters, or actions that should wrap at small widths. It eagerly lays out its children, so a huge data list belongs in a lazy builder instead.

ListView and GridView. ListView scrolls rows; GridView scrolls tiles. Their builder constructors create children as needed. Supply `itemCount`, stable item identities where needed, and a sensible empty state. Avoid a large shrink-wrapped nested list: it can force measurement of many children and weaken lazy rendering.

SingleChildScrollView. This scrolls one child, often a small Column such as a settings form. It is not an efficient substitute for ListView.builder with thousands of records. In the small shopping demo, a shrink-wrapped grid is acceptable because it contains exactly two products; a production catalogue should use a single lazy scroll structure.

2.3 Inputs, choices, and feedback

TextField edits text. A TextEditingController reads and changes its contents. **TextFormField** adds integration with Form validation. Labels describe fields; hints are examples and should not replace labels. A password field can obscure text, but this affects display, not transport security. Set a suitable keyboard type, and still validate because pasted input can be anything.

Buttons represent actions. FilledButton suits a primary action; OutlinedButton or TextButton can represent lower emphasis. IconButton needs an understandable tooltip or semantic label when the icon alone is ambiguous. Passing `null` as the callback disables a button. Disable duplicate submissions while work is in flight, and show progress.

Checkbox represents an independent yes/no choice. **Radio** represents one selection from a group. **Switch** usually changes an on/off setting immediately. Current Flutter APIs include RadioGroup for

coordinating radio selection; use the API supported by your SDK rather than copying obsolete group parameters blindly. **Dropdown** controls choose one value from a compact menu. **Slider** chooses an approximate value on a range; it is poor for exact quantities that users need to type.

Card groups related content visually. **Dialog** interrupts for a focused decision, such as confirming a destructive action. **BottomSheet** presents a temporary panel, for example size selection. **SnackBar** gives brief feedback without replacing persistent error information. Do not show "Payment successful" in a snackbar as your only order record.

Drawer contains secondary navigation in a side panel. **NavigationBar** switches among a small set of primary destinations. Switching destinations and pushing a detail route solve different problems. Decide which screen state should survive a tab switch.

Complete widget workshop

This runnable workshop combines a validated form, radio group, dropdown, checkbox, switch, slider, chips, a card, dialog, bottom sheet, drawer, and bottom navigation. It also demonstrates Stack, Positioned, a gradient, rounded corners, spacing, and scrolling. Create a Flutter project and use the file as lib/main.dart, or save it under lib/examples/widget_gallery.dart and run that target.

flutter_examples/widget_gallery.dart | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

widget_gallery.dart - lines 1-24

```
001 import 'package:flutter/material.dart';
002 void main() => runApp(MaterialApp(theme: ThemeData(colorSchemeSeed: Colors.indigo), home:
    const Gallery()));
003 class Gallery extends StatefulWidget {
004   const Gallery({super.key});
005   @override
006   State<Gallery> createState() => _GalleryState();
007 }
008 class _GalleryState extends State<Gallery> {
009   final form = GlobalKey<FormState>();
010   final name = TextEditingController();
011   String size = 'M', colour = 'Blue';
012   bool gift = false, updates = true;
013   double budget = 1000;
014   int tab = 0;
015   @override
016   void dispose() { name.dispose(); super.dispose(); }
017   Future<void> confirm() async {
018     if (!form.currentState!.validate()) return;
019     FocusScope.of(context).unfocus();
020     final accepted = await showDialog<bool>(context: context, builder: (dialogContext) =>
    AlertDialog(
021       title: const Text('Save preferences?'), content: Text('${name.text.trim()} / $size /
    $colour'),
022       actions: [TextButton(onPressed: () => Navigator.pop(dialogContext, false), child:
    const Text('Cancel')),
023         FilledButton(onPressed: () => Navigator.pop(dialogContext, true), child: const
    Text('Save'))]));
024     if (!mounted || accepted != true) return;
```

Line-by-line explanation

1. Import the named library or module so the following code can use its types and functions.

2. Declare the method and its return type. Its parameters describe the information supplied by the caller.

3. Declare class Gallery extends StatefulWidget and begin the fields or methods that define this type.

4. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

5. Tell the Dart analyzer this declaration implements or replaces an inherited member.

6. Declare the method and its return type. Its parameters describe the information supplied by the caller.

7. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

8. Declare class _GalleryState extends State<Gallery> and begin the fields or methods that define this type.

9. Keep one identity for the form across rebuilds.

10. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

11. Declare a typed field or value and initialize it from the expression on the right.

12. Declare a typed field or value and initialize it from the expression on the right.

13. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

14. Declare a typed field or value and initialize it from the expression on the right.

15. Tell the Dart analyzer this declaration implements or replaces an inherited member.

16. Declare the method and its return type. Its parameters describe the information supplied by the caller.

17. Declare the method and its return type. Its parameters describe the information supplied by the caller.

18. The key belongs to the visible form when this button can run.

19. Dismiss the keyboard before showing a decision dialog.

20. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

21. Supply this named property, field, or configuration entry with the value shown here.

22. Supply this named property, field, or configuration entry with the value shown here.

23. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

24. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

widget_gallery.dart - lines 25-48

```
025     ScaffoldMessenger.of(context).showSnackBar(const SnackBar(content: Text('Preferences
    accepted locally')));
026   }
027   @override
028   Widget build(BuildContext context) => Scaffold(
029     appBar: AppBar(title: const Text('Widget workshop')),
030     drawer: Drawer(child: SafeArea(child: ListView(children: [
031       const DrawerHeader(child: Text('Shirt Studio')),
032       ListTile(title: const Text('Close menu'), onTap: () => Navigator.pop(context)),
033     ]))),
034     bottomNavigationBar: NavigationBar(selectedIndex: tab, onDestinationSelected: (value)
    => setState(() => tab = value),
035       destinations: const [NavigationDestination(icon: Icon(Icons.tune), label:
    'Preferences'),
036         NavigationDestination(icon: Icon(Icons.person), label: 'Profile')]),
037     body: tab == 1 ? const Center(child: Text('Profile destination')) : Form(key: form,
    child: ListView(
038       padding: const EdgeInsets.all(20), children: [
039         Stack(children: [
040           Container(height: 120, width: double.infinity, alignment: Alignment.center,
041             decoration: BoxDecoration(borderRadius: BorderRadius.circular(16),
042               gradient: const LinearGradient(colors: [Colors.indigo, Colors.teal])),
043             child: const Icon(Icons.checkroom, color: Colors.white, size: 70)),
044           const Positioned(top: 10, right: 10, child: Chip(label: Text('Preview'))),
045         ]),
046         const SizedBox(height: 16),
047         TextFormField(controller: name, decoration: const InputDecoration(labelText: 'Your
    name', border: OutlineInputBorder()),
048           validator: (value) => value == null || value.trim().isEmpty ? 'Enter your name' :
     null),
```

Line-by-line explanation

25. Feedback only; no backend save is claimed.

26. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

27. Tell the Dart analyzer this declaration implements or replaces an inherited member.

28. Declare the method and its return type. Its parameters describe the information supplied by the caller.

29. Supply this named property, field, or configuration entry with the value shown here.

30. Supply this named property, field, or configuration entry with the value shown here.

31. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

32. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

33. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

34. Supply this named property, field, or configuration entry with the value shown here.

35. Supply this named property, field, or configuration entry with the value shown here.

36. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

37. Supply this named property, field, or configuration entry with the value shown here.

38. Supply this named property, field, or configuration entry with the value shown here.

39. Layer a badge over a decorated preview.

40. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

41. Supply this named property, field, or configuration entry with the value shown here.

42. Supply this named property, field, or configuration entry with the value shown here.

43. Supply this named property, field, or configuration entry with the value shown here.

44. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

45. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

46. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

47. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

48. Supply this named property, field, or configuration entry with the value shown here.

widget_gallery.dart - lines 49-65

```
049         RadioGroup<String>(groupValue: size, onChanged: (value) { if (value != null)
    setState(() => size = value); },
050           child: const Row(children: [Radio<String>(value: 'M'), Text('M'),
    Radio<String>(value: 'L'), Text('L')])),
051         DropdownButton<String>(value: colour, isExpanded: true,
052           items: ['Blue', 'White'].map((value) => DropdownMenuItem(value: value, child:
    Text(value))).toList(),
053           onChanged: (value) { if (value != null) setState(() => colour = value); }),
054         CheckboxListTile(title: const Text('Gift wrap'), value: gift, onChanged: (value) =>
     setState(() => gift = value ?? false)),
055         SwitchListTile(title: const Text('Product updates'), value: updates, onChanged:
    (value) => setState(() => updates = value)),
056         Text('Budget: INR ${budget.round()}'),
057         Slider(min: 500, max: 2000, divisions: 15, value: budget, label:
    '${budget.round()}', onChanged: (value) => setState(() => budget = value)),
058         Wrap(spacing: 8, children: const [Chip(label: Text('Cotton')), Chip(label:
    Text('Regular fit')), Chip(label: Text('Easy care'))]),
059         Card(child: Padding(padding: const EdgeInsets.all(12), child: Text('Selected: $size
     / $colour'))),
060         FilledButton(onPressed: confirm, child: const Text('Validate and confirm')),
061         OutlinedButton(onPressed: () => showModalBottomSheet<void>(context: context,
    builder: (_) => const SafeArea(
062           child: Padding(padding: EdgeInsets.all(24), child: Text('Size guide: measure a
    shirt you already own.')))), child: const Text('Open size guide')),
063       ])),
064   );
065 }
```

Line-by-line explanation

49. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

50. Supply this named property, field, or configuration entry with the value shown here.

51. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

52. Supply this named property, field, or configuration entry with the value shown here.

53. Supply this named property, field, or configuration entry with the value shown here.

54. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

55. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

56. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

57. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

58. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

59. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

60. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

61. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

62. Supply this named property, field, or configuration entry with the value shown here.

63. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

64. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

65. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

Leave the name empty and tap Validate: expect an inline error. Enter Asha, choose L and White, enable gift wrap, and move the slider. Tap Validate again: expect a confirmation dialog. Save closes the dialog and shows a snackbar. Open the size guide to see a bottom sheet. Switch to Profile using bottom navigation and back; the selected values remain because their State still owns them. Nothing is saved to a server in this workshop.

2.4 Colour, typography, images, and themes

A theme is shared visual configuration. Set a ColorScheme and text styles once, then use `Theme.of(context)` to read them. This avoids slightly different buttons and headings on every screen. A font is a family of letter shapes; font size, weight, height, and spacing affect readability. Do not encode every heading as an unrelated hard-coded number.

Borders indicate boundaries; border radius rounds corners; shadows suggest elevation. A gradient blends colours. Use these to clarify hierarchy, not to hide a weak layout. Dark mode needs a complete colour scheme, not merely a black background behind dark text. Test text contrast and disabled states.

`Image.asset` loads a bundled file declared under `flutter: assets:` in pubspec.yaml. `Image.network` requests a URL. Supply a known area, fitting behavior, loading feedback, and an error fallback. Large source pictures consume memory even when displayed small; request appropriately sized images. Never rely on a remote image to explain the only way to use a button.

Accessibility means people with different abilities can use the app. Test large text, screen-reader labels, keyboard focus order, sufficient target sizes, and meaningful error messages. Colour alone must not

distinguish success from failure. For a shirt colour swatch, include the colour name as text or semantics.

2.5 Responsive shopping UI project

Responsive layout changes arrangement according to available space. Adaptive behavior also respects platform conventions, input devices, and expectations. Use LayoutBuilder for local constraints and MediaQuery for relevant environment information. Do not classify every width above one arbitrary number as a perfect desktop design.

Objective: create a catalogue, detail route, and cart from local data. Prerequisites: collections, objects, layout, basic state, and callbacks. Create `local_shop` with `flutter create local_shop`, then replace `lib/main.dart`. The program includes its own Product model and data. There is no backend and no real payment.

flutter_examples/local_shop.dart | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

local_shop.dart - lines 1-24

```
001 import 'package:flutter/material.dart';
002 void main() => runApp(MaterialApp(theme: ThemeData(colorSchemeSeed: Colors.teal), home:
    const Shop()));
003 class Product {
004   final int id;
005   final String name;
006   final int price;
007   const Product(this.id, this.name, this.price);
008 }
009 const products = [Product(1, 'Oxford shirt', 79900), Product(2, 'Linen shirt', 99900)];
010 String money(int value) => 'INR ${(value / 100).toStringAsFixed(2)}';
011 class Shop extends StatefulWidget {
012   const Shop({super.key});
013   @override
014   State<Shop> createState() => _ShopState();
015 }
016 class _ShopState extends State<Shop> {
017   final cart = <int, int>{};
018   Future<void> details(Product product) async {
019     final add = await Navigator.of(context).push<bool>(MaterialPageRoute(builder: (_) =>
    Scaffold(
020       appBar: AppBar(title: Text(product.name)),
021       body: Center(child: Column(mainAxisSize: MainAxisSize.min, children: [
022         const Icon(Icons.checkroom, size: 100),
023         Text(money(product.price)),
024         FilledButton(onPressed: () => Navigator.pop(context, true), child: const Text('Add
    to cart')),
```

Line-by-line explanation

1. Import the Material UI toolkit.

2. Give the shop a coherent colour theme.

3. Keep each product's values together.

4. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

5. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

6. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

7. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

8. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

9. Local catalogue; prices are paise.

10. Format a price for display.

11. Declare class Shop extends StatefulWidget and begin the fields or methods that define this type.

12. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

13. Tell the Dart analyzer this declaration implements or replaces an inherited member.

14. Declare the method and its return type. Its parameters describe the information supplied by the caller.

15. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

16. Declare class _ShopState extends State<Shop> and begin the fields or methods that define this type.

17. Map product IDs to quantities.

18. Open a screen and wait for its answer.

19. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

20. Supply this named property, field, or configuration entry with the value shown here.

21. Supply this named property, field, or configuration entry with the value shown here.

22. Use an offline icon so the example needs no image download.

23. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

24. Return true to the caller.

local_shop.dart - lines 25-48

```
025       ])),
026     )));
027     if (mounted && add == true) setState(() => cart.update(product.id, (n) => n + 1,
    ifAbsent: () => 1));
028   }
029   int get total => products.fold(0, (sum, p) => sum + p.price * (cart[p.id] ?? 0));
030   @override
031   Widget build(BuildContext context) => Scaffold(
032     appBar: AppBar(title: const Text('Shirt Studio')),
033     body: LayoutBuilder(builder: (context, constraints) {
034       return ListView(padding: const EdgeInsets.all(16), children: [
035         const Text('Find your everyday shirt', style: TextStyle(fontSize: 24)),
036         GridView.builder(shrinkWrap: true, physics: const NeverScrollableScrollPhysics(),
037           gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
038             crossAxisCount: constraints.maxWidth >= 700 ? 4 : 2, childAspectRatio: 0.85),
039           itemCount: products.length, itemBuilder: (context, i) {
040             final p = products[i];
041             return Card(child: InkWell(onTap: () => details(p), child: Padding(
042               padding: const EdgeInsets.all(12), child: Column(mainAxisAlignment:
    MainAxisAlignment.center, children: [
043                 const Icon(Icons.checkroom, size: 60), Text(p.name), Text(money(p.price)),
044               ])));
045           }),
046         const Text('Your cart', style: TextStyle(fontSize: 22)),
047         if (cart.isEmpty) const Text('Your cart is empty.'),
048         for (final p in products.where((p) => cart.containsKey(p.id)))
```

Line-by-line explanation

25. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

26. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

27. Update only after a confirmed add.

28. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

29. Sum all line totals.

30. Tell the Dart analyzer this declaration implements or replaces an inherited member.

31. Declare the method and its return type. Its parameters describe the information supplied by the caller.

32. Supply this named property, field, or configuration entry with the value shown here.

33. Read the available width, not a hard-coded phone size.

34. Produce the stated result and finish this function call. In a build method the result is a widget description.

35. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

36. Let the outer list own scrolling in this small catalogue.

37. Supply this named property, field, or configuration entry with the value shown here.

38. Use more columns when there is room.

39. Supply this named property, field, or configuration entry with the value shown here.

40. Give the current product a short local name.

41. Produce the stated result and finish this function call. In a build method the result is a widget description.

42. Supply this named property, field, or configuration entry with the value shown here.

43. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

44. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

45. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

46. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

47. Do not show a meaningless blank area.

48. Show only products that are in the cart.

local_shop.dart - lines 49-59

```
049           ListTile(title: Text('${p.name} x${cart[p.id]}'), subtitle: Text(money(p.price *
    cart[p.id]!)),
050             trailing: IconButton(icon: const Icon(Icons.remove_circle_outline),
051               onPressed: () => setState(() {
052                 final n = cart[p.id]!;
053                 if (n == 1) { cart.remove(p.id); } else { cart[p.id] = n - 1; }
054               })),
055         Text('Total ${money(total)}'),
056       ]);
057     }),
058   );
059 }
```

Line-by-line explanation

49. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

50. Supply this named property, field, or configuration entry with the value shown here.

51. Decrease quantity or remove the last item.

52. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

53. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

54. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

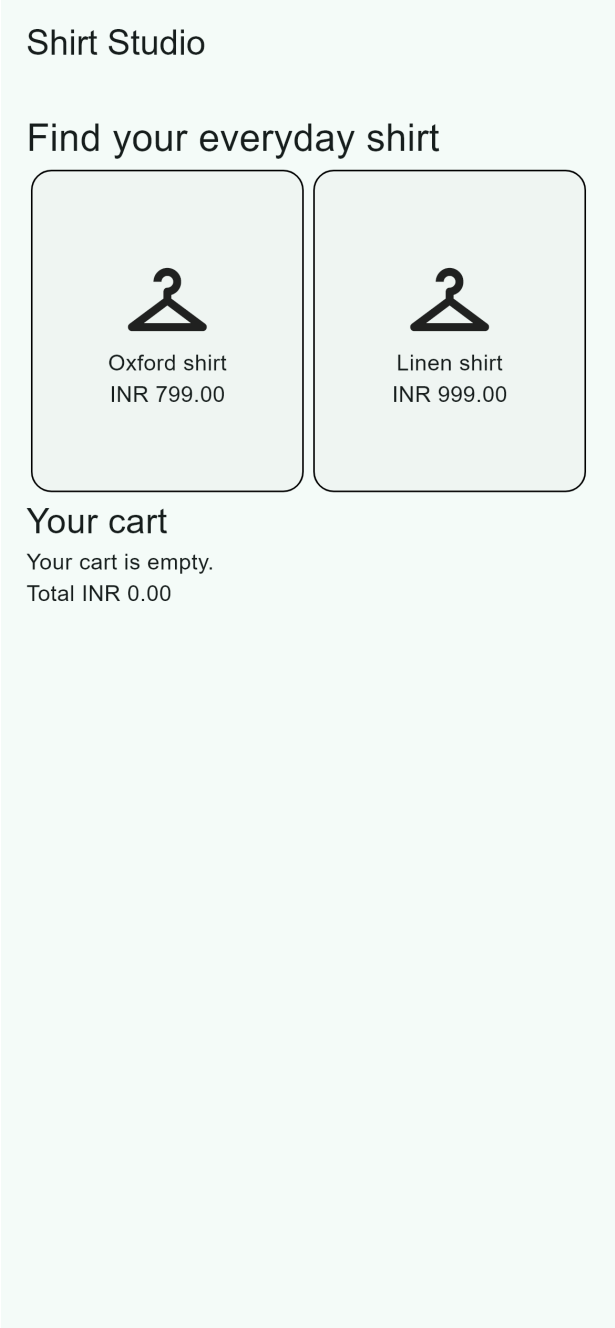
55. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

56. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

57. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

58. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

59. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.



Expected screen rendered from the Flutter example in a widget test. Fonts and platform details may differ on your device.

Tap Oxford, then Add to cart. The detail screen returns `true`; the home screen increments the quantity. Add it twice and expect `Oxford shirt x2` and total `INR 1598.00`. Remove once and expect quantity one. Remove again and expect the empty-cart message. A hot restart empties the cart because this lesson keeps data

only in memory.

The ! in the cart rows is justified by the filter that includes only IDs present in the map and by synchronous access on the same isolate. This is a narrow invariant, not a habit to copy into arbitrary JSON parsing. The catalogue grid deliberately uses local icons; replace them with declared assets as an exercise.

Modify the UI to add a profile route and a size selector. The current model has no variants, so do not pretend a selected size affects inventory until you introduce a variant identifier. Debug overflow with the Flutter Inspector and smaller window widths. Explain the difference between how the shop looks and what its data model can actually represent.

Level 3 | Application fundamentals

3.1 State, lifecycle, and ownership

1. Concept. State is information that can change and affect behavior or display. Local state belongs to one screen or component. Shared state is needed by several parts of the application. Ownership means deciding which object is responsible for storing and changing a value.

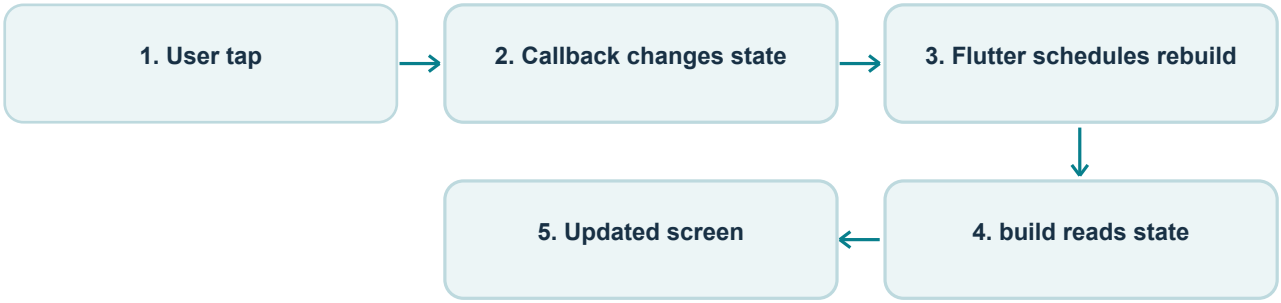
2. Why it exists. A screen must remember text, selections, loading results, and previous actions. Without clear ownership, two screens can disagree about the cart.

3. Flutter purpose. State triggers new UI descriptions. Keep one authoritative value and derive related values where practical, such as deriving total from cart lines instead of storing an independently editable total.

4. Analogy. One stock ledger is authoritative; several display boards read from it. If every board maintains an independent stock number, they eventually disagree.

5-7. Example and code. Return to the complete counter listing. Its `count` field is the authoritative value; its Text is a display derived from that value. Every line is explained below that listing. Change the count once and let the UI follow it.

8. Visual.



9. Internally. `initState` runs when the State object is inserted. Use it for one-time setup. `didChangeDependencies` can respond to inherited dependencies. `didUpdateWidget` responds to a replacement widget configuration at the same State. `dispose` releases resources when the State is removed permanently. After `await`, check `mounted` before updating a State or using its context for navigation.

10. Mistakes. Never make the callback passed to `setState` asynchronous. Perform awaited work outside it, then update fields synchronously. Cancel subscriptions and dispose controllers. Hot reload preserving state can explain why a changed initial value does not appear until restart.

11. Exercise. Add a loading flag to a button that waits one second. Disable the button during the wait and handle the screen being closed.

12. Project usage. Product filters may be local; cart and authentication usually need broader ownership. Persisted server state and temporary UI state are different categories.

3.2 Navigation, routes, and data

A route is one navigable screen or location. Navigator manages a stack: push adds a route; pop removes the top route and can return a result. MaterialPageRoute supplies common page transition behavior. In the local shop, the Product is passed through a constructor closure, and `Navigator.pop(context, true)` returns a Boolean to the awaiting caller.

Named routes associate strings with route builders. They are useful to recognize in older code, but Flutter documentation recommends more capable routing approaches for advanced navigation, deep links, and web URLs [S3]. A deep link opens a specific app location from outside the app. A router must parse it, validate its parameters, check authentication, and handle a missing item.

Do not pass secrets in URL query strings. For a product page, pass a stable product ID and fetch or locate the data. Pressing Back should behave predictably; a successful login should not expose an obsolete login form behind the account screen. A client route guard improves UX but never replaces backend authorization.

3.3 Forms, controllers, validation, focus, and keyboards

A Form groups fields. A GlobalKey<FormState> lets the owning State request validation. A validator returns an error message for invalid input and null for valid input. Keep the key and controllers outside `build`, dispose controllers, and display errors near their fields.

Validate two different things: shape and business meaning. `int.tryParse` checks whether input represents an integer. A rule such as "quantity must be between one and available stock" checks meaning. Client validation gives quick feedback; the server must repeat authoritative validation because clients can be modified.

Focus identifies the control receiving keyboard input. A FocusNode can coordinate movement from one field to the next, but it too needs disposal. A scrollable form avoids keyboard overflow. On submission, dismiss focus only when that helps the flow, preserve entered values if the server rejects the request, and show a retry action.

Practical 2: calculator

Create `calculator_lab`, replace `lib/main.dart`, and run it. This project reads user input, rejects invalid numbers, implements four operations, and cleans up two controllers.

flutter_examples/calculator.dart | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

calculator.dart - lines 1-24

```
001 import 'package:flutter/material.dart';
002 void main() => runApp(const MaterialApp(home: Calculator()));
003 class Calculator extends StatefulWidget {
004   const Calculator({super.key});
005   @override
006   State<Calculator> createState() => _CalculatorState();
007 }
008 class _CalculatorState extends State<Calculator> {
009   final a = TextEditingController();
010   final b = TextEditingController();
011   String result = 'Enter two numbers';
012   void calculate(String operation) {
013     final x = double.tryParse(a.text);
014     final y = double.tryParse(b.text);
015     if (x == null || y == null || !x.isFinite || !y.isFinite) {
016       setState(() => result = 'Enter valid finite numbers');
017       return;
018     }
019     if (operation == '/' && y == 0) {
020       setState(() => result = 'Cannot divide by zero');
021       return;
022     }
023     final value = switch (operation) {
024       '+' => x + y,
```

Line-by-line explanation

1. Make the Material widget library available.

2. Launch the calculator screen.

3. The screen owns two input values and a result.

4. Standard widget constructor with an optional key.

5. Tell the Dart analyzer this declaration implements or replaces an inherited member.

6. Attach the mutable state.

7. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

8. Declare class _CalculatorState extends State<Calculator> and begin the fields or methods that define this type.

9. Read and control the first text field.

10. Read and control the second text field.

11. Initial result text.

12. One handler implements the selected operation.

13. Convert the first input or receive null.

14. Convert the second input or receive null.

15. Reject missing, invalid, and infinite inputs.

16. Show validation feedback.

17. Stop before using invalid inputs.

18. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

19. Handle the undefined division case explicitly.

20. Explain the problem on screen.

21. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

22. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

23. Select the expression matching the operator.

24. Add the values.

calculator.dart - lines 25-48

```
025       '-' => x - y,
026       '*' => x * y,
027       _ => x / y,
028     };
029     setState(() => result = '$value');
030   }
031   @override
032   void dispose() {
033     a.dispose();
034     b.dispose();
035     super.dispose();
036   }
037   @override
038   Widget build(BuildContext context) => Scaffold(
039     appBar: AppBar(title: const Text('Calculator')),
040     body: Padding(
041       padding: const EdgeInsets.all(24),
042       child: ListView(children: [
043         TextField(controller: a, decoration: const InputDecoration(labelText: 'First
    number')),
044         TextField(controller: b, decoration: const InputDecoration(labelText: 'Second
    number')),
045         Wrap(spacing: 12, children: [
046           for (final op in ['+', '-', '*', '/'])
047             FilledButton(onPressed: () => calculate(op), child: Text(op)),
048         ]),
```

Line-by-line explanation

25. Subtract the second value.

26. Multiply the values.

27. The remaining supported operation is division.

28. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

29. Convert the answer into display text.

30. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

31. Tell the Dart analyzer this declaration implements or replaces an inherited member.

32. Release resources when this state leaves the tree permanently.

33. Stop the first controller's listeners.

34. Stop the second controller's listeners.

35. Finish the framework cleanup.

36. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

37. Tell the Dart analyzer this declaration implements or replaces an inherited member.

38. Return the full screen for the current state.

39. Identify the screen.

40. Keep controls away from screen edges.

41. Use 24 logical pixels on every side.

42. Allow scrolling when a keyboard reduces the viewport.

43. First input.

44. Second input.

45. Move buttons to another line if necessary.

46. Create one button for each operation.

47. Capture this operator in the callback.

48. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

calculator.dart - lines 49-53

```
049         Text(result, style: Theme.of(context).textTheme.headlineSmall),
050       ]),
051     ),
052   );
053 }
```

Line-by-line explanation

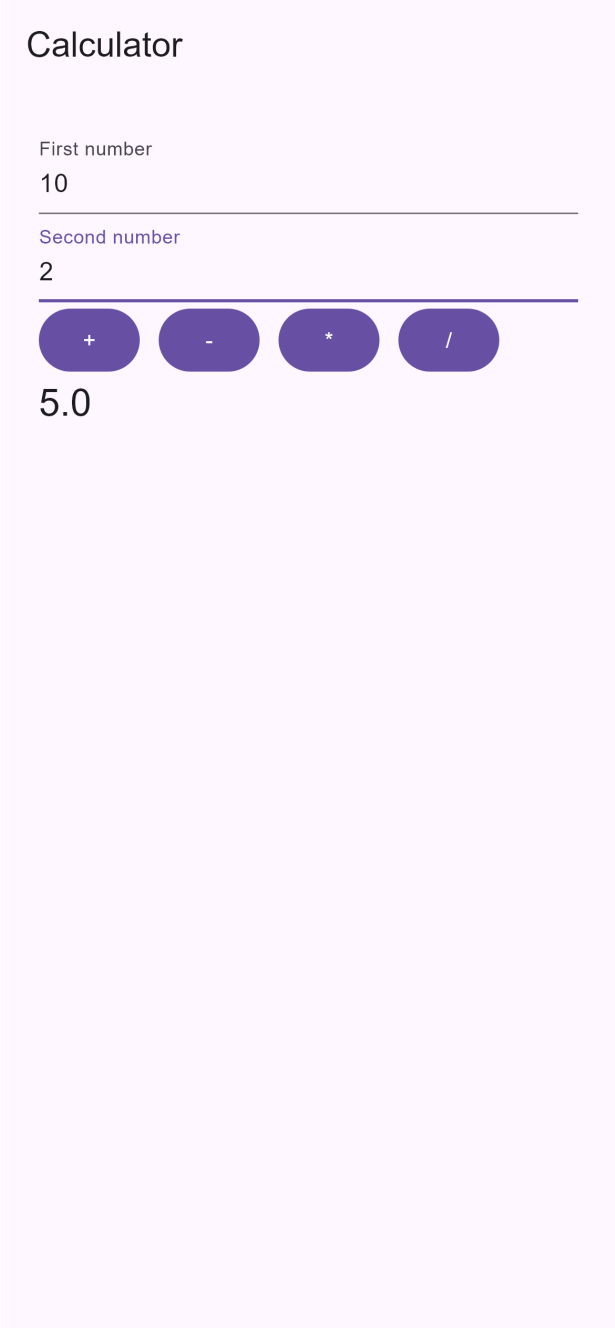
49. Display the latest result.

50. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

51. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

52. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

53. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.



Expected screen rendered from the Flutter example in a widget test. Fonts and platform details may differ on your device.

Tap division after entering 10 and 2 to match the illustration. Enter zero in the second field to see a meaningful error. Modify the example to add a clear button. This is a general numeric calculator; do not reuse floating-point arithmetic for authoritative order money.

Practical 3: to-do application

Create `todos_lab`, replace `lib/main.dart`, and run it. Add `Learn widgets`, then `Build cart`; delete the first row. Expect one remaining row. This lesson intentionally stores tasks only in memory.

flutter_examples/todos.dart | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

todos.dart - lines 1-24

```
001 import 'package:flutter/material.dart';
002 void main() => runApp(const MaterialApp(home: Todos()));
003 class Todos extends StatefulWidget {
004   const Todos({super.key});
005   @override
006   State<Todos> createState() => _TodosState();
007 }
008 class _TodosState extends State<Todos> {
009   final input = TextEditingController();
010   final tasks = <String>[];
011   void add() {
012     final text = input.text.trim();
013     if (text.isEmpty) return;
014     setState(() => tasks.add(text));
015     input.clear();
016   }
017   @override
018   void dispose() {
019     input.dispose();
020     super.dispose();
021   }
022   @override
023   Widget build(BuildContext context) => Scaffold(
024     appBar: AppBar(title: const Text('Tasks')),
```

Line-by-line explanation

1. Import UI components.

2. Open the task list.

3. Declare class Todos extends StatefulWidget and begin the fields or methods that define this type.

4. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

5. Tell the Dart analyzer this declaration implements or replaces an inherited member.

6. Declare the method and its return type. Its parameters describe the information supplied by the caller.

7. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

8. Declare class _TodosState extends State<Todos> and begin the fields or methods that define this type.

9. Own the task-entry field's text.

10. Store tasks for this running session.

11. Validate and append a task.

12. Remove spaces at both ends.

13. Ignore blank tasks.

14. Add the task and rebuild the list.

15. Prepare the field for the next entry.

16. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

17. Tell the Dart analyzer this declaration implements or replaces an inherited member.

18. Declare the method and its return type. Its parameters describe the information supplied by the caller.

19. Clean up the controller.

20. Release the resource when its owner no longer needs it.

21. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

22. Tell the Dart analyzer this declaration implements or replaces an inherited member.

23. Declare the method and its return type. Its parameters describe the information supplied by the caller.

24. Supply this named property, field, or configuration entry with the value shown here.

todos.dart - lines 25-40

```
025     body: Column(children: [
026       Padding(padding: const EdgeInsets.all(16), child: Row(children: [
027         Expanded(child: TextField(controller: input, onSubmitted: (_) => add(),
028           decoration: const InputDecoration(labelText: 'New task'))),
029         IconButton(onPressed: add, icon: const Icon(Icons.add)),
030       ])),
031       Expanded(child: tasks.isEmpty
032         ? const Center(child: Text('No tasks yet'))
033         : ListView.builder(itemCount: tasks.length, itemBuilder: (context, index) {
034             return ListTile(title: Text(tasks[index]),
035               trailing: IconButton(icon: const Icon(Icons.delete_outline),
036                 onPressed: () => setState(() => tasks.removeAt(index))));
037           })),
038     ]),
039   );
040 }
```

Line-by-line explanation

25. Put the input above the scrolling area.

26. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

27. Let the keyboard submit too.

28. Supply this named property, field, or configuration entry with the value shown here.

29. A second way to submit.

30. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

31. Give the remaining height to either the empty state or the list.

32. Explain why the list has no rows.

33. Build visible rows on demand.

34. Display the task at this position.

35. Supply this named property, field, or configuration entry with the value shown here.

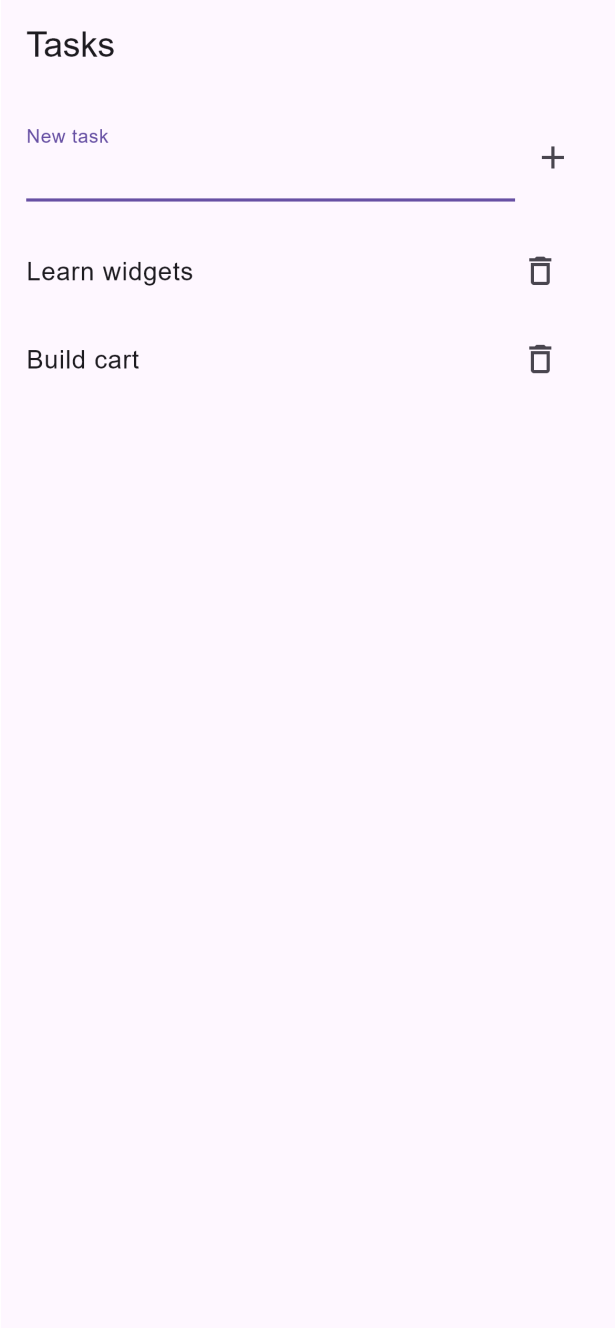
36. Remove the selected task.

37. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

38. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

39. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

40. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.



Expected screen rendered from the Flutter example in a widget test. Fonts and platform details may differ on your device.

The outer Column gives the input a natural height and Expanded gives the list a bounded remainder. That is why the ListView can scroll correctly. Add completion state using a typed Task object with an ID instead of maintaining a separate parallel list of Boolean values. Stable IDs become important when rows hold their own state or are reordered.

3.4 Futures, async, await, and FutureBuilder

Concept. A Future represents a value or error that will become available later. `async` lets a function await such work. `await` pauses that function's continuation; it does not freeze the whole interface while waiting for I/O. It also does not move expensive CPU calculations to another thread automatically.

Why. Network and disk operations take unpredictable time. The UI needs loading, success, empty, and error states. A spinner alone is not a complete design.

Analogy. You order a meal, receive a ticket, and do other things until it is ready. The ticket is a promise of a result, not the meal itself.

```
Future<String> loadName() async {
  await Future<void>.delayed(const Duration(milliseconds: 100));
  return 'Oxford shirt';
}
Future<void> main() async {
  print('Loading');
  final name = await loadName();
  print(name);
}
```

Line 1 declares a future text result. Line 2 simulates a delay without blocking the event loop. Line 3 completes with the name. Line 4 ends the loader. Line 5 starts an asynchronous entry point. Line 6 prints immediately. Line 7 waits for the result. Line 8 prints it, and line 9 ends execution. Expect `Loading`, then `Oxford shirt` after a short pause.

FutureBuilder can render according to a Future's connection state, data, or error. Create a fetch future in initialization or when its inputs change, rather than creating a fresh fetch on every build. Otherwise incidental rebuilds can repeat requests. A refresh action should deliberately replace the future. For large CPU-bound parsing, investigate isolates after measuring the problem; an isolate has separate memory and exchanges messages.

Exercise: make `loadName` throw a StateError and catch it in `main`. Then implement a retry button in a small Flutter screen. In the shop, this pattern underlies catalogue loading, order submission, and saving addresses.

Complete FutureBuilder experiment

Run this independent file in a Flutter project. First expect a loading spinner, then Oxford and Linen. Turn the failure switch on and reload: expect a readable error. Turn it off and reload: expect recovery. A switch change alone rebuilds the UI but does not create a new Future, which is the central lesson.

flutter_examples/future_builder.dart | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

future_builder.dart - lines 1-24

```
001 import 'package:flutter/material.dart';
002 void main() => runApp(const MaterialApp(home: FutureCatalogue()));
003 class FutureCatalogue extends StatefulWidget {
004   const FutureCatalogue({super.key});
005   @override
006   State<FutureCatalogue> createState() => _FutureCatalogueState();
007 }
008 class _FutureCatalogueState extends State<FutureCatalogue> {
009   late Future<List<String>> future;
010   bool failNext = false;
011   Future<List<String>> load() async {
012     final shouldFail = failNext;
013     await Future<void>.delayed(const Duration(milliseconds: 700));
014     if (shouldFail) throw StateError('Simulated connection failure');
015     return ['Oxford', 'Linen'];
016   }
017   @override
018   void initState() { super.initState(); future = load(); }
019   void refresh() => setState(() => future = load());
020   @override
021   Widget build(BuildContext context) => Scaffold(
022     appBar: AppBar(title: const Text('Async catalogue')),
023     body: Column(children: [
024       SwitchListTile(title: const Text('Fail the next request'), value: failNext,
    onChanged: (value) => setState(() => failNext = value)),
```

Line-by-line explanation

1. Import the named library or module so the following code can use its types and functions.

2. Declare the method and its return type. Its parameters describe the information supplied by the caller.

3. Declare class FutureCatalogue extends StatefulWidget and begin the fields or methods that define this type.

4. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

5. Tell the Dart analyzer this declaration implements or replaces an inherited member.

6. Declare the method and its return type. Its parameters describe the information supplied by the caller.

7. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

8. Declare class _FutureCatalogueState extends State<FutureCatalogue> and begin the fields or methods that define this type.

9. Store one future rather than starting work in every build.

10. A deliberate failure switch makes the error state easy to learn.

11. Declare the method and its return type. Its parameters describe the information supplied by the caller.

12. Snapshot the choice for this request.

13. Wait for this asynchronous operation before continuing the current function; other event-loop work can still run.

14. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

15. Produce the stated result and finish this function call. In a build method the result is a widget description.

16. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

17. Tell the Dart analyzer this declaration implements or replaces an inherited member.

18. Declare the method and its return type. Its parameters describe the information supplied by the caller.

19. Start a new request only after an explicit action.

20. Tell the Dart analyzer this declaration implements or replaces an inherited member.

21. Declare the method and its return type. Its parameters describe the information supplied by the caller.

22. Supply this named property, field, or configuration entry with the value shown here.

23. Supply this named property, field, or configuration entry with the value shown here.

24. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

future_builder.dart - lines 25-35

```
025       FilledButton(onPressed: refresh, child: const Text('Reload')),
026       Expanded(child: FutureBuilder<List<String>>(future: future, builder: (context,
    snapshot) {
027         if (snapshot.connectionState != ConnectionState.done) return const Center(child:
    CircularProgressIndicator());
028         if (snapshot.hasError) return const Center(child: Text('Could not load. Turn
    failure off and reload.'));
029         final values = snapshot.data ?? [];
030         if (values.isEmpty) return const Center(child: Text('No products'));
031         return ListView(children: [for (final name in values) ListTile(title:
    Text(name))]);
032       })),
033     ]),
034   );
035 }
```

Line-by-line explanation

25. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

26. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

27. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

28. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

29. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

30. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

31. Produce the stated result and finish this function call. In a build method the result is a widget description.

32. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

33. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

34. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

35. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

3.5 JSON and typed models

JSON is a text format for exchanging structured data. It has objects, arrays, strings, numbers, Boolean values, and null. JSON keys use double quotes. JSON is not a database and not a Dart object until decoded.

```
import 'dart:convert';
void main() {
  const raw = '{"id":1,"name":"Oxford","price_paise":79900}';
  final data = jsonDecode(raw) as Map<String, dynamic>;
  final id = data['id'] as int;
  final name = data['name'] as String;
  print('$id: $name');
}
```

Line 1 imports conversion utilities. Line 2 starts execution. Line 3 contains JSON text. Line 4 decodes and asserts the expected object shape. Lines 5-6 read expected field types. Line 7 prints `1: Oxford`. Line 8 closes the function. Type casts can throw if the server contract changes; catch failures at the repository boundary and report a malformed response. For larger apps, use explicit model factories or generated serialization with tests.

3.6 Local storage and the notes project

Memory disappears when the process ends. Preferences store small settings such as theme choice. Secure storage uses platform-specific protections for sensitive values such as refresh tokens on native devices. SQLite is a local relational database suitable for structured offline records. Neither preferences nor a local database becomes a trustworthy payment authority merely because it persists.

Create `notes_lab`. Run `flutter pub add shared_preferences`. Replace `lib/main.dart` with the next listing. The async preferences API avoids depending on a potentially stale in-memory cache [S4]. Store only a small learning note here; a real notes app needs durable database semantics, conflict handling, backup/export, and possibly encryption according to its requirements.

flutter_examples/notes.dart | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

notes.dart - lines 1-24

```
001 import 'package:flutter/material.dart';
002 import 'package:shared_preferences/shared_preferences.dart';
003 void main() => runApp(const MaterialApp(home: Notes()));
004 class Notes extends StatefulWidget {
005   const Notes({super.key});
006   @override
007   State<Notes> createState() => _NotesState();
008 }
009 class _NotesState extends State<Notes> {
010   final input = TextEditingController();
011   final storage = SharedPreferencesAsync();
012   bool ready = false;
013   bool saving = false;
014   String message = 'Loading...';
015   @override
016   void initState() {
017     super.initState();
018     load();
019   }
020   Future<void> load() async {
021     try {
022       final text = await storage.getString('note') ?? '';
023       if (!mounted) return;
024       input.text = text;
```

Line-by-line explanation

1. Import Flutter widgets.

2. Import the preferences plugin.

3. Start the note editor.

4. Declare class Notes extends StatefulWidget and begin the fields or methods that define this type.

5. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

6. Tell the Dart analyzer this declaration implements or replaces an inherited member.

7. Declare the method and its return type. Its parameters describe the information supplied by the caller.

8. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

9. Declare class _NotesState extends State<Notes> and begin the fields or methods that define this type.

10. Hold the editable text.

11. Use the asynchronous preferences API.

12. Prevent edits before the saved value has loaded.

13. Prevent overlapping saves.

14. Explain current work to the user.

15. Tell the Dart analyzer this declaration implements or replaces an inherited member.

16. Declare the method and its return type. Its parameters describe the information supplied by the caller.

17. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

18. Begin loading once when the state is created.

19. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

20. Return a future because disk access takes time.

21. Begin an operation whose failures will be handled by the following exception handler.

22. Read the saved value or start empty.

23. Stop if this screen was removed while waiting.

24. Populate the editor only after successful loading.

notes.dart - lines 25-48

```
025       setState(() { ready = true; message = 'Ready'; });
026     } catch (_) {
027       if (mounted) setState(() => message = 'Load failed. Reopen the app.');
028     }
029   }
030   Future<void> save() async {
031     setState(() => saving = true);
032     try {
033       await storage.setString('note', input.text);
034       if (mounted) setState(() => message = 'Saved');
035     } catch (_) {
036       if (mounted) setState(() => message = 'Could not save. Try again.');
037     } finally {
038       if (mounted) setState(() => saving = false);
039     }
040   }
041   @override
042   void dispose() { input.dispose(); super.dispose(); }
043   @override
044   Widget build(BuildContext context) => Scaffold(
045     appBar: AppBar(title: const Text('My note')),
046     body: Padding(padding: const EdgeInsets.all(24), child: ListView(children: [
047       TextField(controller: input, enabled: ready && !saving, maxLines: 8,
048         decoration: const InputDecoration(labelText: 'Note', border:
    OutlineInputBorder())),
```

Line-by-line explanation

25. Enable editing.

26. Handle the specified failure rather than continuing through the successful path.

27. Keep editing disabled to avoid overwriting unseen data.

28. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

29. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

30. Declare the method and its return type. Its parameters describe the information supplied by the caller.

31. Disable another save until this one finishes.

32. Begin an operation whose failures will be handled by the following exception handler.

33. Write the current note.

34. Report successful completion.

35. Handle the specified failure rather than continuing through the successful path.

36. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

37. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

38. Restore the button even after a failure.

39. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

40. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

41. Tell the Dart analyzer this declaration implements or replaces an inherited member.

42. Release the text controller.

43. Tell the Dart analyzer this declaration implements or replaces an inherited member.

44. Declare the method and its return type. Its parameters describe the information supplied by the caller.

45. Supply this named property, field, or configuration entry with the value shown here.

46. Supply this named property, field, or configuration entry with the value shown here.

47. Disable changes during loading or saving.

48. Supply this named property, field, or configuration entry with the value shown here.

notes.dart - lines 49-53

```
049       FilledButton(onPressed: ready && !saving ? save : null, child: const Text('Save')),
050       Text(message),
051     ])),
052   );
053 }
```

Line-by-line explanation

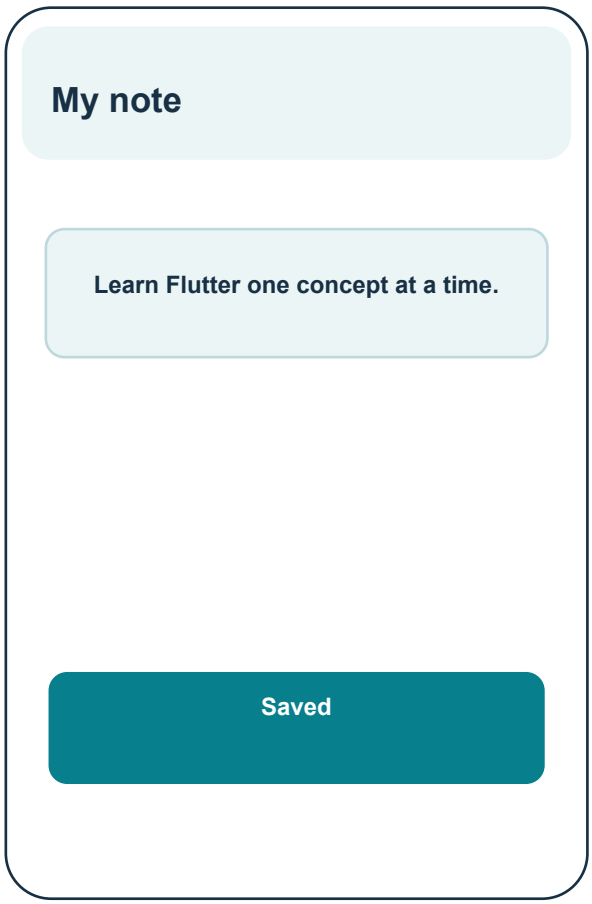
49. A null callback disables the button.

50. Show loading, success, or failure feedback.

51. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

52. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

53. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.



Expected-state illustration: shows the intended information and flow, not an exact device screenshot.

Enter a note, save it, close and reopen the app. Expect the same text. If you type before loading finishes and then an old value arrives, your text could be overwritten; the example prevents this by disabling editing until ready. Modify it to show a character count and a "last saved" indicator without storing passwords in preferences.

Level 4 | State management and professional frontend

4.1 Choosing a state approach

`setState` fits state owned by one small component. Provider commonly exposes objects through the widget tree and works well with ChangeNotifier. Riverpod supplies provider-based dependency and state composition with explicit reading and watching. Bloc structures changes through events and states, encouraging a more formal flow. Each can produce good software; complexity comes from unclear ownership and poorly defined transitions, not merely the package name.

We use Riverpod for the focused shared-state lesson. The complete capstone keeps its small screen controller explicit so its network flow is easy to follow; an exercise migrates that controller into providers. Do not mix four state frameworks to satisfy a checklist.

Concept. A provider is a declaration describing how a value or controller is obtained. A notifier owns state and exposes operations that change it. Dependency injection means giving a component the objects it needs instead of making it secretly create global dependencies.

Why. A cart badge, cart page, and checkout summary need consistent state. A repository should be replaceable with a fake during tests. Riverpod's ProviderScope owns a container for these values, and consumers watch the values they need [S5].

Analogy. A central order desk maintains one record. Different departments subscribe to the relevant part instead of making independent copies.

4.2 Complete Riverpod cart quantity

Create `riverpod_lab`, run `flutter pub add flutter_riverpod`, and use the complete file below. This example uses NotifierProvider, not legacy StateProvider. Riverpod 3 separates some older APIs; do not mix imports from an old tutorial with a new package without checking migration guidance [S5].

flutter_examples/riverpod_cart.dart | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

riverpod_cart.dart - lines 1-23

```
001 import 'package:flutter/material.dart';
002 import 'package:flutter_riverpod/flutter_riverpod.dart';
003 final cartProvider = NotifierProvider<Cart, int>(Cart.new);
004 class Cart extends Notifier<int> {
005   @override
006   int build() => 0;
007   void add() => state++;
008   void remove() { if (state > 0) state--; }
009 }
010 void main() => runApp(const ProviderScope(child: MaterialApp(home: CartPage())));
011 class CartPage extends ConsumerWidget {
012   const CartPage({super.key});
013   @override
014   Widget build(BuildContext context, WidgetRef ref) {
015     final quantity = ref.watch(cartProvider);
016     return Scaffold(appBar: AppBar(title: const Text('Shared cart')),
017       body: Center(child: Column(mainAxisSize: MainAxisSize.min, children: [
018         Text('Quantity: $quantity'),
019         FilledButton(onPressed: () => ref.read(cartProvider.notifier).add(), child: const
    Text('Add shirt')),
020         TextButton(onPressed: () => ref.read(cartProvider.notifier).remove(), child: const
    Text('Remove')),
021       ])));
022   }
023 }
```

Line-by-line explanation

1. Import Flutter UI types.

2. Import Riverpod's widget integration and providers.

3. Describe how to create and expose a Cart controller.

4. Own one integer state and the operations that change it.

5. Tell the Dart analyzer this declaration implements or replaces an inherited member.

6. Initial state is an empty cart.

7. Replacing state notifies listeners.

8. Enforce a non-negative quantity.

9. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

10. Create the provider container above the app.

11. Give build access to a WidgetRef.

12. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

13. Tell the Dart analyzer this declaration implements or replaces an inherited member.

14. Build with both UI context and provider access.

15. Subscribe to the current quantity.

16. Produce the stated result and finish this function call. In a build method the result is a widget description.

17. Supply this named property, field, or configuration entry with the value shown here.

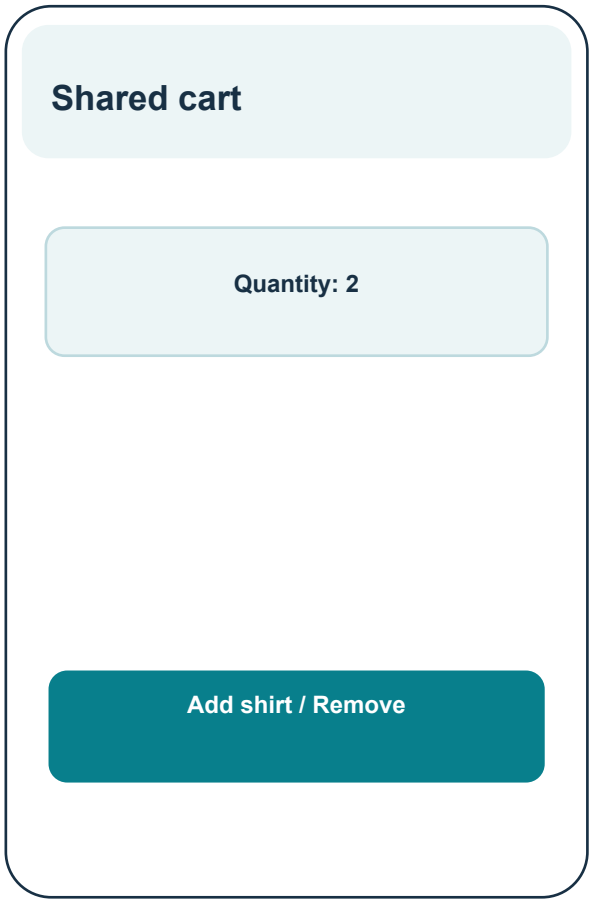
18. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

19. Invoke an action without adding a subscription.

20. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

21. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

22. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.
23. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.



Expected-state illustration: shows the intended information and flow, not an exact device screenshot.

`ref.watch` subscribes during build. `ref.read` obtains a value for a one-time action such as a button press. Watching the notifier alone does not subscribe to every state change. Keep side effects in controller methods, not in build. For a map-based cart, publish a new map instead of modifying the same map instance and expecting identity-based notifications to notice.

Internal flow. The container lazily creates the provider's controller, asks it for initial state, and notifies dependents when its state changes. Automatic disposal is a lifetime choice, not a universal rule; some state should survive a screen transition. Family providers represent parameterized state, such as a product by ID. Clear user-specific state when the authenticated user changes.

Complete asynchronous provider example

Use the same flutter_riverpod dependency as the quantity example. This program first shows loading, then a list. Refresh invalidates the cached read so it can run again. Replace the returned list with a deliberate thrown exception to exercise the Retry branch; restore the loader to recover.

flutter_examples/async_provider.dart | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

async_provider.dart - lines 1-24

```
001 import 'package:flutter/material.dart';
002 import 'package:flutter_riverpod/flutter_riverpod.dart';
003 final namesProvider = FutureProvider<List<String>>((ref) async {
004   await Future<void>.delayed(const Duration(milliseconds: 700));
005   return ['Oxford', 'Linen'];
006 });
007 void main() => runApp(const ProviderScope(child: MaterialApp(home: AsyncPage())));
008 class AsyncPage extends ConsumerWidget {
009   const AsyncPage({super.key});
010   @override
011   Widget build(BuildContext context, WidgetRef ref) {
012     final result = ref.watch(namesProvider);
013     return Scaffold(appBar: AppBar(title: const Text('Provider catalogue')),
014       body: result.when(
015         loading: () => const Center(child: CircularProgressIndicator()),
016         error: (error, stack) => Center(child: FilledButton(onPressed: () =>
    ref.invalidate(namesProvider), child: const Text('Retry'))),
017         data: (names) => ListView(children: [
018           if (names.isEmpty) const ListTile(title: Text('No products')),
019           for (final name in names) ListTile(title: Text(name)),
020           TextButton(onPressed: () => ref.invalidate(namesProvider), child: const
    Text('Refresh')),
021         ]),
022       ));
023   }
024 }
```

Line-by-line explanation

1. Import the named library or module so the following code can use its types and functions.

2. Import the named library or module so the following code can use its types and functions.

3. Declare a cached asynchronous read managed by a provider.

4. Wait for this asynchronous operation before continuing the current function; other event-loop work can still run.

5. Produce the stated result and finish this function call. In a build method the result is a widget description.

6. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

7. Declare the method and its return type. Its parameters describe the information supplied by the caller.

8. Declare class AsyncPage extends ConsumerWidget and begin the fields or methods that define this type.

9. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

10. Tell the Dart analyzer this declaration implements or replaces an inherited member.

11. Declare the method and its return type. Its parameters describe the information supplied by the caller.

12. Rebuild when the asynchronous state changes.

13. Produce the stated result and finish this function call. In a build method the result is a widget description.

14. Supply this named property, field, or configuration entry with the value shown here.

15. Supply this named property, field, or configuration entry with the value shown here.

16. Supply this named property, field, or configuration entry with the value shown here.

17. Supply this named property, field, or configuration entry with the value shown here.

18. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

19. Repeat the following body once for each value in the stated sequence.

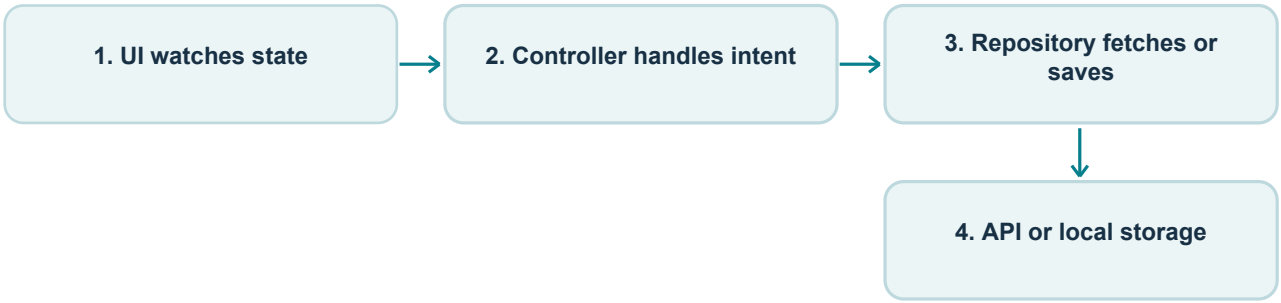
20. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

21. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.
22. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.
23. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.
24. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

4.3 Async providers, loading, errors, and repositories

A FutureProvider suits an asynchronous read. AsyncNotifierProvider suits an asynchronous controller that also has operations, such as saving or refreshing a cart. An AsyncValue expresses loading, data, or error. Empty data is a valid successful result, not necessarily a failure. Give each state its own UI: loading indicator; readable retryable error; useful empty message; or list.

A repository is an application-facing interface for reading and writing data. The UI asks it for products instead of knowing HTTP paths, token headers, and JSON field names. Business logic decides which operation is allowed. The repository performs data access. A provider connects the dependencies.



For refresh behavior, decide whether to show stale data while loading or temporarily replace it. If two searches overlap, prevent an older response from overwriting a newer query's results by using request identifiers, cancellation where supported, or parameterized providers. A spinner is not a solution to race conditions.

Common mistakes. Storing BuildContext in a long-lived controller can retain an obsolete UI location. Returning raw dynamic JSON from every provider spreads parsing errors across screens. Keeping authenticated data after logout can reveal the previous user's information. Derive totals from lines and prices instead of synchronizing several independent totals by hand.

Exercise. Replace the integer cart with an immutable map of variant IDs to quantities. Add a derived total provider. Override the product repository with a fake returning two shirts in a test. Then simulate a timeout and verify the retry state. The commercial connection is one predictable source of cart and session state across the app.

Level 5 | Backend, APIs, and databases

5.1 Frontend, backend, and database

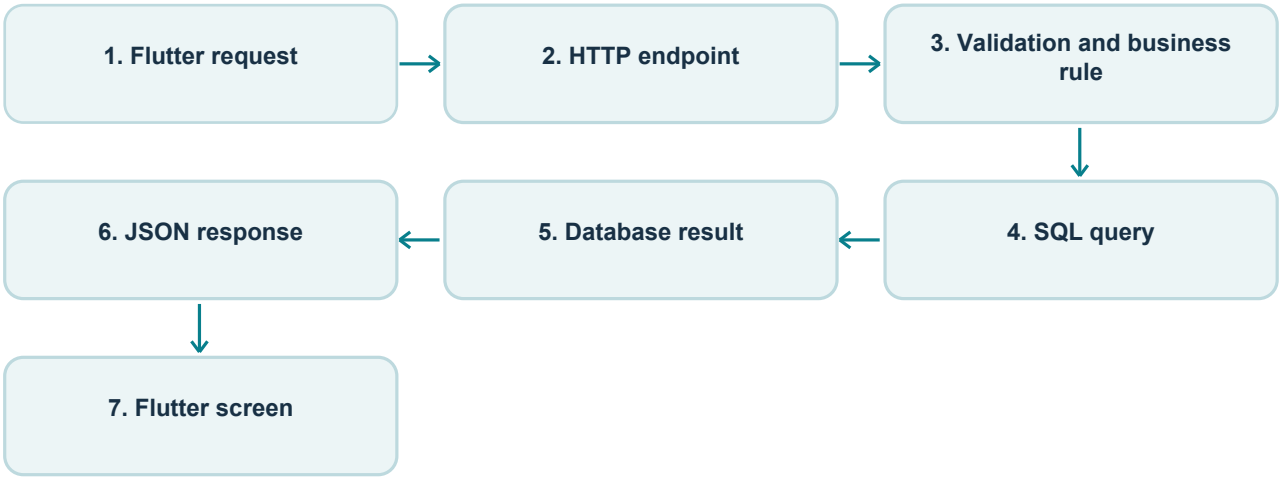
1. Concept. The frontend is the application a person interacts with. The backend is software running in a controlled server environment that receives requests and applies authoritative rules. A database stores structured information so it can be queried and updated reliably.

2. Why it exists. Two customers need the same stock information. A phone can be lost, disconnected, or deliberately modified. A commercial system needs records and checks that do not depend on trusting one phone.

3. Flutter purpose. Flutter collects input and presents results. It asks the backend to sign in, list products, reserve inventory, create orders, and read history. It must not connect directly to PostgreSQL with a shared administrator password embedded in the app.

4. Analogy. The shop counter is the frontend. The staff-only order desk is the backend. The stock and sales ledger is the database.

5. Small example. A request asks for employee records. The server queries a table and returns JSON. Flutter decodes it into rows.



6-7. Code and explanation. The Employee API and Flutter client below are complete examples of this flow. Follow the files in order before running both processes.

9. Internally. A server process listens on a network address and port. An HTTP server such as Uvicorn receives the connection and invokes the FastAPI application. FastAPI matches the path and method, validates data, calls your function, and serializes its response. The database is a different service or, in the SQLite starting configuration, a file accessed through a database engine.

10. Mistakes. `localhost` means the machine on which a process runs. On an Android emulator, `127.0.0.1` refers to the emulator, not your development PC. Android's common emulator host alias is `10.0.2.2`. A physical phone needs a reachable development-machine address and appropriate network configuration.

11. Exercise. Draw which process runs on which machine. Disconnect the backend and explain the Flutter failure without changing the widget layout.

12. Project usage. The same pattern carries products, orders, sessions, and admin operations.

5.2 Python before FastAPI

Python uses indentation to group instructions. A variable assignment is `name = 'Oxford'`; a list is `['S', 'M']`; a dictionary is `{'name': 'Oxford'}`. `def` declares a function. `return` sends a value back to its caller. `None` means no value. Python's `True` and `False` are capitalized. Imports make other modules available.

```
def subtotal(price_paise: int, quantity: int) -> int:
    if quantity < 1:
        raise ValueError('Quantity must be positive')
    return price_paise * quantity

print(subtotal(79900, 2))
```

Line 1 defines a function with type hints. Line 2 checks the business condition. Line 3 raises an error if it fails. Line 4 returns the result. Line 5 is a blank separator with no effect. Line 6 calls and prints the function result: `159800`. Python type hints alone do not validate untrusted incoming data; Pydantic models supply runtime validation in the API.

A virtual environment keeps project packages separate. In a new backend folder run `python -m venv .venv`, then use `.venv\Scripts\python.exe -m pip install -r requirements.txt`. Calling the environment's interpreter explicitly avoids PowerShell activation-policy confusion. A requirements file lists package constraints; a resolved lock or exact tested requirements file records an actual environment.

A decorator is syntax that transforms or registers the function beneath it. `@app.get('/employees')` tells FastAPI which function handles a GET request to that path. `with` manages a resource lifetime. A context manager closes a connection or commits/rolls back a transaction when the block exits. `**dictionary` expands dictionary keys as named arguments.

5.3 HTTP and REST without mystery

HTTP is a request-response protocol. A request contains a method, path, headers, and optionally a body. A response contains a status code, headers, and optionally a body. Headers are metadata such as `Content-Type: application/json` and `Authorization: Bearer ...`. HTTPS is HTTP carried over TLS, which encrypts transport and authenticates the server when certificates are properly validated.

GET retrieves a representation and should not perform a business mutation. POST commonly creates a resource or starts an operation. PUT replaces the representation defined by the endpoint. PATCH changes selected fields. DELETE removes or deactivates a resource according to the API contract. An operation is idempotent if repeating the same operation has the same intended effect as applying it once. This does not require repeated responses to have identical status codes; deleting an already deleted record may return 404.

REST is an architectural style organized around resources and their representations. For this course, use predictable paths such as `/employees/12`, meaningful methods, stateless request authentication, and documented responses. Not every action naturally becomes CRUD; `/orders/12/cancel` can clearly express a business operation with rules.

Status codes are part of the contract. 200 indicates success with a response, 201 creation, 204 success without a body, 400 an invalid request, 401 missing/invalid authentication, 403 insufficient permission, 404 no accessible resource, 409 a conflict with current state, 422 validation failure in these FastAPI examples, and 500 an unexpected server failure. A client should not parse every response as if it were a product list.

CORS is the browser's cross-origin access mechanism. An origin includes scheme, hostname, and port. CORS errors affect browser calls even when command-line calls work. Allow the actual frontend origin; do not treat CORS as authentication, because non-browser clients can call the API directly.

5.4 SQL, tables, keys, and relationships

A relational database organizes records into tables. A row is one record; a column is one attribute. A primary key identifies a row uniquely. A foreign key refers to another table's key and can enforce that the referenced row exists. SQL is the language used to query and change relational data. PostgreSQL is a database system that implements SQL and additional capabilities.

CRUD means create, read, update, and delete. `INSERT` creates rows, `SELECT` reads them, `UPDATE` changes them, and `DELETE` removes them. A WHERE condition limits affected rows. Forgetting WHERE in an UPDATE can change every row. A transaction groups operations into one unit of success or failure.

```
CREATE TABLE departments (
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL UNIQUE
);
CREATE TABLE employees (
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL,
  department_id INTEGER REFERENCES departments(id)
);
INSERT INTO departments(name) VALUES ('Packing');
INSERT INTO employees(name, department_id) VALUES ('Asha', 1);
SELECT e.name, d.name AS department
FROM employees e
JOIN departments d ON d.id = e.department_id;
```

Lines 1-4 create a department table with an automatically generated key and a required unique name. Lines 5-9 create employees and an optional department relationship. Line 10 inserts Packing. Line 11 creates Asha in department 1 in this fresh database example. Line 12 chooses the desired output columns and renames one result column. Line 13 names the employee source with a short alias. Line 14 joins matching departments. Expect Asha and Packing. In a nonempty database, retrieve the inserted key with `RETURNING id` instead of assuming it is one.

Use parameters rather than concatenating user input into SQL. SQLAlchemy in the examples creates bound parameters for expressions such as `employees.c.id == employee_id`. Validation and parameters solve different problems: one enforces allowed input; the other keeps values separate from SQL instructions.

Practical 7/9: complete employee management system

Objective: add, view, edit, delete, and search employee records. This is a local learning API without authentication. Run it on your development machine only. Later, protect these routes with the authorization dependency from Level 6 before treating them as an administrative application.

Create `employee_project/backend`, place `main.py` there, and create the requirements file printed in Level 20 into `employee_project`. The Flutter project is a sibling folder called `employee_ui`. Each file has one role: Python main defines the data table and endpoints; requirements declares packages; Flutter main displays a form and list.

```
cd employee_project
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
cd backend
..\.venv\Scripts\python.exe -m uvicorn main:app --reload
```

`main:app` means the object named `app` in the Python module `main`. `--reload` is for local development. Visit `http://127.0.0.1:8000/docs` to inspect and try the generated API interface. The local SQLite database file is created on first import.

employee_api/main.py | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

main.py - lines 1-24

```
001 import os
002 from fastapi import FastAPI, HTTPException, Query
003 from fastapi.middleware.cors import CORSMiddleware
004 from pydantic import BaseModel, Field, ConfigDict
005 from sqlalchemy import create_engine, MetaData, Table, Column, Integer, String, select
006 from sqlalchemy.exc import IntegrityError
007
008 url = os.getenv('DATABASE_URL', 'sqlite:///employees.db')
009 engine = create_engine(url)
010 metadata = MetaData()
011 employees = Table('employees', metadata,
012     Column('id', Integer, primary_key=True),
013     Column('name', String(80), nullable=False),
014     Column('email', String(120), nullable=False, unique=True))
015 metadata.create_all(engine)
016 app = FastAPI(title='Employee learning API')
017 app.add_middleware(CORSMiddleware, allow_origins=['http://localhost:5173'],
018     allow_methods=['GET', 'POST', 'PUT', 'DELETE'], allow_headers=['Content-Type'])
019
020 class EmployeeIn(BaseModel):
021     model_config = ConfigDict(str_strip_whitespace=True, extra='forbid')
022     name: str = Field(min_length=1, max_length=80)
023     email: str = Field(min_length=3, max_length=120, pattern=r'^[^\s@]+@[^\s@]+\.[^\s@]+$')
024
```

Line-by-line explanation

1. Read configuration from the process environment.

2. Import request routing, errors, and query validation.

3. Permit the specific local browser origin.

4. Describe and validate request bodies.

5. Use database-independent SQL expressions.

6. Detect a database constraint conflict.

7. Blank line separating related pieces of code; it performs no operation.

8. Default to a local file; PostgreSQL is selected with DATABASE_URL.

9. Create the database connection manager.

10. Collect the table definitions.

11. Declare one table named employees.

12. Each employee receives a unique integer identifier.

13. Require a name.

14. Prevent two rows using the same email string.

15. Create missing tables for this first local lesson, not a production migration system.

16. Create the API application.

17. Match the Flutter web port used in the book.

18. Allow only the needed browser operations.

19. Blank line separating related pieces of code; it performs no operation.

20. Require the complete editable representation for create and PUT.

21. Trim text and reject unknown fields.

22. Accept a nonempty bounded name.

23. Perform basic email-shape validation.

24. Blank line separating related pieces of code; it performs no operation.

main.py - lines 25-48

```
025 @app.get('/employees')
026 def list_employees(q: str = Query('', max_length=80)):
027     with engine.connect() as db:
028         query = select(employees).where(employees.c.name.contains(q, autoescape=True))
029         return [dict(row) for row in
    db.execute(query.order_by(employees.c.id).limit(100)).mappings()]
030
031 @app.post('/employees', status_code=201)
032 def add_employee(value: EmployeeIn):
033     try:
034         with engine.begin() as db:
035             result = db.execute(employees.insert().values(**value.model_dump()))
036             return {'id': result.inserted_primary_key[0], **value.model_dump()}
037     except IntegrityError:
038         raise HTTPException(409, 'Email is already in use')
039
040 @app.put('/employees/{employee_id}')
041 def edit_employee(employee_id: int, value: EmployeeIn):
042     try:
043         with engine.begin() as db:
044             result = db.execute(employees.update().where(employees.c.id ==
    employee_id).values(**value.model_dump()))
045             if result.rowcount != 1:
046                 raise HTTPException(404, 'Employee not found')
047             return {'id': employee_id, **value.model_dump()}
048     except IntegrityError:
```

Line-by-line explanation

25. Register the list and search endpoint.

26. Define this named Python operation and its parameters. The indented body runs when the function is called.

27. Release the borrowed connection when the block finishes.

28. Bind the search value and escape LIKE wildcards.

29. Return up to 100 predictable rows.

30. Blank line separating related pieces of code; it performs no operation.

31. Use 201 to indicate a created resource.

32. Define this named Python operation and its parameters. The indented body runs when the function is called.

33. Begin an operation whose failures will be handled by the following exception handler.

34. Commit on success and roll back on failure.

35. Insert only the validated fields.

36. Return the generated ID and saved values.

37. Handle the specified failure rather than continuing through the successful path.

38. Translate the database conflict into a useful API response.

39. Blank line separating related pieces of code; it performs no operation.

40. Replace the editable fields of an existing employee.

41. Define this named Python operation and its parameters. The indented body runs when the function is called.

42. Begin an operation whose failures will be handled by the following exception handler.

43. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

44. Scope the update to the requested ID.

45. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

46. Do not report success for a missing record.

47. Produce the stated result and finish this function call. In a build method the result is a widget description.

48. Handle the specified failure rather than continuing through the successful path.

main.py - lines 49-56

```
049         raise HTTPException(409, 'Email is already in use')
050
051 @app.delete('/employees/{employee_id}', status_code=204)
052 def delete_employee(employee_id: int):
053     with engine.begin() as db:
054         result = db.execute(employees.delete().where(employees.c.id == employee_id))
055         if result.rowcount != 1:
056             raise HTTPException(404, 'Employee not found')
```

Line-by-line explanation

49. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

50. Blank line separating related pieces of code; it performs no operation.

51. Deletion returns no representation.

52. Define this named Python operation and its parameters. The indented body runs when the function is called.

53. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

54. Delete only one matching record.

55. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

56. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

The database connection is opened within each operation and returned promptly. `engine.begin()` provides a transaction. Pydantic rejects blank names and unexpected fields. The email pattern checks basic structure; it does not prove a mailbox exists or normalize every international address. The unique database constraint handles concurrent duplicate requests more reliably than a pre-check alone.

Create the frontend in another terminal:

```
flutter create employee_ui
cd employee_ui
flutter pub add http
flutter run -d chrome --web-port 5173
```

Replace `lib/main.dart` with this file before running. The web port matches the CORS origin. If your browser uses `127.0.0.1` rather than `localhost` in its address, add that exact origin to the local allowlist.

flutter_examples/employees.dart | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

employees.dart - lines 1-24

```
001 import 'dart:convert';
002 import 'package:flutter/material.dart';
003 import 'package:http/http.dart' as http;
004 const base = String.fromEnvironment('API_BASE', defaultValue: 'http://127.0.0.1:8000');
005 void main() => runApp(const MaterialApp(home: Employees()));
006 class Employees extends StatefulWidget {
007   const Employees({super.key});
008   @override
009   State<Employees> createState() => _EmployeesState();
010 }
011 class _EmployeesState extends State<Employees> {
012   final name = TextEditingController();
013   final email = TextEditingController();
014   final search = TextEditingController();
015   final client = http.Client();
016   List<Map<String, dynamic>> rows = [];
017   int? editing;
018   bool busy = false;
019   String message = '';
020   @override
021   void initState() { super.initState(); refresh(); }
022   Future<void> refresh() => perform(() async {
023     final uri = Uri.parse('$base/employees').replace(queryParameters: {'q': search.text});
024     final response = await client.get(uri).timeout(const Duration(seconds: 10));
```

Line-by-line explanation

1. Encode requests and decode JSON responses.

2. Import the named library or module so the following code can use its types and functions.

3. Keep HTTP names under a short prefix.

4. Override for an Android emulator or remote server.

5. Declare the method and its return type. Its parameters describe the information supplied by the caller.

6. Declare class Employees extends StatefulWidget and begin the fields or methods that define this type.

7. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

8. Tell the Dart analyzer this declaration implements or replaces an inherited member.

9. Declare the method and its return type. Its parameters describe the information supplied by the caller.

10. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

11. Declare class _EmployeesState extends State<Employees> and begin the fields or methods that define this type.

12. Hold the editable name.

13. Hold the editable email.

14. Hold a search query.

15. Reuse connections across requests.

16. Store the server's current list.

17. Null means create; an ID means edit that employee.

18. Declare a typed field or value and initialize it from the expression on the right.

19. Declare a typed field or value and initialize it from the expression on the right.

20. Tell the Dart analyzer this declaration implements or replaces an inherited member.

21. Load once when the screen appears.

22. Declare the method and its return type. Its parameters describe the information supplied by the caller.

23. Encode the query safely.

24. Avoid an indefinitely waiting UI.

employees.dart - lines 25-48

```
025     check(response);
026     rows = (jsonDecode(response.body) as List).map((x) => Map<String, dynamic>.from(x as
    Map)).toList();
027   });
028   void check(http.Response response) {
029     if (response.statusCode >= 400) throw Exception('HTTP ${response.statusCode}:
    ${response.body}');
030   }
031   Future<void> perform(Future<void> Function() action) async {
032     if (busy) return;
033     setState(() { busy = true; message = ''; });
034     try { await action(); } catch (error) { message = '$error'; }
035     if (mounted) setState(() => busy = false);
036   }
037   Future<void> save() async {
038     await perform(() async {
039       final uri = Uri.parse('$base/employees${editing == null ? '' : '/$editing'}');
040       final body = jsonEncode({'name': name.text.trim(), 'email': email.text.trim()});
041       const headers = {'Content-Type': 'application/json'};
042       final response = editing == null
043         ? await client.post(uri, headers: headers, body: body).timeout(const
    Duration(seconds: 10))
044         : await client.put(uri, headers: headers, body: body).timeout(const
    Duration(seconds: 10));
045       check(response);
046       if (!mounted) return;
047       editing = null; name.clear(); email.clear();
048     });
```

Line-by-line explanation

25. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

26. Convert the decoded rows.

27. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

28. Declare the method and its return type. Its parameters describe the information supplied by the caller.

29. Do not treat error JSON as successful data.

30. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

31. Share loading and failure handling.

32. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

33. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

34. Keep an understandable failure visible.

35. Publish the result only if still on screen.

36. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

37. Declare the method and its return type. Its parameters describe the information supplied by the caller.

38. Wait for this asynchronous operation before continuing the current function; other event-loop work can still run.

39. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

40. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

41. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

42. Choose create or replace according to the current mode.

43. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

44. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

45. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

46. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

47. Reset only after successful saving.

48. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

employees.dart - lines 49-72

```
049     if (mounted && message.isEmpty) await refresh();
050   }
051   Future<void> remove(int id) async {
052     await perform(() async { check(await
    client.delete(Uri.parse('$base/employees/$id')).timeout(const Duration(seconds: 10))); });
053     if (mounted && message.isEmpty) await refresh();
054   }
055   @override
056   void dispose() { name.dispose(); email.dispose(); search.dispose(); client.close();
    super.dispose(); }
057   @override
058   Widget build(BuildContext context) => Scaffold(
059     appBar: AppBar(title: const Text('Employees')),
060     body: ListView(padding: const EdgeInsets.all(20), children: [
061       TextField(controller: name, decoration: const InputDecoration(labelText: 'Name')),
062       TextField(controller: email, decoration: const InputDecoration(labelText: 'Email')),
063       FilledButton(onPressed: busy ? null : save, child: Text(editing == null ? 'Add
    employee' : 'Save changes')),
064       TextButton(onPressed: busy ? null : () => setState(() { editing = null; name.clear();
     email.clear(); }), child: const Text('Clear form')),
065       TextField(controller: search, decoration: const InputDecoration(labelText: 'Search
    name')),
066       OutlinedButton(onPressed: busy ? null : refresh, child: const Text('Search /
    reload')),
067       if (busy) const LinearProgressIndicator(),
068       if (message.isNotEmpty) Text(message),
069       if (!busy && rows.isEmpty) const Text('No employees found'),
070       for (final row in rows) ListTile(title: Text(row['name'] as String), subtitle:
    Text(row['email'] as String),
071         onTap: busy ? null : () => setState(() { editing = row['id'] as int; name.text =
    row['name'] as String; email.text = row['email'] as String; }),
072         trailing: IconButton(onPressed: busy ? null : () => remove(row['id'] as int), icon:
     const Icon(Icons.delete_outline))),
```

Line-by-line explanation

49. Load the canonical values from the server.

50. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

51. Declare the method and its return type. Its parameters describe the information supplied by the caller.

52. Wait for this asynchronous operation before continuing the current function; other event-loop work can still run.

53. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

54. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

55. Tell the Dart analyzer this declaration implements or replaces an inherited member.

56. Release input and network resources.

57. Tell the Dart analyzer this declaration implements or replaces an inherited member.

58. Declare the method and its return type. Its parameters describe the information supplied by the caller.

59. Supply this named property, field, or configuration entry with the value shown here.

60. Supply this named property, field, or configuration entry with the value shown here.

61. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

62. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

63. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

64. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

65. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

66. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

67. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

68. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

69. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

70. Repeat the following body once for each value in the stated sequence.

71. Supply this named property, field, or configuration entry with the value shown here.

72. Supply this named property, field, or configuration entry with the value shown here.

employees.dart - lines 73-75

```
073     ]),
074   );
075 }
```

Line-by-line explanation

73. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

74. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

75. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.



Expected-state illustration: shows the intended information and flow, not an exact device screenshot.

Add Asha with `asha@example.com`. Expect a new row and HTTP 201. Tap that row, change the name to Asha Rao, and save; expect the same ID and new name. Search `Rao`, then clear the search. Delete the row and expect no rows. Try a duplicate email and a blank name; expect 409 and 422 respectively. A browser error saying it cannot fetch is different from a server response saying a value is invalid.

Switch to PostgreSQL. Create a local database and user using PostgreSQL's administration tools. Set an environment variable such as `$env:DATABASE_URL='postgresql+psycopg://course:local-password@localhost:5432/employee_course'` in the terminal that launches Uvicorn. Use credentials created for your own local installation; the string is an example, not a pre-existing account. Restart the API. Tables in SQLite are not automatically copied to PostgreSQL. This is a new database unless you explicitly migrate data.

Request walkthrough. Pressing Add reads both controllers; jsonEncode builds the body; the HTTP package posts it; CORS permits the development origin; FastAPI parses it; Pydantic validates it; SQLAlchemy inserts within a transaction; the database assigns the key; FastAPI serializes the result; the client checks the status, clears the form, and reloads the list. Every arrow has a possible failure and a place to diagnose it.

Challenge. Add a department field and a maximum result limit. Then protect all employee mutations with an admin dependency. Do not add a client-supplied `is_admin` checkbox and mistake it for permission enforcement.

Level 6 | Authentication and sign-in

6.1 Identity and permission are different questions

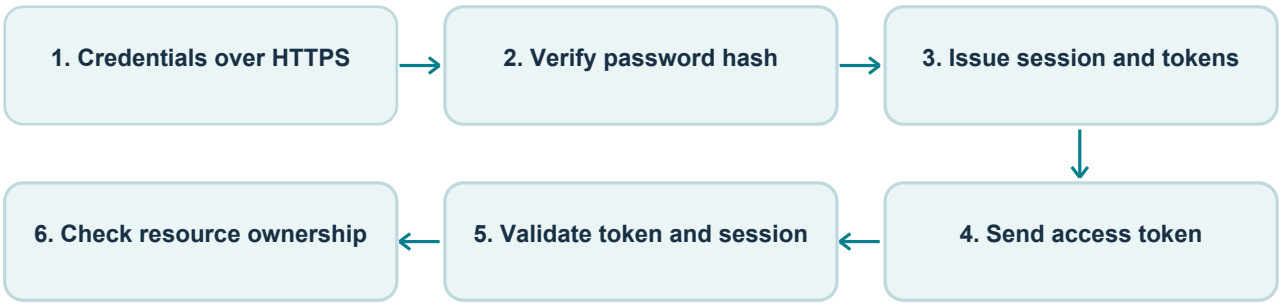
1. Concept. Authentication establishes who is making a request. Authorization decides whether that identity may perform an operation on a resource. A session represents an authenticated relationship over time.

2. Why it exists. A valid customer should be able to view their own order but not another customer's order or every payment in the business.

3. Flutter purpose. Flutter obtains credentials, sends them over HTTPS, handles session expiration, and presents appropriate screens. The server checks permissions for every protected operation, even if the client hides the corresponding button.

4. Analogy. A building entry badge identifies you. The lock on the finance room still decides whether your badge can open that particular door.

5. Example. Login returns an access token. A request sends that token in an Authorization header. The backend validates it and scopes the SQL lookup to the authenticated user ID.



6.2 Password hashing, JWTs, and token expiry

Hashing transforms a password into a value suitable for verification without storing the original password. Password hashing must be deliberately costly and salted. Argon2 is used through pwdlib in the example, following the current FastAPI tutorial's library choices [S6]. A salt is random per-password data incorporated into the stored format; the library manages it. Hashing a human password with plain SHA-256 is not an adequate replacement.

A JWT is a signed claims container. Claims are facts such as subject (`sub`), expiry (`exp`), issuer (`iss`), and audience (`aud`). The payload is normally readable, not encrypted. A signature detects unauthorized changes; it does not hide the contents. Never include passwords or payment secrets in it.

An access token is short-lived and sent to protected APIs. A refresh token obtains a replacement access token without asking the user to type a password every few minutes. This example uses a cryptographically random opaque refresh token, stores only its digest, rotates it on use, and has an absolute seven-day session lifetime. A rotation changes the token secret so an already used secret cannot be used again.

Logout revokes the server session. The example checks session validity on each protected request, so existing access tokens stop working after revocation. A purely stateless JWT system without such a check would generally continue accepting a valid token until expiry unless it added another revocation mechanism. State the behavior you actually implement.

6.3 Complete authentication module

The capstone is split into `db.py` for database definitions, `auth.py` for identity and session operations, `main.py` for products/orders/admin operations, and `payments.py` for the provider boundary. All four must be present when running the full app. Read the schema now because the authentication module depends on its `users` and `sessions` tables.

shop_api/db.py | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

db.py - lines 1-24

```
001 import os
002 from sqlalchemy import create_engine, MetaData, Table, Column, Integer, String, Text,
    ForeignKey, UniqueConstraint, CheckConstraint, event
003
004 engine = create_engine(os.getenv('DATABASE_URL', 'sqlite:///shop.db'))
005 metadata = MetaData()
006 users = Table('users', metadata,
007     Column('id', Integer, primary_key=True),
008     Column('email', String(120), nullable=False, unique=True),
009     Column('password_hash', Text, nullable=False),
010     Column('role', String(20), nullable=False, default='customer'))
011 sessions = Table('sessions', metadata,
012     Column('id', String(64), primary_key=True),
013     Column('user_id', ForeignKey('users.id'), nullable=False),
014     Column('refresh_hash', String(64), unique=True, nullable=False),
015     Column('expires', Integer, nullable=False),
016     Column('revoked', Integer, nullable=False, default=0))
017 variants = Table('product_variants', metadata,
018     Column('id', Integer, primary_key=True),
019     Column('name', String(100), nullable=False),
020     Column('size', String(10), nullable=False),
021     Column('colour', String(30), nullable=False),
022     Column('price_paise', Integer, nullable=False),
023     Column('stock', Integer, nullable=False),
024     CheckConstraint('stock >= 0'), CheckConstraint('price_paise > 0'))
```

Line-by-line explanation

1. Obtain configuration without embedding credentials in source.

2. Import the named library or module so the following code can use its types and functions.

3. Blank line separating related pieces of code; it performs no operation.

4. Use PostgreSQL by setting DATABASE_URL.

5. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

6. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

7. Declare this database column, its data type, and the constraints shown in the arguments.

8. Declare this database column, its data type, and the constraints shown in the arguments.

9. Never store the original password.

10. Declare this database column, its data type, and the constraints shown in the arguments.

11. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

12. Random session identifier carried by access tokens.

13. Declare this database column, its data type, and the constraints shown in the arguments.

14. Store only the SHA-256 digest of a random refresh secret.

15. Unix seconds simplify the small example across databases.

16. Declare this database column, its data type, and the constraints shown in the arguments.

17. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

18. Declare this database column, its data type, and the constraints shown in the arguments.

19. Declare this database column, its data type, and the constraints shown in the arguments.

20. Declare this database column, its data type, and the constraints shown in the arguments.

21. Declare this database column, its data type, and the constraints shown in the arguments.

22. Declare this database column, its data type, and the constraints shown in the arguments.

23. Declare this database column, its data type, and the constraints shown in the arguments.

24. Add a database-enforced rule that applies even when competing requests reach the table.

db.py - lines 25-48

```
025 orders = Table('orders', metadata,
026     Column('id', Integer, primary_key=True),
027     Column('user_id', ForeignKey('users.id'), nullable=False),
028     Column('request_key', String(80), nullable=False),
029     Column('fingerprint', String(64), nullable=False),
030     Column('address', Text, nullable=False),
031     Column('total_paise', Integer, nullable=False),
032     Column('status', String(30), nullable=False),
033     Column('created', Integer, nullable=False),
034     Column('gateway_order', String(100), unique=True),
035     Column('payment_id', String(100), unique=True),
036     UniqueConstraint('user_id', 'request_key'))
037 items = Table('order_items', metadata,
038     Column('id', Integer, primary_key=True),
039     Column('order_id', ForeignKey('orders.id'), nullable=False),
040     Column('variant_id', ForeignKey('product_variants.id'), nullable=False),
041     Column('name', String(160), nullable=False),
042     Column('price_paise', Integer, nullable=False),
043     Column('quantity', Integer, nullable=False),
044     CheckConstraint('quantity > 0'))
045 events = Table('payment_events', metadata,
046     Column('event_id', String(160), primary_key=True),
047     Column('received', Integer, nullable=False))
048 audit = Table('audit_logs', metadata,
```

Line-by-line explanation

25. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

26. Declare this database column, its data type, and the constraints shown in the arguments.

27. Declare this database column, its data type, and the constraints shown in the arguments.

28. Declare this database column, its data type, and the constraints shown in the arguments.

29. Detect reuse of an idempotency key for different input.

30. Keep an address snapshot rather than a mutable address reference alone.

31. Declare this database column, its data type, and the constraints shown in the arguments.

32. Declare this database column, its data type, and the constraints shown in the arguments.

33. Declare this database column, its data type, and the constraints shown in the arguments.

34. Declare this database column, its data type, and the constraints shown in the arguments.

35. Declare this database column, its data type, and the constraints shown in the arguments.

36. Add a database-enforced rule that applies even when competing requests reach the table.

37. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

38. Declare this database column, its data type, and the constraints shown in the arguments.

39. Declare this database column, its data type, and the constraints shown in the arguments.

40. Declare this database column, its data type, and the constraints shown in the arguments.

41. Snapshot the displayed description at purchase time.

42. Declare this database column, its data type, and the constraints shown in the arguments.

43. Declare this database column, its data type, and the constraints shown in the arguments.

44. Add a database-enforced rule that applies even when competing requests reach the table.

45. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

46. Prevent applying the same provider event twice.

47. Declare this database column, its data type, and the constraints shown in the arguments.

48. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

db.py - lines 49-70

```
049     Column('id', Integer, primary_key=True),
050     Column('actor_id', ForeignKey('users.id'), nullable=False),
051     Column('action', String(100), nullable=False),
052     Column('resource_id', Integer, nullable=False),
053     Column('created', Integer, nullable=False))
054 
055 if engine.dialect.name == 'sqlite':
056     @event.listens_for(engine, 'connect')
057     def enable_foreign_keys(connection, _):
058         connection.execute('PRAGMA foreign_keys=ON')
059 
060 def initialize():
061     metadata.create_all(engine)
062     with engine.begin() as db:
063         if not db.execute(variants.select().limit(1)).first():
064             db.execute(variants.insert(), [
065                 dict(name='Oxford shirt', size='M', colour='Blue', price_paise=79900,
    stock=10),
066                 dict(name='Linen shirt', size='L', colour='White', price_paise=99900,
    stock=8)])
067 
068 if __name__ == '__main__':
069     initialize()
070     print('Local schema and catalogue ready')
```

Line-by-line explanation

49. Declare this database column, its data type, and the constraints shown in the arguments.

50. Declare this database column, its data type, and the constraints shown in the arguments.

51. Declare this database column, its data type, and the constraints shown in the arguments.

52. Declare this database column, its data type, and the constraints shown in the arguments.

53. Declare this database column, its data type, and the constraints shown in the arguments.

54. Blank line separating related pieces of code; it performs no operation.

55. SQLite requires foreign-key enforcement to be enabled per connection.

56. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

57. Define this named Python operation and its parameters. The indented body runs when the function is called.

58. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

59. Blank line separating related pieces of code; it performs no operation.

60. Explicit development setup; replace create_all with migrations in production.

61. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

62. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

63. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

64. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

65. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

66. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

67. Blank line separating related pieces of code; it performs no operation.

68. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

69. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

70. Write this value to the console so the lesson result can be inspected.

shop_api/auth.py | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

auth.py - lines 1-24

```
001 import os, time, secrets, hashlib
002 import jwt
003 from pwdlib import PasswordHash
004 from fastapi import APIRouter, Depends, HTTPException
005 from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
006 from pydantic import BaseModel, Field, ConfigDict
007 from sqlalchemy import select
008 from sqlalchemy.exc import IntegrityError
009 from db import engine, users, sessions
010
011 secret = os.environ['JWT_SECRET']
012 if len(secret) < 32:
013     raise RuntimeError('JWT_SECRET must contain at least 32 random characters')
014 hasher = PasswordHash.recommended()
015 dummy_hash = hasher.hash(secrets.token_urlsafe(32))
016 bearer = HTTPBearer()
017 router = APIRouter()
018
019 class Credentials(BaseModel):
020     model_config = ConfigDict(extra='forbid')
021     email: str = Field(min_length=3, max_length=120, pattern=r'^[^\s@]+@[^\s@]+\.[^\s@]+$')
022     password: str = Field(min_length=12, max_length=128)
023
024 class RefreshIn(BaseModel):
```

Line-by-line explanation

1. Configuration, time, cryptographic randomness, and digest utilities.

2. Use a maintained library for signing and validating JWTs.

3. Use Argon2 through the recommended password-hash configuration.

4. Import the named library or module so the following code can use its types and functions.

5. Import the named library or module so the following code can use its types and functions.

6. Import the named library or module so the following code can use its types and functions.

7. Import the named library or module so the following code can use its types and functions.

8. Import the named library or module so the following code can use its types and functions.

9. Import the named library or module so the following code can use its types and functions.

10. Blank line separating related pieces of code; it performs no operation.

11. Fail at startup if no signing secret was supplied.

12. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

13. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

14. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

15. Reduce account-existence timing differences during login.

16. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

17. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

18. Blank line separating related pieces of code; it performs no operation.

19. Declare class Credentials and begin the fields or methods that define this type.

20. Reject a client trying to supply an admin role.

21. Supply this named property, field, or configuration entry with the value shown here.

22. A learning minimum; add breached-password screening for production.

23. Blank line separating related pieces of code; it performs no operation.

24. Declare class RefreshIn and begin the fields or methods that define this type.

auth.py - lines 25-48

```
025     refresh_token: str = Field(min_length=20, max_length=200)
026
027 def digest(value):
028     return hashlib.sha256(value.encode()).hexdigest()
029
030 def access(user_id, sid):
031     now = int(time.time())
032     return jwt.encode(dict(sub=str(user_id), sid=sid, iat=now, exp=now+900,
033         iss='shirt-studio', aud='shirt-client'), secret, algorithm='HS256')
034
035 def issue(user_id):
036     sid, refresh = secrets.token_urlsafe(24), secrets.token_urlsafe(48)
037     with engine.begin() as db:
038         db.execute(sessions.insert().values(id=sid, user_id=user_id,
039             refresh_hash=digest(refresh), expires=int(time.time())+604800, revoked=0))
040     return dict(access_token=access(user_id, sid), refresh_token=refresh,
    token_type='bearer')
041
042 @router.post('/signup', status_code=201)
043 def signup(value: Credentials):
044     email = value.email.strip().lower()
045     password_hash = hasher.hash(value.password)
046     try:
047         with engine.begin() as db:
048             uid = db.execute(users.insert().values(email=email,
    password_hash=password_hash,
```

Line-by-line explanation

25. Supply this named property, field, or configuration entry with the value shown here.

26. Blank line separating related pieces of code; it performs no operation.

27. Define this named Python operation and its parameters. The indented body runs when the function is called.

28. Hash a high-entropy random token, not a human password.

29. Blank line separating related pieces of code; it performs no operation.

30. Define this named Python operation and its parameters. The indented body runs when the function is called.

31. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

32. Produce the stated result and finish this function call. In a build method the result is a widget description.

33. Issue a 15-minute signed access token.

34. Blank line separating related pieces of code; it performs no operation.

35. Define this named Python operation and its parameters. The indented body runs when the function is called.

36. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

37. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

38. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

39. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

40. Produce the stated result and finish this function call. In a build method the result is a widget description.

41. Blank line separating related pieces of code; it performs no operation.

42. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

43. Define this named Python operation and its parameters. The indented body runs when the function is called.

44. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

45. Password processing is separate from the database transaction.

46. Begin an operation whose failures will be handled by the following exception handler.

47. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

48. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

auth.py - lines 49-72

```
049                 role='customer')).inserted_primary_key[0]
050     except IntegrityError:
051         raise HTTPException(409, 'Account cannot be created with that email')
052     return dict(id=uid, email=email)
053
054 @router.post('/login')
055 def login(value: Credentials):
056     with engine.connect() as db:
057         user = db.execute(select(users).where(users.c.email ==
    value.email.strip().lower())).mappings().first()
058     valid = hasher.verify(value.password, user['password_hash'] if user else dummy_hash)
059     if not user or not valid:
060         raise HTTPException(401, 'Invalid credentials')
061     return issue(user['id'])
062
063 def current(credentials: HTTPAuthorizationCredentials = Depends(bearer)):
064     try:
065         claims = jwt.decode(credentials.credentials, secret, algorithms=['HS256'],
066             audience='shirt-client', issuer='shirt-studio', options={'require': ['sub',
    'sid', 'exp', 'iat']})
067         uid, sid = int(claims['sub']), claims['sid']
068     except (jwt.InvalidTokenError, ValueError, KeyError):
069         raise HTTPException(401, 'Sign in again')
070     with engine.connect() as db:
071         session = db.execute(select(sessions).where(sessions.c.id == sid,
072             sessions.c.user_id == uid, sessions.c.revoked == 0,
```

Line-by-line explanation

49. Clients can create only ordinary customers.

50. Handle the specified failure rather than continuing through the successful path.

51. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

52. Produce the stated result and finish this function call. In a build method the result is a widget description.

53. Blank line separating related pieces of code; it performs no operation.

54. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

55. Define this named Python operation and its parameters. The indented body runs when the function is called.

56. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

57. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

58. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

59. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

60. Use the same message for a missing account and a wrong password.

61. Produce the stated result and finish this function call. In a build method the result is a widget description.

62. Blank line separating related pieces of code; it performs no operation.

63. Define this named Python operation and its parameters. The indented body runs when the function is called.

64. Begin an operation whose failures will be handled by the following exception handler.

65. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

66. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

67. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

68. Handle the specified failure rather than continuing through the successful path.

69. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

70. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

71. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

72. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

auth.py - lines 73-96

```
073             sessions.c.expires > int(time.time()))).mappings().first()
074         user = db.execute(select(users.c.id, users.c.email, users.c.role).where(users.c.id
    == uid)).mappings().first()
075     if not session or not user:
076         raise HTTPException(401, 'Sign in again')
077     return dict(user, sid=sid)
078 
079 def admin(user=Depends(current)):
080     if user['role'] != 'admin':
081         raise HTTPException(403, 'Administrator permission required')
082     return user
083 
084 @router.post('/refresh')
085 def refresh(value: RefreshIn):
086     replacement = secrets.token_urlsafe(48)
087     with engine.begin() as db:
088         old = db.execute(select(sessions).where(sessions.c.refresh_hash ==
    digest(value.refresh_token),
089             sessions.c.revoked == 0, sessions.c.expires >
    int(time.time()))).mappings().first()
090         if not old:
091             raise HTTPException(401, 'Sign in again')
092         changed = db.execute(sessions.update().where(sessions.c.id == old['id'],
093             sessions.c.refresh_hash == digest(value.refresh_token), sessions.c.revoked ==
    0).values(refresh_hash=digest(replacement)))
094         if changed.rowcount != 1:
095             raise HTTPException(401, 'Refresh token already used')
096     return dict(access_token=access(old['user_id'], old['id']), refresh_token=replacement,
    token_type='bearer')
```

Line-by-line explanation

73. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

74. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

75. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

76. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

77. Never return the stored password hash.

78. Blank line separating related pieces of code; it performs no operation.

79. Define this named Python operation and its parameters. The indented body runs when the function is called.

80. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

81. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

82. Produce the stated result and finish this function call. In a build method the result is a widget description.

83. Blank line separating related pieces of code; it performs no operation.

84. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

85. Define this named Python operation and its parameters. The indented body runs when the function is called.

86. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

87. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

88. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

89. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

90. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

91. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

92. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

93. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

94. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

95. Only one concurrent rotation wins.

96. Produce the stated result and finish this function call. In a build method the result is a widget description.

auth.py - lines 97-105

```
097
098 @router.post('/logout', status_code=204)
099 def logout(user=Depends(current)):
100     with engine.begin() as db:
101         db.execute(sessions.update().where(sessions.c.id == user['sid']).values(revoked=1))
102
103 @router.get('/me')
104 def me(user=Depends(current)):
105     return dict(id=user['id'], email=user['email'], role=user['role'])
```

Line-by-line explanation

97. Blank line separating related pieces of code; it performs no operation.

98. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

99. Define this named Python operation and its parameters. The indented body runs when the function is called.

100. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

101. Invalidate both refresh and future access checks.

102. Blank line separating related pieces of code; it performs no operation.

103. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

104. Define this named Python operation and its parameters. The indented body runs when the function is called.

105. Produce the stated result and finish this function call. In a build method the result is a widget description.

Endpoint contract. POST `/signup` accepts email and password and always creates a customer. POST `/login` verifies credentials and creates a session. POST `/refresh` rotates a refresh token. POST `/logout` revokes the current session. GET `/me` returns only public profile fields. Password hashes never appear in responses.

Setup. From the `shop_api` folder, install the requirements printed in Level 20 in your virtual environment. Generate a local random secret in PowerShell using `$env:JWT_SECRET = python -c "import secrets; print(secrets.token_urlsafe(48))"`. Use the same interpreter/environment arrangement as your project. Set `$env:PAYMENT_MODE='demo'`. Run `python db.py`, then `python -m uvicorn main:app --reload`. In another terminal run the Flutter client in Level 20. Changing the signing secret invalidates existing tokens; keep stable environment configuration during a session.

Expected result. Sign up with a unique email and at least twelve password characters. Sign in and call `/me` with the access token. Expect the correct ID, email, and customer role. Alter one token character and expect rejection. Refresh once; the old refresh token must stop working. Log out; `/me` must reject the old access token. These behaviors are tested in the backend tests printed in Level 13.

Internals and limits. Password verification is deliberately expensive. Rate-limit login and signup to prevent resource exhaustion. The dummy password hash reduces timing differences for nonexistent accounts but does not create perfect timing indistinguishability. The example rejects reuse of an old refresh token; it does not retain a history that identifies the entire token family and revokes it when reuse is detected. Add that family tracking, concurrent-refresh coordination, and stolen-token response before a production rollout.

6.4 Secure storage and session UX

On native Flutter platforms, a secure-storage plugin can use platform-protected storage. Install `flutter_secure_storage` and follow the target-specific setup instructions [S7]. A write operation is `await storage.write(key: 'refresh', value: token)`, a read is `await storage.read(key: 'refresh')`, and deletion removes the stored secret on logout. This API example assumes `storage` is a FlutterSecureStorage instance in an initialized Flutter app; it is not a standalone console program.

Browser storage has different protections. Do not promise that a web plugin makes tokens inaccessible to malicious JavaScript executing in the same origin. A backend-for-frontend with secure HttpOnly cookies is an alternative browser architecture; it requires a deliberate CSRF design. The provided learning client keeps tokens in memory and asks for sign-in after restart. It does not claim durable secure login across every platform.

Handle "session expired" as a normal state. Attempt one coordinated refresh, retry the original request with the same idempotency key if applicable, and ask the user to sign in if refresh fails. Do not create an infinite refresh loop. Preserve a harmless draft address where appropriate, clear private cached data on logout, and never log raw tokens.

Exercise. Add email verification and password reset as separate flows using random, short-lived, single-use tokens stored as digests. Return generic reset-request responses to reduce account enumeration. Explain why changing a password should revoke other sessions. These are required product features for many real services, not behaviors already implemented by the small module.

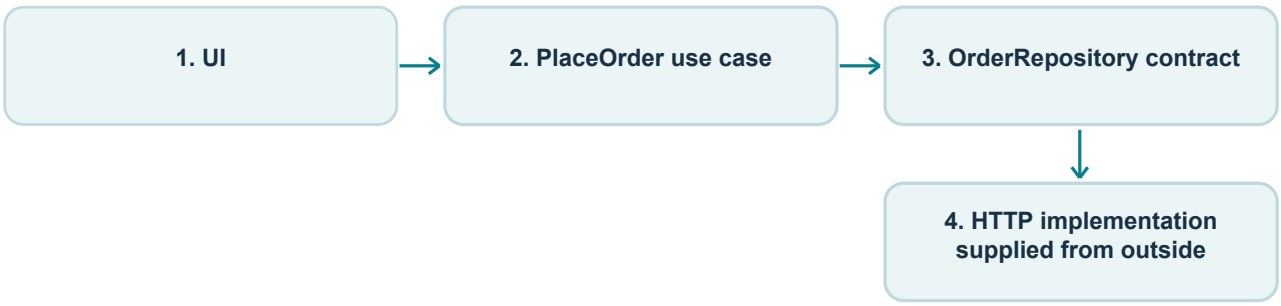
Level 7 | Clean Architecture and SOLID

7.1 Why a large main.dart becomes difficult

A file that draws buttons, builds SQL, parses HTTP, calculates discounts, and stores tokens has many reasons to change. A new API field can break a widget; testing a price rule may require launching the app. Separation of concerns means grouping responsibilities so changes have understandable boundaries.

Presentation handles visual state and user input. Application code coordinates a use case such as placing an order. Domain code describes business concepts and rules. Data/infrastructure code communicates with HTTP, databases, files, or gateways. A model shaped like external JSON is not automatically the same as a domain entity.

The dependency rule matters more than folder names. Domain rules should not import Flutter or a particular database driver. An interface can live near the rule that needs it, while a concrete data implementation depends on that interface. Runtime calls may go toward the database; source-code dependencies can point toward a domain contract. Do not draw a simple chain and infer that the domain must depend on the data implementation.



7.2 S: Single Responsibility

Simple meaning: give a unit one coherent reason to change. Bad design makes Receipt both calculate totals and send emails. A tax change and an email-provider change then modify the same class. Better design separates pricing from notification and lets a use case coordinate them.

```
class Receipt {
  int total(int price, int quantity) => price * quantity;
}
class ReceiptMessage {
  String format(int total) => 'Total paise: $total';
}
void main() {
  final total = Receipt().total(79900, 2);
  print(ReceiptMessage().format(total));
}
```

Line 1 introduces calculation responsibility. Line 2 implements it. Line 3 closes that class. Line 4 introduces formatting responsibility. Line 5 implements it. Line 6 closes the formatter. Line 7 starts the caller. Line 8 calculates 159800. Line 9 formats and prints `Total paise: 159800`. Line 10 closes main. The improvement

is independent testing and independent change; splitting every two lines into a class would not automatically improve design.

7.3 O: Open/Closed

Simple meaning: provide useful extension points without repeatedly rewriting stable callers. Bad design puts `if holiday`, `else if member`, and more special cases into every checkout. Better design represents a discount policy behind a common contract.

```
abstract interface class Discount {
  int apply(int subtotal);
}
class NoDiscount implements Discount {
  @override
  int apply(int subtotal) => subtotal;
}
class TenPercent implements Discount {
  @override
  int apply(int subtotal) => subtotal - (subtotal * 10 ~/ 100);
}
int checkout(int amount, Discount policy) => policy.apply(amount);
void main() => print(checkout(10000, TenPercent()));
```

Line 1 declares the contract. Line 2 requires an integer transformation. Line 3 closes it. Lines 4-7 implement unchanged pricing. Lines 8-11 implement a specifically defined ten-percent discount with integer truncation of the discount amount. Line 12 uses whichever policy it receives. Line 13 prints `9000`. The two override annotations confirm contract methods. A new policy can be added without changing checkout. Define rounding and allowed discount ranges; the pattern itself does not guarantee correct pricing.

7.4 L: Liskov Substitution

Simple meaning: an implementation must keep the promises callers rely on. Bad design says every PaymentMethod can refund but implements a method that throws "unsupported" for one valid subtype. The caller cannot safely use the declared contract. Better design separates payment from refundable-payment capability or narrows the original contract to behavior every implementation actually supports.

```
abstract interface class Payment {
  String pay(int paise);
}
class DemoPayment implements Payment {
  @override
  String pay(int paise) {
    if (paise <= 0) throw ArgumentError('Positive amount required');
    return 'demo-receipt-$paise';
  }
}
void main() {
  Payment payment = DemoPayment();
  print(payment.pay(100));
}
```

Lines 1-3 define a payment contract whose documented accepted amounts are positive. Line 4 supplies an implementation. Line 5 checks the override. Line 6 starts the method. Line 7 enforces the same allowed-input contract, not a stricter hidden requirement. Line 8 returns a demo identifier, line 9 closes the method, and line 10 closes the class. Lines 11-14 use the interface and print `demo-receipt-100`. In real code the contract must also define error outcomes and whether a returned identifier implies success, pending, or merely acceptance.

7.5 I: Interface Segregation

Simple meaning: do not force a client to depend on operations it does not need. Bad design requires a read-only product page to depend on a Store interface with delete-user, refund, and stock-write methods. Better design supplies a CatalogueReader contract to that page and a separate InventoryWriter to management features.

```
abstract interface class CatalogueReader {
  List<String> names();
}
class LocalCatalogue implements CatalogueReader {
  @override
  List<String> names() => ['Oxford', 'Linen'];
}
void showCatalogue(CatalogueReader source) {
  for (final name in source.names()) { print(name); }
}
void main() => showCatalogue(LocalCatalogue());
```

Lines 1-3 declare just the read capability. Lines 4-7 implement it. Line 8 accepts that narrow capability. Line 9 prints each name. Line 10 closes the function. Line 11 runs it. Expect Oxford and Linen. This boundary reduces accidental coupling; it does not replace server authorization.

7.6 D: Dependency Inversion and injection

Simple meaning: a high-level rule should depend on the capability it needs, not a hard-coded technical provider. Bad design creates an HTTP client inside every pricing or UI method. Better design accepts a repository through a constructor. Injection is the act of supplying that dependency; inversion describes which direction the source dependency points.

```
abstract interface class PriceSource {
  int priceFor(int id);
}
class FakePrices implements PriceSource {
  @override
  int priceFor(int id) => 79900;
}
class Quote {
  final PriceSource source;
  Quote(this.source);
  int total(int id, int quantity) => source.priceFor(id) * quantity;
}
void main() => print(Quote(FakePrices()).total(1, 2));
```

Lines 1-3 define the capability. Lines 4-7 supply a fake for the lesson. Line 8 begins the use-case object. Line 9 stores the abstraction. Line 10 injects it. Line 11 calculates using that source. Line 12 ends the class. Line 13 prints `159800`. A real implementation would read from an appropriate trusted data source; the fake intentionally does not model missing IDs.

7.7 Apply the architecture without building a maze

For a growing Flutter shop, start with `features/catalogue/`, `features/cart/`, `features/auth/`, and `features/orders/`. Within a feature, separate presentation from a controller and repository when the code needs those boundaries. Put shared HTTP configuration in a small shared infrastructure folder. Keep domain value types such as Money independent of widgets.

Use cases are named intentions: CreateOrder, CancelOrder, and AdjustStock. A data source is the low-level HTTP or database access component. A repository converts between that mechanism and the application's model. An entity has business identity, such as Order 42. A value object is identified by its values, such as a currency and amount combination.

Exercise: move Product and money formatting out of the local shop screen, inject a catalogue repository, and test it with fake data. Then replace the fake with HTTP without changing the screen's business decisions. Do not introduce interfaces for every private helper solely to imitate an enterprise diagram.

Level 8 | Designing the e-commerce application

8.1 Start from requirements, not screens

Concept. A requirement describes behavior the system must support. An acceptance criterion states an observable result that proves that behavior. "Have a cart" is vague. "Adding the same variant twice increases its quantity to two, and removing the last unit removes the line" is testable.

The shop sells shirts only. A product describes a style, such as Oxford cotton shirt. A variant is a sellable combination such as Blue / M. Inventory belongs to a variant and possibly a stock location. A photograph may belong to a product or a colour, depending on the catalogue. Price may vary by variant. Never identify a cart line solely by a product name.

The customer journey is discover, choose variant, add quantity, provide an address, create an order, pay, and track fulfillment. The management journey is maintain catalogue, receive inventory, process orders, investigate payments, and record refunds. These journeys meet at the same backend rules.

Exercise. Write acceptance criteria for sold-out variants, changed prices, failed payment, and changing an address after purchase. Answer whether you allow guest checkout, partial refunds, returns, multiple warehouses, and tax-inclusive prices. Those choices change the data model. The learning implementation uses registered users, one stock pool, INR, no shipping/tax calculation, and full demo refunds.

8.2 Product discovery, details, and filters

A listing endpoint should accept bounded pagination and specific filters such as size, colour, and price range. Search text is a query value, not a piece of SQL. Return stable product and variant IDs. Apply sorting on the server when pagination is involved; sorting only the currently loaded page does not sort the entire catalogue.

A detail screen needs a title, accessible images, price, available options, quantity selection, stock/availability feedback, and an action with a clear result. When a selected size is unavailable, disable purchase and explain why. Do not silently substitute a different size. Network images need a placeholder and failure state.

The capstone lists each seeded variant directly to keep the first connected implementation small. The local UI project has a separate detail route. Combining the two is a deliberate refactor: pass the variant ID to a detail route, load that variant, then add the ID and quantity to the cart. Do not duplicate the entire catalogue object as the server's authoritative order input.

8.3 Cart and checkout as state transitions

A cart expresses purchase intent; it is not a paid order and usually does not guarantee reserved inventory. A server-created order snapshots the chosen description, price, quantity, address, and totals. The backend decides the price at the instant defined by your policy. If it differs from a displayed quote, ask for confirmation instead of surprising the user after charging.

For this learning implementation, creating an order reserves stock immediately by decrementing available stock in the same transaction. The unpaid order can be cancelled before a gateway attempt exists, releasing the reservation exactly once. Production needs an expiry worker and a clear reconciliation policy for abandoned attempts. A timeout is an unknown outcome, not proof that creation failed.



Idempotency protects a retry. Generate a random request key for one logical operation; keep it and its exact request body until the outcome is known. The database enforces uniqueness for user plus key. Store a request fingerprint so the same key cannot silently create a different order. A UI button being disabled does not protect against network retries or two devices.

8.4 Orders, cancellation, and refunds

An order is a business record, not a mutable view of the current catalogue. Changing a shirt's price tomorrow must not change yesterday's receipt. Store order-item price and description snapshots. Store a delivery-address snapshot; editing the customer's saved address should not rewrite an order already being packed.

An order state machine allows only selected transitions. For example, awaiting_payment can become paid or cancelled under defined conditions. Paid can become fulfillment_pending, then packed, shipped, and delivered. A refund tracks money returning to a customer; a return tracks goods coming back. These are related but different processes.

Cancellation before payment can release stock. Cancellation after a payment attempt begins must coordinate with a possible late capture. A refund must not automatically restock a damaged item or an item still in the customer's possession. Record the physical inventory adjustment separately, with an audit trail.

The capstone uses a compact state set: awaiting_payment, payment_setup, payment_setup_unknown, paid, cancelled, and refunded. It does not implement shipping-provider integration. The point is to make each implemented transition explicit rather than adding a free-form "set status" API that lets any client jump anywhere.

8.5 Complete order and admin API

Read the schema and authentication module in Level 6 first. This module depends on both. It creates order snapshots, checks ownership, performs stock adjustments, and exposes the learning payment/refund operations. The optional gateway module follows in Level 9.

shop_api/main.py | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

main.py - lines 1-24

```
001 import os, time, json, hashlib
002 from fastapi import FastAPI, Depends, HTTPException, Header, Query
003 from fastapi.middleware.cors import CORSMiddleware
004 from pydantic import BaseModel, Field, ConfigDict
005 from sqlalchemy import select, update
006 from sqlalchemy.exc import IntegrityError
007 from db import engine, variants, orders, items, audit
008 from auth import router, current, admin
009
010 app = FastAPI(title='Shirt Studio learning backend')
011 app.include_router(router)
012 app.add_middleware(CORSMiddleware, allow_origins=['http://localhost:5173'],
013     allow_methods=['GET', 'POST', 'PATCH'], allow_headers=['Authorization', 'Content-Type',
     'Idempotency-Key'])
014
015 class LineIn(BaseModel):
016     model_config = ConfigDict(extra='forbid')
017     variant_id: int = Field(gt=0)
018     quantity: int = Field(ge=1, le=10)
019
020 class OrderIn(BaseModel):
021     model_config = ConfigDict(extra='forbid', str_strip_whitespace=True)
022     address: str = Field(min_length=10, max_length=500)
023     lines: list[LineIn] = Field(min_length=1, max_length=20)
024
```

Line-by-line explanation

1. Import the named library or module so the following code can use its types and functions.

2. Import the named library or module so the following code can use its types and functions.

3. Import the named library or module so the following code can use its types and functions.

4. Import the named library or module so the following code can use its types and functions.

5. Import the named library or module so the following code can use its types and functions.

6. Import the named library or module so the following code can use its types and functions.

7. Import the named library or module so the following code can use its types and functions.

8. Import the named library or module so the following code can use its types and functions.

9. Blank line separating related pieces of code; it performs no operation.

10. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

11. Add signup, login, refresh, logout, and profile endpoints.

12. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

13. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

14. Blank line separating related pieces of code; it performs no operation.

15. Declare class LineIn and begin the fields or methods that define this type.

16. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

17. Supply this named property, field, or configuration entry with the value shown here.

18. Bound the learning project's maximum quantity per variant.

19. Blank line separating related pieces of code; it performs no operation.

20. Declare class OrderIn and begin the fields or methods that define this type.

21. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

22. Supply this named property, field, or configuration entry with the value shown here.

23. Supply this named property, field, or configuration entry with the value shown here.

24. Blank line separating related pieces of code; it performs no operation.

main.py - lines 25-48

```
025 class StockIn(BaseModel):
026     delta: int = Field(ge=-1000, le=1000)
027
028 @app.get('/health')
029 def health():
030     return {'status': 'ok'}
031
032 @app.get('/products')
033 def products(q: str = Query('', max_length=80), after: int = 0, limit: int = Query(20,
    ge=1, le=100)):
034     with engine.connect() as db:
035         query = select(variants).where(variants.c.id > after,
036             variants.c.name.contains(q,
    autoescape=True)).order_by(variants.c.id).limit(limit)
037         return [dict(row) for row in db.execute(query).mappings()]
038
039 def owned(db, oid, uid):
040     row = db.execute(select(orders).where(orders.c.id == oid, orders.c.user_id ==
    uid)).mappings().first()
041     if not row:
042         raise HTTPException(404, 'Order not found')
043     return row
044
045 @app.post('/orders', status_code=201)
046 def create_order(value: OrderIn, key: str = Header(alias='Idempotency-Key', min_length=8,
    max_length=80), user=Depends(current)):
047     quantities = {}
048     for line in value.lines:
```

Line-by-line explanation

25. Declare class StockIn and begin the fields or methods that define this type.

26. Apply an adjustment rather than overwriting concurrent stock changes.

27. Blank line separating related pieces of code; it performs no operation.

28. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

29. Define this named Python operation and its parameters. The indented body runs when the function is called.

30. Liveness only; add a separate readiness check for dependencies.

31. Blank line separating related pieces of code; it performs no operation.

32. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

33. Define this named Python operation and its parameters. The indented body runs when the function is called.

34. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

35. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

36. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

37. Produce the stated result and finish this function call. In a build method the result is a widget description.

38. Blank line separating related pieces of code; it performs no operation.

39. Scope resource lookup to the authenticated owner.

40. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

41. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

42. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

43. Produce the stated result and finish this function call. In a build method the result is a widget description.

44. Blank line separating related pieces of code; it performs no operation.

45. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

46. Define this named Python operation and its parameters. The indented body runs when the function is called.

47. Normalize duplicate variant lines before pricing and reserving stock.

48. Repeat the following body once for each value in the stated sequence.

main.py - lines 49-72

```
049         quantities[line.variant_id] = quantities.get(line.variant_id, 0) + line.quantity
050     if any(n > 10 for n in quantities.values()):
051         raise HTTPException(422, 'Maximum ten per variant')
052     payload = json.dumps([value.address, sorted(quantities.items())], separators=(',',
    ':'))
053     fingerprint = hashlib.sha256(payload.encode()).hexdigest()
054     def replay(db):
055         row = db.execute(select(orders).where(orders.c.user_id == user['id'],
    orders.c.request_key == key)).mappings().first()
056         if row and row['fingerprint'] != fingerprint:
057             raise HTTPException(409, 'Idempotency key belongs to different input')
058         return dict(row) if row else None
059     try:
060         with engine.begin() as db:
061             previous = replay(db)
062             if previous:
063                 return previous
064             oid = db.execute(orders.insert().values(user_id=user['id'], request_key=key,
065                 fingerprint=fingerprint, address=value.address, total_paise=0,
066                 status='awaiting_payment',
    created=int(time.time()))).inserted_primary_key[0]
067             total = 0
068             for vid, qty in sorted(quantities.items()):
069                 row = db.execute(select(variants).where(variants.c.id ==
    vid).with_for_update()).mappings().first()
070                 if not row:
071                     raise HTTPException(404, 'Variant not found')
072                 result = db.execute(update(variants).where(variants.c.id == vid,
```

Line-by-line explanation

49. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

50. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

51. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

52. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

53. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

54. Define this named Python operation and its parameters. The indented body runs when the function is called.

55. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

56. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

57. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

58. Produce the stated result and finish this function call. In a build method the result is a widget description.

59. Begin an operation whose failures will be handled by the following exception handler.

60. Header, stock reservations, and line snapshots succeed or fail together.

61. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

62. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

63. Produce the stated result and finish this function call. In a build method the result is a widget description.

64. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

65. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

66. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

67. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

68. Lock rows in a consistent order to reduce deadlocks.

69. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

70. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

71. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

72. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

main.py - lines 73-96

```
073                     variants.c.stock >= qty).values(stock=variants.c.stock - qty))
074                 if result.rowcount != 1:
075                     raise HTTPException(409, 'Insufficient stock')
076                 total += row['price_paise'] * qty
077                 db.execute(items.insert().values(order_id=oid, variant_id=vid,
078                     name=f"{row['name']} / {row['size']} / {row['colour']}",
079                     price_paise=row['price_paise'], quantity=qty))
080             db.execute(orders.update().where(orders.c.id == oid).values(total_paise=total))
081             return dict(owned(db, oid, user['id']))
082     except IntegrityError:
083         with engine.connect() as db:
084             previous = replay(db)
085             if previous:
086                 return previous
087         raise HTTPException(409, 'Order conflict; retry with the same key')
088
089 @app.get('/orders')
090 def my_orders(user=Depends(current)):
091     with engine.connect() as db:
092         return [dict(r) for r in db.execute(select(orders).where(orders.c.user_id ==
    user['id']).order_by(orders.c.id.desc()).limit(100)).mappings()]
093
094 @app.get('/orders/{oid}')
095 def order_detail(oid: int, user=Depends(current)):
096     with engine.connect() as db:
```

Line-by-line explanation

73. The database checks availability atomically.

74. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

75. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

76. Ignore any client-suggested price.

77. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

78. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

79. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

80. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

81. Produce the stated result and finish this function call. In a build method the result is a widget description.

82. A simultaneous retry can collide on the unique request key.

83. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

84. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

85. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

86. Produce the stated result and finish this function call. In a build method the result is a widget description.

87. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

88. Blank line separating related pieces of code; it performs no operation.

89. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

90. Define this named Python operation and its parameters. The indented body runs when the function is called.

91. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

92. Produce the stated result and finish this function call. In a build method the result is a widget description.

93. Blank line separating related pieces of code; it performs no operation.

94. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

95. Define this named Python operation and its parameters. The indented body runs when the function is called.

96. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

main.py - lines 97-120

```
097         order = dict(owned(db, oid, user['id']))
098         order['lines'] = [dict(r) for r in db.execute(select(items).where(items.c.order_id
    == oid)).mappings()]
099         return order
100
101 @app.post('/orders/{oid}/cancel')
102 def cancel(oid: int, user=Depends(current)):
103     with engine.begin() as db:
104         order = owned(db, oid, user['id'])
105         if order['status'] == 'cancelled':
106             return {'status': 'cancelled'}
107         if order['gateway_order']:
108             raise HTTPException(409, 'Payment attempt exists; reconcile it before releasing
     stock')
109         changed = db.execute(orders.update().where(orders.c.id == oid,
110             orders.c.status == 'awaiting_payment',
    orders.c.gateway_order.is_(None)).values(status='cancelled'))
111         if changed.rowcount != 1:
112             raise HTTPException(409, 'Order cannot be cancelled in its current state')
113         for item in db.execute(select(items).where(items.c.order_id == oid)).mappings():
114             db.execute(variants.update().where(variants.c.id ==
    item['variant_id']).values(stock=variants.c.stock+item['quantity']))
115     return {'status': 'cancelled'}
116
117 @app.post('/orders/{oid}/simulate-payment')
118 def simulate_payment(oid: int, user=Depends(current)):
119     if os.getenv('PAYMENT_MODE') != 'demo':
120         raise HTTPException(404, 'Not found')
```

Line-by-line explanation

97. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

98. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

99. Produce the stated result and finish this function call. In a build method the result is a widget description.

100. Blank line separating related pieces of code; it performs no operation.

101. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

102. Define this named Python operation and its parameters. The indented body runs when the function is called.

103. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

104. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

105. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

106. Produce the stated result and finish this function call. In a build method the result is a widget description.

107. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

108. Avoid a late-capture cancellation race.

109. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

110. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

111. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

112. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

113. Repeat the following body once for each value in the stated sequence.

114. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

115. Produce the stated result and finish this function call. In a build method the result is a widget description.

116. Blank line separating related pieces of code; it performs no operation.

117. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

118. Define this named Python operation and its parameters. The indented body runs when the function is called.

119. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

120. The learning-only endpoint is unavailable unless explicitly enabled.

main.py - lines 121-144

```
121     with engine.begin() as db:
122         order = owned(db, oid, user['id'])
123         if order['status'] == 'paid':
124             return {'status': 'paid', 'mode': 'demo'}
125         changed = db.execute(orders.update().where(orders.c.id == oid,
126             orders.c.status == 'awaiting_payment',
    orders.c.gateway_order.is_(None)).values(status='paid', payment_id=f'demo_{oid}'))
127         if changed.rowcount != 1:
128             raise HTTPException(409, 'Payment state conflict')
129     return {'status': 'paid', 'mode': 'demo'}
130
131 @app.get('/admin/orders')
132 def all_orders(user=Depends(admin)):
133     with engine.connect() as db:
134         return [dict(r) for r in
    db.execute(select(orders).order_by(orders.c.id.desc()).limit(100)).mappings()]
135
136 @app.patch('/admin/stock/{vid}')
137 def adjust_stock(vid: int, value: StockIn, user=Depends(admin)):
138     with engine.begin() as db:
139         result = db.execute(variants.update().where(variants.c.id == vid,
140             variants.c.stock + value.delta >= 0).values(stock=variants.c.stock +
    value.delta))
141         if result.rowcount != 1:
142             raise HTTPException(409, 'Variant missing or adjustment would make stock
    negative')
143         db.execute(audit.insert().values(actor_id=user['id'], action=f'stock adjustment
    {value.delta}',
144             resource_id=vid, created=int(time.time())))
```

Line-by-line explanation

121. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

122. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

123. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

124. Produce the stated result and finish this function call. In a build method the result is a widget description.

125. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

126. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

127. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

128. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

129. No money moved; never label this as a real gateway payment.

130. Blank line separating related pieces of code; it performs no operation.

131. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

132. Define this named Python operation and its parameters. The indented body runs when the function is called.

133. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

134. Produce the stated result and finish this function call. In a build method the result is a widget description.

135. Blank line separating related pieces of code; it performs no operation.

136. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

137. Define this named Python operation and its parameters. The indented body runs when the function is called.

138. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

139. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

140. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

141. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

142. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

143. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

144. Write the audit record in the same transaction.

main.py - lines 145-160

```
145     return {'status': 'adjusted'}
146
147 @app.post('/admin/orders/{oid}/demo-refund')
148 def demo_refund(oid: int, user=Depends(admin)):
149     if os.getenv('PAYMENT_MODE') != 'demo':
150         raise HTTPException(404, 'Not found')
151     with engine.begin() as db:
152         result = db.execute(orders.update().where(orders.c.id == oid,
153             orders.c.status == 'paid', orders.c.payment_id ==
    f'demo_{oid}').values(status='refunded'))
154         if result.rowcount != 1:
155             raise HTTPException(409, 'No refundable demo payment')
156         db.execute(audit.insert().values(actor_id=user['id'], action='demo full refund',
    resource_id=oid, created=int(time.time())))
157     return {'status': 'refunded', 'mode': 'demo'}
158
159 from payments import router as payment_router
160 app.include_router(payment_router)
```

Line-by-line explanation

145. Produce the stated result and finish this function call. In a build method the result is a widget description.

146. Blank line separating related pieces of code; it performs no operation.

147. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

148. Define this named Python operation and its parameters. The indented body runs when the function is called.

149. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

150. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

151. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

152. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

153. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

154. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

155. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

156. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

157. Refund and physical restocking are deliberately separate operations.

158. Blank line separating related pieces of code; it performs no operation.

159. Load the optional real-provider endpoints after the app definitions.

160. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

Expected API exercise. Sign in, request two units of variant 1, and send an Idempotency-Key. The seeded price is 79,900 paise, so expect total 159,800 paise and stock to fall from ten to eight. Repeat the exact request with the same key: expect the same order ID and stock still eight. Reuse that key for three units: expect 409. Sign in as another customer and ask for the first customer's order: expect 404.

Line-level reasoning. The stock condition is evaluated by the database in the UPDATE, not solely in an earlier client read. The transaction includes the order header, every line, and every stock change, so failure partway through rolls back the whole order. PostgreSQL row locks keep the read price stable against competing updates in this path. SQLite has different concurrency characteristics and is not evidence that PostgreSQL concurrency behavior has been load-tested.

Level 9 | Payments that can be trusted

9.1 Payment architecture and terminology

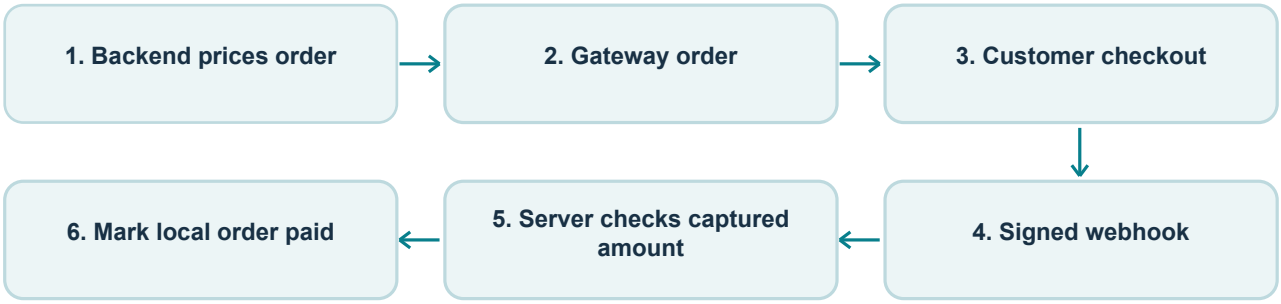
1. Concept. A payment gateway coordinates payment collection with payment networks and providers. A gateway order identifies the amount and currency to be collected. A payment ID identifies a particular payment attempt or transaction. These are different from your application's order ID.

2. Why it exists. Payment processing needs specialized infrastructure. Your app should not collect and store raw card details just because it can display a text field.

3. Flutter purpose. The app starts an approved checkout flow and displays status. The backend owns the amount, creates the gateway order, checks evidence, and updates the order. The client is not trusted to declare that money arrived.

4. Analogy. A customer showing a "paid" picture is not the same as the bank confirming a transfer to the shop.

5. Small example. The server calculates 159,800 paise, creates a gateway order in INR, and sends only the public checkout fields to Flutter. The provider later sends a signed event. The server confirms that the captured payment matches the expected order, amount, and currency.



9.2 Signatures and webhooks

A signature provides evidence that a message came from a party with a secret and that its signed data was not changed. A webhook is an HTTP request sent by an external service when an event occurs. It can arrive late, arrive more than once, or arrive out of the order you expected.

Razorpay's checkout signature and webhook signature are different verification paths. Checkout verification uses the server-stored order ID together with the payment ID and the key secret according to the integration contract. Webhook verification uses the original request body bytes and the configured webhook secret [S8, S9]. Re-encoding parsed JSON can change bytes and invalidate verification.

The module below makes signed captured-payment webhooks authoritative. It does not trust a mobile success callback. It checks the amount, currency, provider order mapping, current local state, and duplicate event ID before applying a transition. This is still a learning integration: operational reconciliation, secret rotation, retry queues, dispute handling, and complete refund workflows need additional implementation and gateway testing.

9.3 Complete server integration example

Set PAYMENT_MODE to `razorpay` only when using your own test-mode provider account and credentials. Configure RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, and RAZORPAY_WEBHOOK_SECRET in the backend environment. The key secret and webhook secret never go into Flutter. Expose the webhook through a controlled HTTPS test endpoint and configure the provider to send captured-payment events. Local unit tests verify signature logic with generated data; they do not prove a live account integration works.

shop_api/payments.py | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

payments.py - lines 1-24

```
001 import os, time, hmac, hashlib, json
002 import httpx
003 from fastapi import APIRouter, Depends, HTTPException, Request
004 from sqlalchemy import select
005 from sqlalchemy.exc import IntegrityError
006 from db import engine, orders, events
007 from auth import current
008
009 router = APIRouter()
010
011 @router.post('/orders/{oid}/gateway-order')
012 def gateway_order(oid: int, user=Depends(current)):
013     if os.getenv('PAYMENT_MODE') != 'razorpay':
014         raise HTTPException(404, 'Not found')
015     key, secret = os.environ['RAZORPAY_KEY_ID'], os.environ['RAZORPAY_KEY_SECRET']
016     with engine.begin() as db:
017         order = db.execute(select(orders).where(orders.c.id == oid, orders.c.user_id ==
    user['id'])).mappings().first()
018         if not order:
019             raise HTTPException(404, 'Order not found')
020         if order['gateway_order'] and order['status'] == 'awaiting_payment':
021             return dict(key=key, order_id=order['gateway_order'],
    amount=order['total_paise'], currency='INR')
022         changed = db.execute(orders.update().where(orders.c.id == oid, orders.c.status ==
    'awaiting_payment',
023             orders.c.gateway_order.is_(None)).values(status='payment_setup'))
024         if changed.rowcount != 1:
```

Line-by-line explanation

1. Import the named library or module so the following code can use its types and functions.

2. Import the named library or module so the following code can use its types and functions.

3. Import the named library or module so the following code can use its types and functions.

4. Import the named library or module so the following code can use its types and functions.

5. Import the named library or module so the following code can use its types and functions.

6. Import the named library or module so the following code can use its types and functions.

7. Import the named library or module so the following code can use its types and functions.

8. Blank line separating related pieces of code; it performs no operation.

9. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

10. Blank line separating related pieces of code; it performs no operation.

11. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

12. Define this named Python operation and its parameters. The indented body runs when the function is called.

13. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

14. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

15. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

16. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

17. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

18. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

19. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

20. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

21. Produce the stated result and finish this function call. In a build method the result is a widget description.

22. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

23. Claim creation so a duplicate request cannot create another attempt.

24. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

payments.py - lines 25-48

```
025             raise HTTPException(409, 'Payment setup already started; reconcile before
    retrying')
026     try:
027         response = httpx.post('https://api.razorpay.com/v1/orders', auth=(key, secret),
028             json={'amount': order['total_paise'], 'currency': 'INR', 'receipt':
    f'shirt_{oid}',
029                   'notes': {'local_order_id': str(oid)}}, timeout=15)
030         response.raise_for_status()
031         remote = response.json()
032         if remote.get('amount') != order['total_paise'] or remote.get('currency') != 'INR'
    or not remote.get('id'):
033             raise ValueError('Unexpected provider response')
034         with engine.begin() as db:
035             db.execute(orders.update().where(orders.c.id == oid, orders.c.status ==
    'payment_setup')
036                 .values(gateway_order=remote['id'], status='awaiting_payment'))
037     except Exception:
038         with engine.begin() as db:
039             db.execute(orders.update().where(orders.c.id == oid, orders.c.status ==
    'payment_setup').values(status='payment_setup_unknown'))
040         raise HTTPException(502, 'Provider outcome needs reconciliation; do not create
    another order')
041     return dict(key=key, order_id=remote['id'], amount=remote['amount'], currency='INR')
042
043 @router.post('/webhooks/razorpay')
044 async def webhook(request: Request):
045     if os.getenv('PAYMENT_MODE') != 'razorpay':
046         raise HTTPException(404, 'Not found')
047     raw = await request.body()
048     signature = request.headers.get('x-razorpay-signature', '')
```

Line-by-line explanation

25. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

26. Begin an operation whose failures will be handled by the following exception handler.

27. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

28. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

29. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

30. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

31. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

32. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

33. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

34. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

35. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

36. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

37. Handle the specified failure rather than continuing through the successful path.

38. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

39. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

40. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

41. Produce the stated result and finish this function call. In a build method the result is a widget description.

42. Blank line separating related pieces of code; it performs no operation.

43. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

44. Define this named Python operation and its parameters. The indented body runs when the function is called.

45. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

46. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

47. Verify the original bytes, before decoding and re-serializing JSON.

48. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

payments.py - lines 49-72

```
049     expected = hmac.new(os.environ['RAZORPAY_WEBHOOK_SECRET'].encode(), raw,
    hashlib.sha256).hexdigest()
050     if not hmac.compare_digest(expected, signature):
051         raise HTTPException(400, 'Invalid signature')
052     try:
053         value = json.loads(raw)
054         if value['event'] != 'payment.captured':
055             return {'ignored': True}
056         payment = value['payload']['payment']['entity']
057         event_id = request.headers['x-razorpay-event-id']
058         remote_id, pid = payment['order_id'], payment['id']
059     except (ValueError, KeyError, TypeError):
060         raise HTTPException(400, 'Malformed event')
061     try:
062         with engine.begin() as db:
063             if db.execute(select(events).where(events.c.event_id == event_id)).first():
064                 return {'duplicate': True}
065             order = db.execute(select(orders).where(orders.c.gateway_order ==
    remote_id).with_for_update()).mappings().first()
066             if not order:
067                 raise HTTPException(503, 'Order mapping not ready; retry event')
068             if payment.get('status') != 'captured' or payment.get('currency') != 'INR' or
    payment.get('amount') != order['total_paise']:
069                 raise HTTPException(400, 'Payment does not match the order')
070             if order['status'] == 'paid' and order['payment_id'] == pid:
071                 db.execute(events.insert().values(event_id=event_id,
    received=int(time.time())))
072                 return {'duplicate_payment': True}
```

Line-by-line explanation

49. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

50. Use a constant-time signature comparison.

51. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

52. Begin an operation whose failures will be handled by the following exception handler.

53. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

54. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

55. Other events require their own explicit transition handlers.

56. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

57. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

58. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

59. Handle the specified failure rather than continuing through the successful path.

60. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

61. Begin an operation whose failures will be handled by the following exception handler.

62. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

63. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

64. Produce the stated result and finish this function call. In a build method the result is a widget description.

65. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

66. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

67. Do not silently acknowledge a capture you cannot reconcile.

68. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

69. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

70. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

71. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

72. Produce the stated result and finish this function call. In a build method the result is a widget description.

payments.py - lines 73-83

```
073             changed = db.execute(orders.update().where(orders.c.id == order['id'],
    orders.c.status == 'awaiting_payment')
074                 .values(status='paid', payment_id=pid))
075             if changed.rowcount != 1:
076                 raise HTTPException(409, 'Order state requires reconciliation')
077             db.execute(events.insert().values(event_id=event_id,
    received=int(time.time())))
078     except IntegrityError:
079         with engine.connect() as db:
080             if db.execute(select(events).where(events.c.event_id == event_id)).first():
081                 return {'duplicate': True}
082         raise HTTPException(409, 'Payment conflict requires reconciliation')
083     return {'accepted': True}
```

Line-by-line explanation

73. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

74. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

75. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

76. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

77. Commit deduplication with the business transition.

78. Handle the specified failure rather than continuing through the successful path.

79. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

80. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

81. Produce the stated result and finish this function call. In a build method the result is a widget description.

82. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

83. Produce the stated result and finish this function call. In a build method the result is a widget description.

Why payment_setup_unknown exists. A network timeout can happen after the provider created its order but before your server received the response. Blindly creating another provider order loses the one-to-one relationship. This implementation blocks another attempt until an operator or reconciliation job resolves the receipt/notes against provider records. The same issue appears with refunds. A state labelled "unknown" is more honest and safer than pretending the operation failed definitively.

Failure exercise. Test a wrong signature, a valid signature with the wrong amount, the same event twice, a captured event arriving before the local mapping is stored, and an already-cancelled order. Confirm that stock is not released merely because a customer closed the checkout window.

9.4 Connecting Flutter to real checkout

The connected shop source uses simulated payment deliberately. To add the official Razorpay Flutter package for supported native targets, install `razorpay_flutter` and follow its current Android/iOS prerequisites [S8]. Create a Razorpay instance in the State's initialization, register success/error/external-wallet listeners, and clear listeners in dispose. A checkout screen receives the server's public key ID, gateway order ID, amount, and currency, then opens the SDK. Do not embed a private key or calculate the authoritative amount in Flutter.

On a client success event, show "Confirming payment" and fetch your backend order status. On cancellation or an SDK error, show a retry/recovery flow that reads backend state first. Your server may still receive a successful late event. A real checkout screen needs lifecycle testing when the app backgrounds, loses network, or is killed. Browser checkout uses the provider's supported web integration rather than assuming a native Flutter plugin is portable to every target.

Checkout field example - illustrative JSON, not a standalone app.

```
{
  "key": "your_public_test_key_id",
  "order_id": "server_created_gateway_order_id",
  "amount": 159800,
  "currency": "INR",
  "name": "Shirt Studio"
}
```

The key identifies the merchant account publicly for checkout. The order ID comes from the server response. Amount is integer paise for INR. Currency identifies the money unit. Name is display text. Replacing placeholders with secrets is not a valid implementation strategy: only the public key belongs here.

Native test-checkout lab: complete client

Create a separate Flutter project for Android or iOS, add `http` and `razorpay_flutter`, and replace its lib/main.dart with this file. Use the supported platform setup from the provider documentation. Start the backend in Razorpay test mode, create an unpaid order with the connected client, then sign in to this lab as the same customer and enter that order ID. The webhook must reach the backend over HTTPS for authoritative confirmation. This integration listing was reviewed but not executed against a provider account or mobile device.

integrations/razorpay_checkout.dart | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

razorpay_checkout.dart - lines 1-24

```
001 import 'dart:convert';
002 import 'package:flutter/material.dart';
003 import 'package:http/http.dart' as http;
004 import 'package:razorpay_flutter/razorpay_flutter.dart';
005 const base = String.fromEnvironment('API_BASE', defaultValue: 'http://10.0.2.2:8000');
006 void main() => runApp(const MaterialApp(home: GatewayLab()));
007 class GatewayLab extends StatefulWidget {
008   const GatewayLab({super.key});
009   @override
010   State<GatewayLab> createState() => _GatewayLabState();
011 }
012 class _GatewayLabState extends State<GatewayLab> {
013   final email = TextEditingController();
014   final password = TextEditingController();
015   final orderId = TextEditingController();
016   final client = http.Client();
017   late final Razorpay gateway;
018   String? token;
019   String status = 'Use a test-mode account and an existing unpaid order.';
020   bool busy = false;
021   @override
022   void initState() {
023     super.initState();
024     gateway = Razorpay();
```

Line-by-line explanation

1. Import the named library or module so the following code can use its types and functions.

2. Import the named library or module so the following code can use its types and functions.

3. Import the named library or module so the following code can use its types and functions.

4. Import the named library or module so the following code can use its types and functions.

5. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

6. Declare the method and its return type. Its parameters describe the information supplied by the caller.

7. Declare class GatewayLab extends StatefulWidget and begin the fields or methods that define this type.

8. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

9. Tell the Dart analyzer this declaration implements or replaces an inherited member.

10. Declare the method and its return type. Its parameters describe the information supplied by the caller.

11. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

12. Declare class _GatewayLabState extends State<GatewayLab> and begin the fields or methods that define this type.

13. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

14. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

15. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

16. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

17. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

18. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

19. Declare a typed field or value and initialize it from the expression on the right.

20. Declare a typed field or value and initialize it from the expression on the right.

21. Tell the Dart analyzer this declaration implements or replaces an inherited member.

22. Declare the method and its return type. Its parameters describe the information supplied by the caller.

23. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

24. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

razorpay_checkout.dart - lines 25-48

```
025     gateway.on(Razorpay.EVENT_PAYMENT_SUCCESS, success);
026     gateway.on(Razorpay.EVENT_PAYMENT_ERROR, failure);
027     gateway.on(Razorpay.EVENT_EXTERNAL_WALLET, wallet);
028   }
029   void success(PaymentSuccessResponse response) {
030     if (mounted) setState(() => status = 'Checkout returned. Confirm status with the
    server.');
031     verify();
032   }
033   void failure(PaymentFailureResponse response) {
034     if (mounted) setState(() => status = 'Checkout interrupted. Check server status before
    retrying.');
035   }
036   void wallet(ExternalWalletResponse response) {
037     if (mounted) setState(() => status = 'Wallet selected; payment still requires
    confirmation.');
038   }
039   Future<Map<String, dynamic>> request(String path, {Map<String, Object>? body}) async {
040     final headers = {'Content-Type': 'application/json', if (token != null)
    'Authorization': 'Bearer $token'};
041     final uri = Uri.parse('$base$path');
042     final response = body == null
043       ? await client.get(uri, headers: headers).timeout(const Duration(seconds: 20))
044       : await client.post(uri, headers: headers, body: jsonEncode(body)).timeout(const
    Duration(seconds: 20));
045     if (response.statusCode >= 400) throw Exception('HTTP ${response.statusCode}');
046     return jsonDecode(response.body) as Map<String, dynamic>;
047   }
048   Future<void> run(Future<void> Function() work) async {
```

Line-by-line explanation

25. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

26. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

27. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

28. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

29. Declare the method and its return type. Its parameters describe the information supplied by the caller.

30. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

31. A client callback alone does not mark the order paid.

32. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

33. Declare the method and its return type. Its parameters describe the information supplied by the caller.

34. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

35. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

36. Declare the method and its return type. Its parameters describe the information supplied by the caller.

37. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

38. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

39. Declare the method and its return type. Its parameters describe the information supplied by the caller.

40. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

41. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

42. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

43. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

44. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

45. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

46. Produce the stated result and finish this function call. In a build method the result is a widget description.

47. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

48. Declare the method and its return type. Its parameters describe the information supplied by the caller.

razorpay_checkout.dart - lines 49-72

```
049     if (busy) return;
050     setState(() => busy = true);
051     try { await work(); } catch (error) { status = '$error'; }
052     if (mounted) setState(() => busy = false);
053   }
054   Future<void> open() => run(() async {
055     final id = int.tryParse(orderId.text);
056     if (id == null || id <= 0) throw Exception('Enter a valid local order ID');
057     final login = await request('/login', body: {'email': email.text.trim(), 'password':
    password.text});
058     token = login['access_token'] as String;
059     final checkout = await request('/orders/$id/gateway-order', body: {});
060     if (!mounted) return;
061     gateway.open({'key': checkout['key'], 'order_id': checkout['order_id'],
062       'amount': checkout['amount'], 'currency': checkout['currency'], 'name': 'Shirt Studio
     Test'});
063     status = 'Waiting for provider checkout';
064   });
065   Future<void> verify() => run(() async {
066     final id = int.tryParse(orderId.text);
067     if (token == null || id == null) throw Exception('Start checkout first');
068     final order = await request('/orders/$id');
069     status = order['status'] == 'paid' ? 'Server confirms payment' : 'Server status:
    ${order['status']}. Check again shortly.';
070   });
071   @override
072   void dispose() { gateway.clear(); email.dispose(); password.dispose(); orderId.dispose();
     client.close(); super.dispose(); }
```

Line-by-line explanation

49. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

50. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

51. Begin an operation whose failures will be handled by the following exception handler.

52. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

53. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

54. Declare the method and its return type. Its parameters describe the information supplied by the caller.

55. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

56. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

57. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

58. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

59. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

60. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

61. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

62. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

63. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

64. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

65. Declare the method and its return type. Its parameters describe the information supplied by the caller.

66. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

67. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

68. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

69. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

70. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

71. Tell the Dart analyzer this declaration implements or replaces an inherited member.

72. Declare the method and its return type. Its parameters describe the information supplied by the caller.

razorpay_checkout.dart - lines 73-86

```
073   @override
074   Widget build(BuildContext context) => Scaffold(
075     appBar: AppBar(title: const Text('Gateway test lab')),
076     body: ListView(padding: const EdgeInsets.all(20), children: [
077       const Text('Native Android/iOS test integration. Never enter live card data for these
     exercises.'),
078       TextField(controller: email, decoration: const InputDecoration(labelText: 'Customer
    email')),
079       TextField(controller: password, obscureText: true, decoration: const
    InputDecoration(labelText: 'Password')),
080       TextField(controller: orderId, decoration: const InputDecoration(labelText: 'Existing
     unpaid local order ID')),
081       FilledButton(onPressed: busy ? null : open, child: const Text('Open test checkout')),
082       OutlinedButton(onPressed: busy ? null : verify, child: const Text('Read server
    status')),
083       Text(status),
084     ]),
085   );
086 }
```

Line-by-line explanation

73. Tell the Dart analyzer this declaration implements or replaces an inherited member.

74. Declare the method and its return type. Its parameters describe the information supplied by the caller.

75. Supply this named property, field, or configuration entry with the value shown here.

76. Supply this named property, field, or configuration entry with the value shown here.

77. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

78. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

79. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

80. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

81. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

82. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

83. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

84. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

85. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

86. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

The code registers three SDK listeners and clears them on disposal. It opens checkout with server-supplied public fields and reads backend status after a callback. A delayed webhook leaves the screen awaiting confirmation; the refresh button reads again. In a production application, inject the existing session and order model rather than asking the customer to type an internal order ID. This lab exposes those values so you can observe the integration boundary.

9.5 Refunds, idempotency, and reconciliation

A refund should have its own record: local refund ID, order ID, payment ID, requested amount, reason, requester, approval, provider refund ID, and state. Validate that cumulative successful and in-progress refunds cannot exceed the refundable captured amount. Record a pending intent before the external call and use a provider-supported idempotency mechanism where available. Do not assume every endpoint accepts the same idempotency header.

If the provider call times out, reconcile before retrying. Marking a refund successful because an HTTP request was sent is incorrect. Track provider acceptance separately from processed/failed state and handle relevant signed events. A full refund may change an order's financial state while fulfillment and return states remain separate.

The learning backend has a demo-only full-refund endpoint. It changes a demo order and records the actor; it does not send money or implement partial refunds. Use it to build management screens and tests before adding real refund operations. Exercise: design the refund table and write tests for two admins simultaneously requesting refunds against the same remaining balance.

Level 10 | Admin and management systems

10.1 Separate audience, shared backend rules

An admin application serves staff tasks and needs different navigation and information density from the customer app. A dashboard summarizes actionable information: unpaid orders needing reconciliation, low stock, failed fulfillment, and failed refunds. A colourful graph without an operational question is not a useful dashboard.

The learning client builds a separate management mode with `--dart-define=ADMIN_APP=true`. This changes the interface; it grants no permission. The backend's admin dependency reads the user's stored role. A customer launching the management build still receives 403 on administrative endpoints.

Register a learning account, then run `python promote_admin.py your-registered-email` from the backend folder. This is a local administrative operation, not a public signup option. The complete file is shown below. Production provisioning needs controlled staff invitation, audit, account recovery, and stronger authentication such as MFA where appropriate.

shop_api/promote_admin.py | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

promote_admin.py - lines 1-13

```
001 import sys
002 from sqlalchemy import select
003 from db import engine, users
004
005 if len(sys.argv) != 2:
006     raise SystemExit('Usage: python promote_admin.py registered-email')
007 with engine.begin() as db:
008     email = sys.argv[1].strip().lower()
009     row = db.execute(select(users).where(users.c.email == email)).mappings().first()
010     if not row:
011         raise SystemExit('Register this account first')
012     db.execute(users.update().where(users.c.id == row['id']).values(role='admin'))
013 print('Promoted the specified local learning account')
```

Line-by-line explanation

1. Import the named library or module so the following code can use its types and functions.

2. Import the named library or module so the following code can use its types and functions.

3. Import the named library or module so the following code can use its types and functions.

4. Blank line separating related pieces of code; it performs no operation.

5. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

6. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

7. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

8. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

9. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

10. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

11. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

12. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

13. Write this value to the console so the lesson result can be inspected.

10.2 RBAC and permissions

RBAC means role-based access control. A role groups permissions, such as support, warehouse, finance, and administrator. Support might view an order but not issue refunds. Warehouse staff might update shipment state but not change product prices. Finance might approve refunds but not grant administrator roles.

Model permissions as operations on resources. Check both the role and the specific resource scope on the server. Multi-tenant staff should not access a different merchant merely because both share the same role name. Long-lived role claims can become stale after a permission change; use current server-side state or a deliberate invalidation/version mechanism.

10.3 Management modules and their rules

Products. Create a style, description, images, and variants. Validate unique SKUs, nonnegative price rules, image type/size, and publication state. Deactivating a product usually preserves past orders rather than deleting their history.

Inventory. Record movements such as received, reserved, shipped, returned, and corrected. Each movement needs an actor, reason, quantity, and reference. A blind "set stock to 10" can overwrite a purchase that just reduced stock. The example uses atomic deltas and prevents negative available stock.

Orders and customers. Display order snapshots and allowed next actions. Limit exposure of personal information to the job being performed. Search and exports need access control and sensible limits. Do not turn a support tool into unrestricted database access.

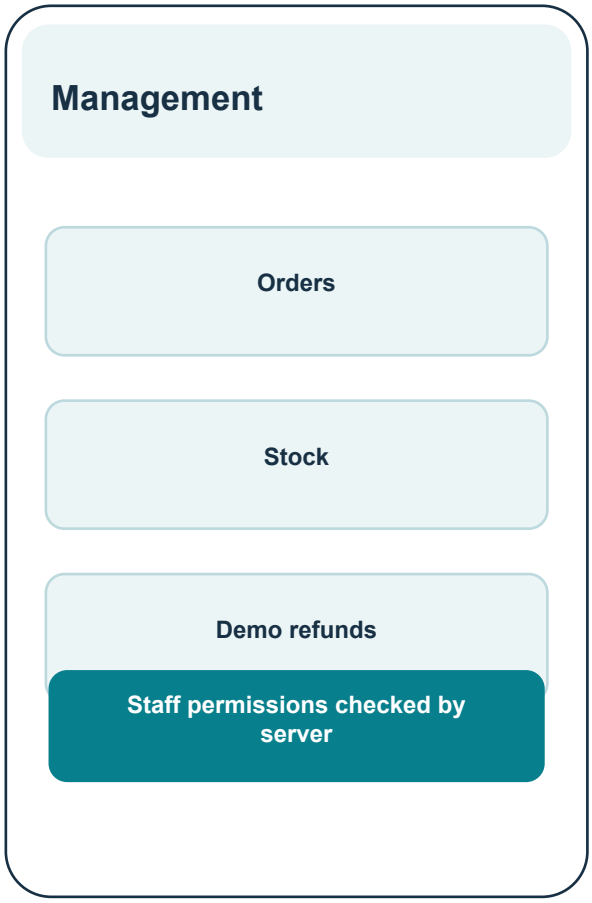
Payments and refunds. Show local and provider identifiers, amount, currency, verification state, and reconciliation history. Require reasons and appropriate permissions for money-moving actions. Consider dual approval for higher-risk actions according to business needs.

Coupons. Define eligibility, valid dates, usage limits, per-customer limits, minimum spend, maximum discount, stacking, and rounding. Enforce redemption counts in a transaction with the order; a UI pre-check is not enough under concurrent checkout.

Reports. Define exactly what a number means. Gross order value, captured revenue, refunded amount, and net revenue are different metrics. Record timezone and time window. Do not sum orders awaiting payment and call it money received.

Users, roles, and audit logs. Role changes and sensitive operations need attribution. An audit record should include actor, action, resource, timestamp, and an appropriate change summary. Protect audit

retention and access; avoid putting passwords, tokens, or unnecessary personal data in it.



Expected-state illustration: shows the intended information and flow, not an exact device screenshot.

Exercise. Add a warehouse role allowed to adjust stock but forbidden to refund. Write one successful and one forbidden API test. Then add a product-creation form that calls a protected endpoint rather than modifying a local list and pretending the product was saved.

Level 11 | Database and data architecture

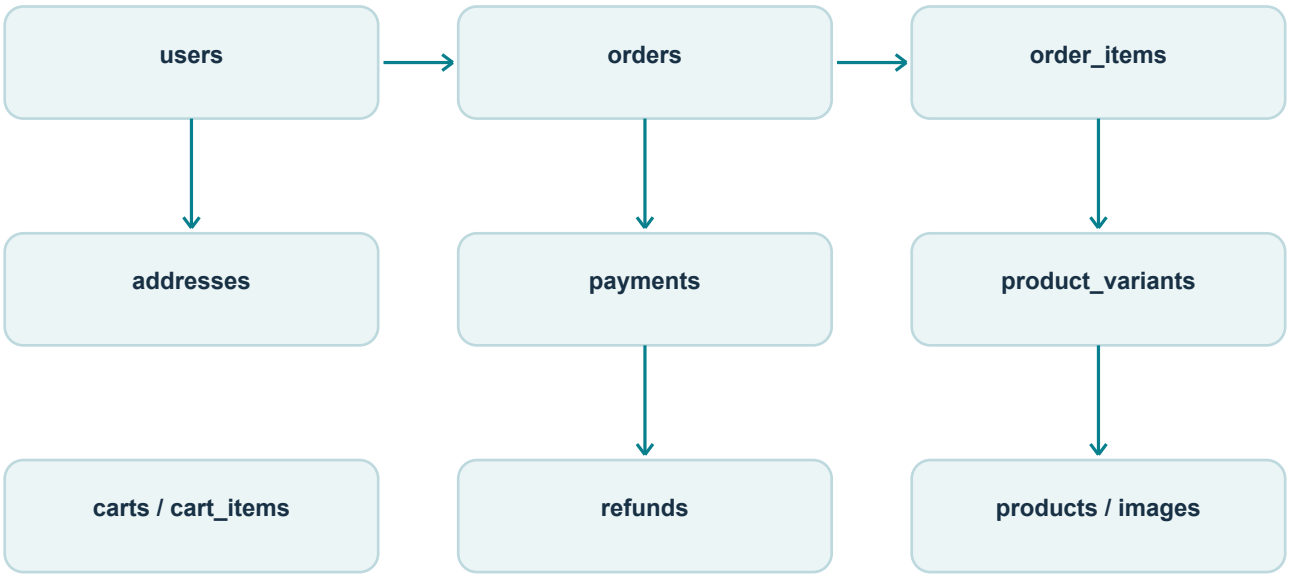
11.1 Design the commercial schema

Normalization organizes facts so they are not inconsistently duplicated. A product name belongs to the product, while a historical order-item name is an intentional snapshot. Normalization is not a prohibition on all duplication; you must distinguish source facts, cached values, and historical records.

`users` owns account identity. `addresses` stores reusable customer address-book entries. `products` stores shared style information. `product_variants` stores SKU, size, colour, and variant price. `product_images` stores image references and ordering. `inventory` or inventory movements tracks quantities by variant and location.

`carts` identifies a customer's current basket. `cart_items` links a cart to variant and quantity. `orders` stores the purchase header, customer, totals, address snapshot, timestamps, and lifecycle state. `order_items` stores immutable purchased-item snapshots. `payments` records attempts, provider IDs, amount, currency, and financial state. `refunds` records separate refund attempts and outcomes.

`coupons` stores discount rules; a redemption table records individual uses. `notifications` or an outbox stores delivery intent and processing state. `audit_logs` records sensitive changes. Session and refresh-token tables support authentication even though they are not customer-visible features.



Conceptual relationships. Foreign-key direction and cardinality are explained in the text.

Learning diagram - component relationships, not a deployment screenshot.

The implemented learning schema combines product and variant descriptions, uses a client-memory cart, and stores compact payment fields on orders. The normalized schema above is the expansion path, not a claim that every table already exists in the demo. Split payment attempts into their own table before supporting multiple attempts, providers, partial refunds, or detailed reconciliation.

11.2 Constraints are the database's last line of defence

A NOT NULL constraint requires a value. UNIQUE prevents duplicate keys such as an external payment ID. A CHECK restricts values, such as quantity greater than zero. A foreign key protects references [S10]. These constraints remain valuable even when the API validates input, because other programs and races can reach the same database.

Choose deletion policy intentionally. Cascading deletion can remove related rows automatically, but financial history often needs retention and deactivation rather than cascades. A database migration that adds a required column must account for existing rows. A constraint violation should become an appropriate API conflict, not a leaked stack trace.

11.3 Transactions and concurrency

Concept. A transaction groups changes so they commit together or roll back together. ACID summarizes atomicity, consistency with enforced rules, isolation from concurrent work according to the chosen level, and durability once committed under the configured database guarantees.

Analogy. Reserve two shirt sizes and create the order as one checkout action. If one reservation fails, do not leave the other size reserved without a valid order.

Two requests can both read stock one. If each later writes zero after accepting a purchase, the system may sell two units. A conditional update such as `UPDATE ... SET stock = stock - 1 WHERE stock >= 1` and a checked affected-row count makes the decision atomic. More complex rules may require row locks, unique constraints, or stricter isolation with retry handling [S11].

Deadlock means transactions wait on one another in a cycle. Acquire resources in a consistent order, keep transactions short, and retry safe transactions according to the database error and idempotency policy. Do not keep a stock transaction open while a human completes a payment form.

11.4 Indexes and query optimization

An index is a data structure that helps the database locate rows without scanning every row. Like a book index, it accelerates certain lookups but consumes space and maintenance work. Add an index for actual query patterns, such as orders by user and descending ID. A composite index's order matters.

Use EXPLAIN and, in an appropriate safe environment, EXPLAIN ANALYZE to inspect a query plan and actual work. Measure rows scanned, time, and repeated calls. A query returning twenty orders can still perform one extra line-item query per order, called the N+1 problem. Use an intentional join or batched query. More indexes do not repair an API that returns every product image and every order line unnecessarily.

Offset pagination skips a number of rows and can become expensive or unstable under changing data. Cursor pagination continues after a stable key. The sample products endpoint uses `id > after`, which is suitable for ID ordering. A price-sorted catalogue requires a cursor that includes price plus a stable tie-breaker, not merely ID.

11.5 Migrations, backups, and recovery

A migration is a versioned database change. Creating missing tables at startup is convenient for a local lesson but does not change existing schemas safely. Use a migration tool such as Alembic with SQLAlchemy, review generated changes, test on a copy, and deploy compatible application changes in stages.

An expand-and-contract change first adds a compatible structure, then updates writers/readers, backfills safely, checks results, and only later removes the obsolete structure. A database rollback may lose data or fail after new writes; application rollback and schema rollback are separate decisions.

A backup is a recoverable copy, not merely a file that exists. PostgreSQL supports several backup approaches, including logical dumps and physical/WAL-based recovery strategies [S12]. Define recovery point objective (acceptable lost time) and recovery time objective (acceptable downtime). Restore into an isolated environment regularly and verify row counts, constraints, and representative business flows. Encrypt and restrict access to backups because they contain the same sensitive data as production.

Exercise. Add a nullable phone field through a migration, backfill it where available, and leave missing values explicit. Restore a local backup into a new database and run the employee CRUD checks against that restored database. Never test destructive restore commands against your only data copy.

Level 12 | Security with concrete examples

12.1 Trust boundaries and HTTPS

A trust boundary is a place where data or control moves between parties with different trust. The phone-to-server boundary is one. The server-to-gateway boundary is another. Treat external values as untrusted even when they came from your own UI. A user can write a different client.

HTTPS protects transport when the certificate chain and hostname are verified. Do not disable certificate verification to make a network error disappear. It does not prove a logged-in user owns an order; authorization still matters. Store secrets in backend configuration or a secret manager, restrict their access, and rotate them when exposed. A value embedded in a mobile binary or web bundle is not a confidential server secret.

12.2 SQL injection, validation, and output handling

SQL injection happens when untrusted text becomes part of SQL instructions. A malicious name should remain a string value, not alter a WHERE clause. Use bound parameters and least-privileged database credentials. Whitelist user-selectable sort fields rather than inserting arbitrary field text into SQL.

Validate type, length, range, and allowed transitions. A field can be syntactically valid JSON but an invalid business request, such as quantity -3. Bound request sizes and list lengths. Reject unexpected price and role fields. Send safe error messages to clients and keep detailed traces in restricted logs.

12.3 XSS and CSRF

XSS, cross-site scripting, occurs when attacker-controlled content executes as code in a browser context. For example, unsafe HTML rendering in a product description or admin preview can expose sessions and staff actions. Flutter's ordinary Text widget is not an HTML interpreter, but embedded webviews, rich HTML packages, custom web integration, and surrounding pages create their own risks. Sanitize where HTML is deliberately allowed, avoid unsafe script injection, and apply browser controls appropriate to the deployment.

CSRF, cross-site request forgery, tricks a browser into making an authenticated request using credentials it attaches automatically, commonly cookies. Use an appropriate combination of same-site cookies, CSRF tokens, and origin validation for cookie-authenticated mutations. Bearer tokens explicitly set in Authorization headers have a different exposure model; this does not make them immune to XSS or theft. CORS alone is not a complete CSRF defence.

12.4 Object authorization and roles

Suppose customer A requests `/orders/42`, which belongs to customer B. Checking only that A is logged in is insufficient. Scope the query to both order ID and authenticated owner. The capstone's `owned` helper does this. For admin actions, check permission on the server and log the action. Never accept the acting user ID from the request body as proof of identity.

Authentication failures, role failures, missing resources, and business conflicts need distinct handling. Returning 404 for a resource outside a user's scope can reduce information leakage. Do not expose an endpoint listing all customer emails merely because your admin screen needs a searchable customer picker.

12.5 Rate limiting, abuse, and resource exhaustion

Rate limiting bounds how often a client or identity can perform operations. Apply stricter limits to login, signup, reset requests, expensive search, exports, and payment actions. In a replicated service, per-process counters are not a global limit; use a shared store or gateway with well-defined behavior. A limit should include a useful retry policy and avoid punishing an entire shared network unnecessarily.

Bound password length before expensive hashing, uploaded-file size before storage, and query result counts before serialization. Use timeouts for outbound calls. Do not retry every failing payment operation automatically. Monitor rejection rates and distinguish malicious traffic from a broken legitimate client.

12.6 Logs, payment security, and a threat-model exercise

Logs should help answer what happened without storing passwords, access tokens, full payment credentials, or unnecessary personal data. Redact sensitive fields before logging. Restrict log access and define retention. Audit records should support attribution and investigation without becoming another uncontrolled copy of secrets.

For payments, server-calculated amounts, signed messages, capture-state checks, duplicate detection, and reconciliation are separate controls. No single check replaces the others. A valid signature for the wrong order must still fail. OWASP's REST guidance provides a useful review reference for authentication, authorization, validation, transport, and error handling [S13].

Exercise: list assets (accounts, money, inventory, personal data), attackers (anonymous caller, ordinary customer, compromised staff account), entry points (APIs, uploads, webhooks), and likely failures. For each, identify a preventive control and a test. Example: a customer edits total_paise to 1; the server rejects the extra field and calculates from stored prices. The printed backend test verifies this concrete case.

Level 13 | Testing from small rules to complete journeys

13.1 What a test proves

1. Concept. A test runs a controlled scenario and checks an expected result. An assertion is the check. A regression is a bug that reappears after a change. Tests preserve important behavior while you modify the system.

2. Why it exists. Manually trying the happy path does not cover invalid input, retries, permissions, timeouts, or concurrent requests.

3. Flutter purpose. Test calculations without UI, widgets without a real backend, and selected complete flows with real integration boundaries.

4. Analogy. A shop checks a weighing scale using a known weight before trusting it with purchases.

5. Small example. Start a counter at zero, tap once, and check that one appears. A test should observe behavior, not merely repeat the same implementation expression on both sides of an equality.

6-7. Complete code. In a project created with the Level 20 commands, the counter lives at `lib/examples/counter.dart` and the project package name is `notes_lab`. Place this test in `test/widget_test.dart`. If you create a differently named project, change the package import to its name and the correct file.

flutter_examples/widget_test.dart | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

widget_test.dart - lines 1-14

```
001 import 'package:flutter/material.dart';
002 import 'package:flutter_test/flutter_test.dart';
003 import 'package:notes_lab/examples/counter.dart';
004 void main() {
005   testWidgets('one tap increments the counter', (tester) async {
006     await tester.pumpWidget(const MaterialApp(home: Counter()));
007     expect(find.text('Count: 0'), findsOneWidget);
008     await tester.tap(find.byIcon(Icons.add));
009     await tester.pump();
010     expect(find.text('Count: 1'), findsOneWidget);
011     expect(find.text('Count: 0'), findsNothing);
012   });
013 }
014
```

Line-by-line explanation

1. Import icons and widget types used in finders.

2. Import Flutter's widget testing tools.

3. Import the counter app after copying counter.dart to lib/main.dart.

4. Declare the method and its return type. Its parameters describe the information supplied by the caller.

5. Run a widget test with a simulated renderer.

6. Build the initial widget tree.

7. Confirm the starting state.

8. Send a tap to the plus button.

9. Render the frame scheduled by setState.

10. Verify the observable result.

11. Ensure stale text disappeared.

12. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

13. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

14. Blank line separating related pieces of code; it performs no operation.

8. Expected output. `All tests passed!` from `flutter test`. The intermediate progress lines vary by tool version. The screen image in the counter chapter is rendered after a tap, with Count: 1.

9. Internally. The widget test framework runs a controlled rendering environment. `pumpWidget` creates the tree; `tap` sends input; `pump` renders scheduled work. `pumpAndSettle` continues until scheduled frames settle, which is unsuitable for a permanently animating spinner without a bound.

10. Mistakes. Finding a button is not proof the business operation succeeded. A passing screenshot comparison does not prove authentication is secure. Mocking every layer can hide incompatible JSON contracts. Use a mix of tests with clear boundaries [S14].

11. Exercise. Add a reset behavior, then a test checking reset after three taps. Make the test fail deliberately by changing the implementation, then restore it.

12. Project usage. Cart totals, validation, loading states, and order actions all need regression coverage.

13.2 Unit, widget, integration, and end-to-end tests

A unit test isolates a small rule or object. Test a discount at zero, the minimum threshold, and one below/above it. Test currency mismatch and invalid quantity. Pure functions make these checks cheap and deterministic.

A widget test renders UI and simulates input. Test empty, loading, data, and error states. Verify text, enabled actions, and navigation results. Use stable keys where visible text is not a suitable identifier. Include larger text and constrained widths for important screens.

An integration test exercises multiple real components together, such as the Flutter application on a device with local persistence. Backend API tests can run through the request layer with a test database. Database integration tests need the actual database engine when relying on its locking, constraints, isolation, or SQL dialect.

An end-to-end test follows a user journey across boundaries: sign in, choose a shirt, create an order, complete a provider test payment, and observe verified status. Keep a smaller number of these because they are slower and more sensitive to environment. Their failures should provide traces, logs, and screenshots, not merely "something failed."

13.3 Complete backend tests

The test file uses a disposable `test_shop.db` in the backend working folder and creates a fresh schema before each test. It must not be pointed at a real database. It sets its own DATABASE_URL before importing the application. The production environment should never invoke this destructive test setup.

shop_api/test_shop.py | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

test_shop.py - lines 1-24

```
001 import os, secrets, json, hmac, hashlib
002 from pathlib import Path
003 os.environ['DATABASE_URL'] = 'sqlite:///test_shop.db'
004 os.environ['JWT_SECRET'] = secrets.token_urlsafe(48)
005 os.environ['PAYMENT_MODE'] = 'demo'
006 from fastapi.testclient import TestClient
007 from db import metadata, engine, initialize
008 from main import app
009
010 def setup_function():
011     metadata.drop_all(engine)
012     initialize()
013
014 def customer(client, email='learner@example.com'):
015     data = {'email': email, 'password': 'a-long-test-password'}
016     assert client.post('/signup', json=data).status_code == 201
017     response = client.post('/login', json=data)
018     assert response.status_code == 200
019     tokens = response.json()
020     return {'Authorization': 'Bearer ' + tokens['access_token']}, tokens
021
022 def make_order(client, headers, key='request-0001', quantity=2):
023     return client.post('/orders', headers={**headers, 'Idempotency-Key': key},
024         json={'address': '10 Cotton Street, Test City', 'lines': [{'variant_id': 1,
    'quantity': quantity}]})
```

Line-by-line explanation

1. Import the named library or module so the following code can use its types and functions.

2. Import the named library or module so the following code can use its types and functions.

3. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

4. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

5. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

6. Import the named library or module so the following code can use its types and functions.

7. Import the named library or module so the following code can use its types and functions.

8. Import the named library or module so the following code can use its types and functions.

9. Blank line separating related pieces of code; it performs no operation.

10. Define this named Python operation and its parameters. The indented body runs when the function is called.

11. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

12. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

13. Blank line separating related pieces of code; it performs no operation.

14. Define this named Python operation and its parameters. The indented body runs when the function is called.

15. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

16. Check the expected result. A mismatch fails the test and identifies a broken contract.

17. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

18. Check the expected result. A mismatch fails the test and identifies a broken contract.

19. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

20. Produce the stated result and finish this function call. In a build method the result is a widget description.

21. Blank line separating related pieces of code; it performs no operation.

22. Define this named Python operation and its parameters. The indented body runs when the function is called.

23. Produce the stated result and finish this function call. In a build method the result is a widget description.

24. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

test_shop.py - lines 25-48

```
025
026 def test_checkout_retries_and_owner_isolation():
027     with TestClient(app) as c:
028         h, _ = customer(c)
029         first = make_order(c, h)
030         assert first.status_code == 201
031         oid = first.json()['id']
032         assert first.json()['total_paise'] == 159800
033         assert make_order(c, h).json()['id'] == oid
034         assert c.get('/products').json()[0]['stock'] == 8
035         assert make_order(c, h, quantity=3).status_code == 409
036         other, _ = customer(c, 'other@example.com')
037         assert c.get(f'/orders/{oid}', headers=other).status_code == 404
038         assert c.get('/admin/orders', headers=h).status_code == 403
039         assert c.post(f'/orders/{oid}/simulate-payment', headers=h).json()['status'] ==
    'paid'
040         assert c.post(f'/orders/{oid}/cancel', headers=h).status_code == 409
041
042 def test_insufficient_stock_rolls_back_and_cancel_releases_once():
043     with TestClient(app) as c:
044         h, _ = customer(c)
045         oid = make_order(c, h, quantity=8).json()['id']
046         assert make_order(c, h, key='request-0002', quantity=3).status_code == 409
047         assert len(c.get('/orders', headers=h).json()) == 1
048         assert c.post(f'/orders/{oid}/cancel', headers=h).status_code == 200
```

Line-by-line explanation

25. Blank line separating related pieces of code; it performs no operation.

26. Define this named Python operation and its parameters. The indented body runs when the function is called.

27. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

28. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

29. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

30. Check the expected result. A mismatch fails the test and identifies a broken contract.

31. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

32. Check the expected result. A mismatch fails the test and identifies a broken contract.

33. Check the expected result. A mismatch fails the test and identifies a broken contract.

34. Check the expected result. A mismatch fails the test and identifies a broken contract.

35. Check the expected result. A mismatch fails the test and identifies a broken contract.

36. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

37. Check the expected result. A mismatch fails the test and identifies a broken contract.

38. Check the expected result. A mismatch fails the test and identifies a broken contract.

39. Check the expected result. A mismatch fails the test and identifies a broken contract.

40. Check the expected result. A mismatch fails the test and identifies a broken contract.

41. Blank line separating related pieces of code; it performs no operation.

42. Define this named Python operation and its parameters. The indented body runs when the function is called.

43. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

44. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

45. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

46. Check the expected result. A mismatch fails the test and identifies a broken contract.

47. Check the expected result. A mismatch fails the test and identifies a broken contract.

48. Check the expected result. A mismatch fails the test and identifies a broken contract.

test_shop.py - lines 49-72

```
049         assert c.post(f'/orders/{oid}/cancel', headers=h).status_code == 200
050         assert c.get('/products').json()[0]['stock'] == 10
051
052 def test_refresh_rotation_and_logout():
053     with TestClient(app) as c:
054         h, tokens = customer(c)
055         refreshed = c.post('/refresh', json={'refresh_token': tokens['refresh_token']})
056         assert refreshed.status_code == 200
057         assert c.post('/refresh', json={'refresh_token':
    tokens['refresh_token']}).status_code == 401
058         new = {'Authorization': 'Bearer ' + refreshed.json()['access_token']}
059         assert c.post('/logout', headers=new).status_code == 204
060         assert c.get('/me', headers=h).status_code == 401
061         assert c.post('/refresh', json={'refresh_token':
    refreshed.json()['refresh_token']}).status_code == 401
062
063 def test_client_price_injection_is_rejected():
064     with TestClient(app) as c:
065         h, _ = customer(c)
066         body = {'address': '10 Cotton Street', 'lines': [{'variant_id': 1, 'quantity': 1}],
     'total_paise': 1}
067         assert c.post('/orders', headers={**h, 'Idempotency-Key': 'request-0001'},
    json=body).status_code == 422
068
069 def test_webhook_signature_amount_and_duplicate():
070     from sqlalchemy import update
071     from db import orders
072     with TestClient(app) as c:
```

Line-by-line explanation

49. Check the expected result. A mismatch fails the test and identifies a broken contract.

50. Check the expected result. A mismatch fails the test and identifies a broken contract.

51. Blank line separating related pieces of code; it performs no operation.

52. Define this named Python operation and its parameters. The indented body runs when the function is called.

53. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

54. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

55. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

56. Check the expected result. A mismatch fails the test and identifies a broken contract.

57. Check the expected result. A mismatch fails the test and identifies a broken contract.

58. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

59. Check the expected result. A mismatch fails the test and identifies a broken contract.

60. Check the expected result. A mismatch fails the test and identifies a broken contract.

61. Check the expected result. A mismatch fails the test and identifies a broken contract.

62. Blank line separating related pieces of code; it performs no operation.

63. Define this named Python operation and its parameters. The indented body runs when the function is called.

64. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

65. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

66. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

67. Check the expected result. A mismatch fails the test and identifies a broken contract.

68. Blank line separating related pieces of code; it performs no operation.

69. Define this named Python operation and its parameters. The indented body runs when the function is called.

70. Import the named library or module so the following code can use its types and functions.

71. Import the named library or module so the following code can use its types and functions.

72. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

test_shop.py - lines 73-93

```
073         h, _ = customer(c)
074         oid = make_order(c, h).json()['id']
075         with engine.begin() as db:
076             db.execute(update(orders).where(orders.c.id ==
    oid).values(gateway_order='order_test'))
077         os.environ['PAYMENT_MODE'] = 'razorpay'
078         os.environ['RAZORPAY_WEBHOOK_SECRET'] = 'local-test-webhook-secret'
079         try:
080             event = {'event': 'payment.captured', 'payload': {'payment': {'entity': {
081                 'id': 'pay_test', 'order_id': 'order_test', 'amount': 159800, 'currency':
    'INR', 'status': 'captured'}}}}
082             raw = json.dumps(event).encode()
083             signature = hmac.new(os.environ['RAZORPAY_WEBHOOK_SECRET'].encode(), raw,
    hashlib.sha256).hexdigest()
084             headers = {'x-razorpay-signature': signature, 'x-razorpay-event-id': 'evt_1',
    'Content-Type': 'application/json'}
085             assert c.post('/webhooks/razorpay', content=raw, headers={**headers,
    'x-razorpay-signature': 'wrong'}).status_code == 400
086             bad = json.dumps({**event, 'payload': {'payment': {'entity':
    {**event['payload']['payment']['entity'], 'amount': 1}}}}).encode()
087             bad_sig = hmac.new(os.environ['RAZORPAY_WEBHOOK_SECRET'].encode(), bad,
    hashlib.sha256).hexdigest()
088             assert c.post('/webhooks/razorpay', content=bad, headers={**headers,
    'x-razorpay-signature': bad_sig}).status_code == 400
089             assert c.post('/webhooks/razorpay', content=raw, headers=headers).json() ==
    {'accepted': True}
090             assert c.post('/webhooks/razorpay', content=raw, headers=headers).json() ==
    {'duplicate': True}
091             assert c.get(f'/orders/{oid}', headers=h).json()['status'] == 'paid'
092         finally:
093             os.environ['PAYMENT_MODE'] = 'demo'
```

Line-by-line explanation

73. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

74. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

75. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

76. Execute the constructed SQL statement on this transaction/connection. Bound values remain separate from SQL instructions.

77. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

78. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

79. Begin an operation whose failures will be handled by the following exception handler.

80. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

81. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

82. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

83. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

84. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

85. Check the expected result. A mismatch fails the test and identifies a broken contract.

86. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

87. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

88. Check the expected result. A mismatch fails the test and identifies a broken contract.

89. Check the expected result. A mismatch fails the test and identifies a broken contract.

90. Check the expected result. A mismatch fails the test and identifies a broken contract.

91. Check the expected result. A mismatch fails the test and identifies a broken contract.

92. Run this cleanup path regardless of success or failure.

93. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

Run `python -m pytest -q` from `shop_api` with the installed requirements. These checks cover duplicate order retry, price calculation, ownership, admin rejection, stock rollback, one-time cancellation release, refresh rotation, logout, rejected client price injection, webhook signature, amount matching, and event deduplication. They do not simulate truly concurrent PostgreSQL transactions, a real gateway, or an actual mobile checkout.

The captured-payment test signs locally constructed JSON with a test secret. That proves your verification and state-transition path accepts or rejects controlled input. It does not prove the provider has the correct webhook URL, the live secret matches, or a real captured event has been delivered.

13.4 A practical test plan for the final shop

For every endpoint, test an ordinary success, invalid input, unauthenticated access, an authenticated but unauthorized user, missing data, and a conflict. For money and inventory, add retries and concurrent attempts. For the client, test offline launch, expired session, loading interrupted by navigation, empty results, long product names, and large text.

Use provider test mode for payments. Test cancellation, failure, delayed capture, duplicate webhook, and refund failure. Never use a real charge merely as a convenient automated test fixture. Keep credentials out of source and CI logs.

Write fixtures as data with clear ownership. A fixture sets up a known starting state. A fake is a simple working replacement, such as an in-memory catalogue repository. A mock usually verifies specific

interactions. Prefer checking business results unless an interaction itself is the contract, such as "do not contact the gateway twice for the same operation."

Level 14 | Performance based on measurement

14.1 Find the bottleneck

A bottleneck is the part limiting overall throughput or response time. A slow product page might be waiting on DNS, downloading oversized images, parsing a huge response, running an expensive SQL query, or rebuilding an expensive UI subtree. Guessing at the widget tree will not repair a slow database.

Measure a representative device and dataset in an appropriate mode. Debug builds include overhead and are not reliable for final frame-performance conclusions. Use Flutter DevTools, network timing, server request durations, and database query plans. Record a baseline, make one targeted change, and compare under similar conditions [S15].

At 60 frames per second, a frame interval is about 16.7 ms; other refresh rates have different budgets. Work exceeding the relevant budget can cause visible jank. CPU-heavy code on the UI isolate can delay frame production even if it is inside an async function.

14.2 Flutter rebuilds, lists, and images

Rebuilding a small widget is normal. The goal is not zero rebuilds; it is avoiding unnecessary expensive work. Keep build methods focused on description, extract stable components, use const where appropriate, and subscribe to the state a component actually needs. Do not fetch or sort a large dataset on every build.

Use builder-based scrolling for large datasets. Supply bounded images and request appropriate resolutions. A thumbnail decoded from a huge photograph can consume far more memory than its displayed dimensions suggest. Cache deliberately, account for failed images, and avoid keeping every visited full-resolution image alive.

Lazy loading means doing work when needed, such as building visible rows or requesting the next page. It must include a stable loading indicator, retry behavior, and an end-of-results condition. Infinite-scroll requests should not launch repeatedly while the previous page is still loading.

14.3 API and database performance

Return only needed fields, bound page size, compress supported responses at the edge, and use connection pooling. Look for repeated requests caused by client lifecycle mistakes. Batch related reads when it simplifies the contract without producing huge unbounded payloads.

Cache data with a defined key, lifetime, and invalidation rule. Public catalogue data is different from customer-specific order history. A shared cache key that omits user or tenant identity can leak private data. Do not cache authorization decisions indefinitely. Cache misses and stale data are expected conditions and need explicit behavior.

An index can improve an actual query plan; it also slows some writes and consumes memory/storage. Avoid long transactions that hold locks while waiting for network services. Use background jobs for work such as generating large exports or sending notifications, while keeping the user informed about job state.

14.4 Performance exercise

Seed 10,000 catalogue rows in a test database. Measure a first-page request, a deep offset request, and a cursor request. Inspect the plans. Then display results with a lazy list and compare a small image with a huge image. Record p50, p95, and error rate rather than only the fastest request. p95 means 95 percent of measured requests completed within that duration. Averages can hide a poor slow tail.

Do not optimize an imaginary million-user scenario before fixing a visible one-second repeated query. Preserve correctness: stale pricing or skipped validation is not an acceptable performance improvement.

Level 15 | Git, deployment, and delivery automation

15.1 Git and GitHub from the beginning

Git records versions of files and the history of changes. A repository is the project plus that history. A commit is a named snapshot with metadata. A branch is a movable reference to a line of development. GitHub hosts repositories and collaboration tools; it is not the same thing as Git.

From a new project folder, run the following only when you intend to start tracking that folder:

```
git init
git status
git add lib pubspec.yaml pubspec.lock test
git commit -m "Add first tested Flutter screen"
git switch -c feature/cart
```

Line 1 creates the repository metadata. Line 2 shows current changes. Line 3 stages selected files. Line 4 records them with a message. Line 5 creates and switches to a feature branch. Configure your own author identity if Git requests it. Review `git diff` before staging and `git diff --staged` before committing. Add secrets, build output, database files, and environment files to an appropriate ignore file.

A remote is another repository location. Fetch downloads history without automatically changing your working files. Pull fetches and integrates according to your configuration. Push sends commits to the remote. A pull request proposes a branch change for review; it is not a command that deploys an app by itself. Merge conflicts require understanding both edits, not choosing a side randomly.

15.2 Environments and configuration

Development is for rapid local work. Staging approximates production for release checks. Production serves real users. Use separate databases, credentials, provider modes, and storage for each. A test payment key must not accidentally point at a production order database.

An environment variable is a process-level configuration value. On PowerShell, `$env:API_BASE='value'` sets one for child processes in that shell. Flutter `--dart-define` embeds compile-time values in the application; it is suitable for public API base URLs, not confidential secrets. Backend environment secrets are still sensitive and must not be printed by diagnostic commands.

15.3 Docker and the local PostgreSQL stack

A container packages an application process with its user-space dependencies and runs with isolation provided by the host. An image is the template used to start containers. A volume stores data independently of a particular container's writable layer [S16]. Deleting and recreating a database container without preserving data is not a migration strategy.

The following Dockerfile uses the project root described in Level 20 as its build context. It installs requirements, copies the backend, and runs as a non-root user. It deliberately does not copy a `.env` file. Use exact tested dependency pins and reviewed image digests for a release.

ops/Dockerfile | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

Dockerfile - lines 1-9

```
001 FROM python:3.12-slim
002 WORKDIR /app
003 COPY requirements.txt /app/requirements.txt
004 RUN pip install --no-cache-dir -r requirements.txt
005 RUN useradd --create-home appuser
006 COPY shop_api/ /app/
007 USER appuser
008 EXPOSE 8000
009 CMD ["python", "-m", "uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Line-by-line explanation

1. Run or configure this container-build instruction in the order shown.

2. Run or configure this container-build instruction in the order shown.

3. Run or configure this container-build instruction in the order shown.

4. Run or configure this container-build instruction in the order shown.

5. Run or configure this container-build instruction in the order shown.

6. Run or configure this container-build instruction in the order shown.

7. Run or configure this container-build instruction in the order shown.

8. Run or configure this container-build instruction in the order shown.

9. Run or configure this container-build instruction in the order shown.

The Compose file creates a private database service and binds the API only to your host's loopback interface. The database is reached by the service name `db` from the API container. The PostgreSQL 18 volume path follows that image's layout; older image versions can differ, so read their upgrade guidance before changing versions.

ops/compose.yaml | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

compose.yaml - lines 1-24

```
001 services:
002   db:
003     image: postgres:18
004     environment:
005       POSTGRES_DB: shirts
006       POSTGRES_USER: course
007       POSTGRES_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD}
008     volumes:
009       - shirt_data:/var/lib/postgresql
010     healthcheck:
011       test: ["CMD-SHELL", "pg_isready -U course -d shirts"]
012       interval: 5s
013       timeout: 3s
014       retries: 10
015   api:
016     build:
017       context: ..
018       dockerfile: ops/Dockerfile
019     environment:
020       DATABASE_URL: postgresql+psycopg://course:${DB_PASSWORD}@db:5432/shirts
021       JWT_SECRET: ${JWT_SECRET:?Set JWT_SECRET}
022       PAYMENT_MODE: demo
023     ports:
024       - "127.0.0.1:8000:8000"
```

Line-by-line explanation

1. Supply this named property, field, or configuration entry with the value shown here.

2. Supply this named property, field, or configuration entry with the value shown here.

3. Supply this named property, field, or configuration entry with the value shown here.

4. Supply this named property, field, or configuration entry with the value shown here.

5. Supply this named property, field, or configuration entry with the value shown here.

6. Supply this named property, field, or configuration entry with the value shown here.

7. Supply this named property, field, or configuration entry with the value shown here.

8. Supply this named property, field, or configuration entry with the value shown here.

9. Configure this nested YAML value; indentation determines which service, job, or step owns it.

10. Supply this named property, field, or configuration entry with the value shown here.

11. Supply this named property, field, or configuration entry with the value shown here.

12. Supply this named property, field, or configuration entry with the value shown here.

13. Supply this named property, field, or configuration entry with the value shown here.

14. Supply this named property, field, or configuration entry with the value shown here.

15. Supply this named property, field, or configuration entry with the value shown here.

16. Supply this named property, field, or configuration entry with the value shown here.

17. Supply this named property, field, or configuration entry with the value shown here.

18. Supply this named property, field, or configuration entry with the value shown here.

19. Supply this named property, field, or configuration entry with the value shown here.

20. Supply this named property, field, or configuration entry with the value shown here.

21. Supply this named property, field, or configuration entry with the value shown here.

22. Supply this named property, field, or configuration entry with the value shown here.

23. Supply this named property, field, or configuration entry with the value shown here.

24. Configure this nested YAML value; indentation determines which service, job, or step owns it.

compose.yaml - lines 25-29

```
025     depends_on:
026       db:
027         condition: service_healthy
028 volumes:
029   shirt_data:
```

Line-by-line explanation

25. Supply this named property, field, or configuration entry with the value shown here.

26. Supply this named property, field, or configuration entry with the value shown here.

27. Supply this named property, field, or configuration entry with the value shown here.

28. Supply this named property, field, or configuration entry with the value shown here.

29. Supply this named property, field, or configuration entry with the value shown here.

From the source root, set DB_PASSWORD to a locally generated password safe for this demonstration connection URL, and set JWT_SECRET to a random secret. Passwords containing URL-reserved characters need URL encoding or a structured connection configuration. Then run:

```
docker compose -f ops/compose.yaml up -d db
docker compose -f ops/compose.yaml build api
docker compose -f ops/compose.yaml run --rm api python db.py
docker compose -f ops/compose.yaml up -d api
docker compose -f ops/compose.yaml logs api
```

Line 1 starts the database. Line 2 builds the backend image. Line 3 explicitly initializes the learning schema in that database. Line 4 starts the API. Line 5 reads its logs. Compose's health dependency waits for database readiness. This learning initialization is replaced by reviewed migrations in a production pipeline. Do not use `down -v` unless you intend to delete the named volume and its data.

15.4 CI/CD and a complete backend workflow

Continuous integration, CI, runs checks when code changes. Continuous delivery prepares a releasable artifact. Continuous deployment goes further and releases automatically according to policy. A pipeline is a sequence of jobs, not a substitute for meaningful tests.

Copy this workflow to `.github/workflows/backend.yml` in a repository whose root matches the Level 20 folder structure. It installs Python dependencies and runs the disposable backend tests [S17]. It does not deploy or need production credentials.

ops/ci.yml | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

ci.yml - lines 1-15

```
001 name: Backend learning checks
002 on: [push, pull_request]
003 permissions:
004   contents: read
005 jobs:
006   backend:
007     runs-on: ubuntu-latest
008     steps:
009       - uses: actions/checkout@v4
010       - uses: actions/setup-python@v5
011         with:
012           python-version: '3.12'
013       - run: python -m pip install -r requirements.txt
014       - run: python -m pytest -q
015         working-directory: shop_api
```

Line-by-line explanation

1. Supply this named property, field, or configuration entry with the value shown here.

2. Supply this named property, field, or configuration entry with the value shown here.

3. Supply this named property, field, or configuration entry with the value shown here.

4. Supply this named property, field, or configuration entry with the value shown here.

5. Supply this named property, field, or configuration entry with the value shown here.

6. Supply this named property, field, or configuration entry with the value shown here.

7. Configure this nested YAML value; indentation determines which service, job, or step owns it.

8. Supply this named property, field, or configuration entry with the value shown here.

9. Configure this nested YAML value; indentation determines which service, job, or step owns it.

10. Configure this nested YAML value; indentation determines which service, job, or step owns it.

11. Supply this named property, field, or configuration entry with the value shown here.

12. Configure this nested YAML value; indentation determines which service, job, or step owns it.

13. Configure this nested YAML value; indentation determines which service, job, or step owns it.

14. Configure this nested YAML value; indentation determines which service, job, or step owns it.

15. Configure this nested YAML value; indentation determines which service, job, or step owns it.

Pin third-party actions to reviewed immutable commit identifiers in a hardened release workflow. Use least-privileged workflow permissions, protected environments, and short-lived cloud credentials where supported. Do not expose secrets to untrusted fork code. Add dependency auditing and a real PostgreSQL integration job when relying on PostgreSQL-specific behavior.

For Flutter, a corresponding check job should install the tested Flutter SDK, run `flutter pub get`, `flutter analyze`, and `flutter test`, then build the target artifact. Store the tested SDK version and lockfile. "Latest" in every build can make two commits behave differently without any source change.

15.5 Android, iOS, and web releases

Android release work includes an application ID, version/build number, permissions, signing configuration, and a release app bundle. `flutter build appbundle` creates the usual store-distribution artifact after configuration. Keep signing credentials out of the repository and follow the current Android release checklist [S18]. Test the release build on real devices.

iOS release work uses the supported macOS/Xcode toolchain, bundle identifier, signing/provisioning, capabilities, and distribution workflow. `flutter build ipa` is part of the build path after setup, not a replacement for account and signing requirements [S19]. Test lifecycle, permissions, and external checkout behavior on devices.

Flutter web builds with `flutter build web`. Hosting must serve the generated static assets, use HTTPS, and route application URLs appropriately for your routing strategy. Browser caching can leave old assets or incompatible clients active, so plan version compatibility between app and API [S20]. Do not place backend keys into web configuration.

15.6 Release, rollback, and maintenance

A release should identify the code revision, dependency set, migration version, configuration, and built artifacts. Run smoke checks after deployment: health, login, product read, a controlled test order in the correct environment, and monitoring. Use staged rollout where the platform supports it.

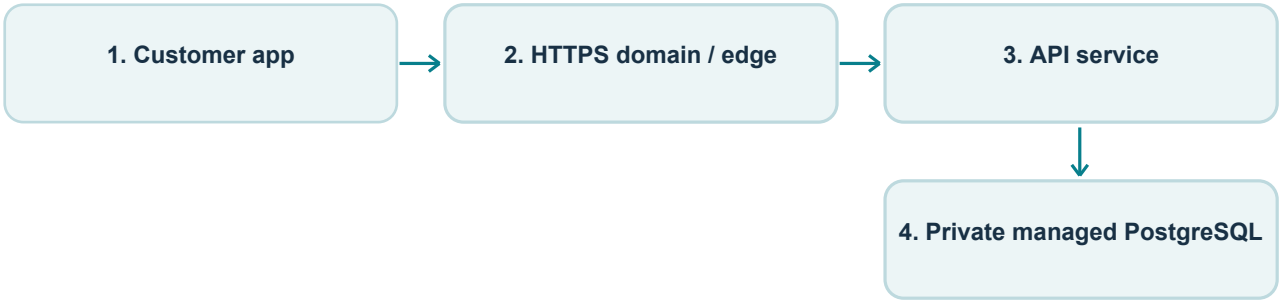
Rollback means restoring a previous working application version, but a database migration may make that version incompatible. Design compatible changes and retain rollback artifacts. After an incident, document the cause, detection gap, mitigation, and a specific prevention test. Maintenance also includes dependency updates, certificate rotation, backup verification, and reviewing unused permissions.

Level 16 | Cloud infrastructure without unnecessary services

16.1 The minimum useful deployment

A server is a computer or process serving requests. Cloud services rent compute, storage, networking, and managed capabilities. A domain is a human-readable name. DNS maps names to routing information. HTTPS protects connections using certificates and TLS. Object storage holds files such as product pictures; it is different from a relational database.

For an early commercial shop, a reasonable starting architecture is a hosted API service, a managed PostgreSQL database, object storage for images, an HTTPS endpoint, secret management, and monitoring. A CDN, or content delivery network, caches suitable content closer to users. It helps public images and static files; it does not make private order data safely public.



Keep the database off the public internet where practical and allow only required application access. Give the application a least-privileged database account. Use managed backups with tested restoration. Separate environments and accounts/projects according to your team's isolation requirements.

16.2 AWS, Azure, and Google Cloud

AWS, Azure, and Google Cloud each offer compute, managed relational databases, object storage, identity, networking, and monitoring. Learn the capability before memorizing every branded service. A managed container service can run the backend image; a managed PostgreSQL service can host the database; object storage can hold product images.

A virtual machine gives more operating-system control and more maintenance responsibility. A managed application/container platform reduces some operational work but imposes its own limits and configuration. Serverless functions can fit event handlers, but request limits, cold starts, database connections, and transaction design still matter. Select a service based on the actual workload and team capacity.

Do not assume hosting a database in the cloud makes its data safe. Configure access, backups, encryption, upgrades, monitoring, and retention. Budget for network egress, storage growth, logs, and backup copies, not only CPU.

16.3 Deployment exercise

Draw the public and private boundaries for your chosen provider. Identify who can access the database, where secrets live, how certificates renew, where images are uploaded, and who receives alerts. Deploy the API to staging using a test database, run migrations as a separate controlled step, and verify the client uses that staging URL.

For direct-to-object-storage uploads, use short-lived scoped upload authorization generated by the backend, validate allowed type and size, and check the resulting object before publication. An uploaded filename should not become an arbitrary filesystem path. Serve public product images separately from private customer documents.

Level 17 | Monitoring and production operation

17.1 Logs, metrics, traces, and crash reports

A log records an event. A metric is a numeric measurement over time. A trace follows a request across components. A crash report records a client failure with diagnostic context. Use each for the question it answers rather than logging every internal value indiscriminately.

Give requests correlation identifiers so an API error can be connected to relevant logs. Measure request count, latency, error rate, and saturation of critical resources. Track database connection use, slow queries, lock waits, disk usage, and replication/backup health as applicable. Client crash reports should include app version and safe diagnostic context without leaking tokens or personal data.

17.2 Define useful service indicators

An SLI, service-level indicator, is a measured behavior such as successful checkout availability. An SLO, service-level objective, is its target over a defined window. Define the numerator, denominator, exclusions, and measurement source. A healthy `/health` response does not prove that customers can pay.

Business indicators include orders stuck awaiting confirmation, provider payments without local paid orders, refund failures, negative stock attempts, and notification backlogs. These can reveal failures that CPU charts miss. Alert on actionable symptoms and sustained threshold breaches, not every harmless retry.

17.3 Incident response and reconciliation

When an alert fires, first determine scope and impact. Preserve evidence, stop a harmful operation if necessary, and communicate a factual status. Use a runbook: a written sequence for diagnosing and mitigating a known class of failure. Avoid making several unrelated changes while investigating.

For payment reconciliation, compare provider transactions with local records by immutable identifiers, amount, currency, and state. An unmatched payment needs investigation, not deletion. Re-run idempotent processing where appropriate and record manual interventions. For inventory, compare movements and reservations instead of guessing a new stock number.

17.4 Backups are an ongoing operation

Monitor backup success and age, but also schedule restore drills. A backup that cannot be decrypted or whose credentials were lost is not useful. Document recovery owners, access, steps, expected duration, and data-loss assumptions. Keep recovery access secure but usable during an outage.

Exercise: stop the local backend, observe the client's error state, restore service, and retry without creating a duplicate order. Then simulate a webhook processing failure and replay the same event. Write a short

incident note explaining what the customer sees, what is known, and what still needs verification.

Level 18 | Scalability and practical architecture

18.1 Start with a modular monolith

A monolith is an application deployed as one unit. Modular means its internal responsibilities have clear boundaries. A modular monolith can contain auth, catalogue, inventory, orders, and payments modules without turning each into a separate network service. This is often easier to develop, test, deploy, and operate for a small team.

Microservices split capabilities into independently deployed services. They can support independent scaling and team ownership, but introduce network failure, distributed tracing, cross-service consistency, deployment coordination, and data ownership problems. They are not a prize awarded for writing professional code.

Extract a service when there is evidence: a distinct scaling need, a clear business boundary, a team that can own it, and operational capability to support it. Moving every Python file into a different container does not produce good boundaries. Shared database writes across nominal services can leave you with the complexity of distribution and the coupling of a monolith.

18.2 Load balancing, scaling, and availability

Vertical scaling gives a server more resources. Horizontal scaling adds instances. A load balancer distributes requests among healthy instances. Stateless application instances are easier to replace because session and business state live in shared durable systems rather than one process's memory.

The capstone stores sessions and orders in a database, but its operational design still needs connection limits, migrations, rate limits, and idempotency under multiple instances. A local in-memory queue or per-process rate limiter will not become shared automatically when you add replicas.

High availability reduces downtime through redundancy and failure handling. Multiple API replicas do not help if every replica depends on one unprotected database or a broken shared configuration. Disaster recovery plans for larger failures, including data restoration and regional outage. State the acceptable recovery time and lost data; "highly available" without a target is vague.

18.3 Redis, caching, and message queues

Redis is an in-memory data service often used for caches, counters, and coordination patterns. It is not automatically the right source of truth for orders. Define persistence, eviction, failure behavior, and recovery before storing critical state. A cache entry expiring should not erase the only record of a payment.

A message queue stores work for later processing. A producer publishes a message; a consumer handles it. Acknowledgement tells the queue that work has been handled. Many practical delivery systems can deliver a message more than once, so consumers need idempotency. A dead-letter queue holds work that repeatedly failed for inspection and controlled replay.

Example: after an order becomes paid, send a receipt. If you commit the order but crash before publishing the message, the receipt intent is lost. An outbox solves this by recording a message intent in the same database transaction as the paid transition. A worker publishes pending intents, marks progress, and tolerates duplicates. The recipient consumer deduplicates by a stable event ID.



18.4 API gateway and event-driven design

An API gateway is an entry layer that can route, enforce selected cross-cutting policies, and observe traffic. It does not replace business authorization in services. A simple hosted reverse proxy may be sufficient before you need a specialized gateway.

Event-driven architecture lets components react to recorded facts such as OrderPaid. An event should name something that happened, carry a stable ID and schema version, and contain enough safe information for its consumers. A command asks for something to happen, such as RefundPayment; it may be rejected. Confusing commands and facts can lead to consumers treating unvalidated intent as confirmed truth.

Exercise: design an outbox row with event ID, aggregate ID, type, version, payload, created time, and delivery state. Explain recovery if the worker crashes after publishing but before marking the row sent. "Exactly once" is not a useful claim unless you define the boundary and prove how effects are deduplicated.

Level 19 | Advanced architecture in plain language

19.1 Domain-driven design (DDD)

DDD organizes software around the language and rules of the business. Start by talking precisely about product, variant, reservation, capture, cancellation, and refund. A bounded context is a boundary within which a model and its words have a clear meaning. "Customer" in support and "customer" in billing may contain different information and rules.

An aggregate is a consistency boundary managed through a root. An Order aggregate can ensure its line totals and allowed transitions remain consistent. It should not load the entire business graph for every operation. A value object such as Money combines amount and currency and enforces valid values.

The following runnable Python file demonstrates a small domain rule and a read projection. It is an in-memory learning example, not a complete event store or accounting ledger.

advanced/order_domain.py | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

order_domain.py - lines 1-24

```
001 from dataclasses import dataclass
002
003 @dataclass(frozen=True)
004 class Money:
005     paise: int
006     currency: str = 'INR'
007     def __post_init__(self):
008         if self.paise < 0:
009             raise ValueError('Money must not be negative')
010
011 @dataclass(frozen=True)
012 class Paid:
013     order_id: int
014     amount: Money
015
016 class Order:
017     def __init__(self, order_id: int, amount: Money):
018         self.id, self.amount, self.status = order_id, amount, 'awaiting_payment'
019     def record_capture(self, captured: Money) -> Paid:
020         if self.status != 'awaiting_payment':
021             raise ValueError('Order is not awaiting payment')
022         if captured != self.amount:
023             raise ValueError('Captured amount does not match')
024         self.status = 'paid'
```

Line-by-line explanation

1. Import the named library or module so the following code can use its types and functions.

2. Blank line separating related pieces of code; it performs no operation.

3. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

4. Declare class Money and begin the fields or methods that define this type.

5. Supply this named property, field, or configuration entry with the value shown here.

6. Supply this named property, field, or configuration entry with the value shown here.

7. Define this named Python operation and its parameters. The indented body runs when the function is called.

8. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

9. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

10. Blank line separating related pieces of code; it performs no operation.

11. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

12. Declare class Paid and begin the fields or methods that define this type.

13. Supply this named property, field, or configuration entry with the value shown here.

14. Supply this named property, field, or configuration entry with the value shown here.

15. Blank line separating related pieces of code; it performs no operation.

16. Declare class Order and begin the fields or methods that define this type.

17. Define this named Python operation and its parameters. The indented body runs when the function is called.

18. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

19. Define this named Python operation and its parameters. The indented body runs when the function is called.

20. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

21. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

22. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

23. Stop normal execution and raise this specific error. FastAPI HTTPException becomes the stated HTTP response.

24. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

order_domain.py - lines 25-39

```
025         return Paid(self.id, self.amount)
026
027 def project_revenue(events: list[Paid]) -> int:
028     seen, total = set(), 0
029     for event in events:
030         if event.order_id not in seen:
031             seen.add(event.order_id)
032             total += event.amount.paise
033     return total
034
035 if __name__ == '__main__':
036     order = Order(42, Money(159800))
037     event = order.record_capture(Money(159800))
038     print(order.status)
039     print(project_revenue([event, event]))
```

Line-by-line explanation

25. Produce the stated result and finish this function call. In a build method the result is a widget description.

26. Blank line separating related pieces of code; it performs no operation.

27. Define this named Python operation and its parameters. The indented body runs when the function is called.

28. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

29. Repeat the following body once for each value in the stated sequence.

30. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

31. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

32. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

33. Produce the stated result and finish this function call. In a build method the result is a widget description.

34. Blank line separating related pieces of code; it performs no operation.

35. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

36. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

37. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

38. Write this value to the console so the lesson result can be inspected.

39. Write this value to the console so the lesson result can be inspected.

Run `python order_domain.py`. Expect `paid` and `159800`. The duplicate event does not double the projected revenue because the projection tracks order IDs in this simplified one-payment-per-order model. A real projection should deduplicate by event ID, preserve checkpoints durably, and model refunds and currencies explicitly. Try a different captured amount: the domain method rejects it before changing state.

19.2 CQRS

CQRS means command-query responsibility segregation. Separate the model used to change state from the model used to read it when the needs differ. The order command path may enforce locks and business rules, while an admin dashboard reads a precomputed daily revenue view.

You can separate code paths inside one service and database; CQRS does not require microservices or event sourcing. If a read model is updated asynchronously, users may briefly see old results. Explain that delay in the UI and define how to read authoritative state immediately after a critical operation.

Use it when complex write rules and read shapes justify the extra synchronization work. A five-field CRUD screen does not automatically need two databases and an event bus.

19.3 Event sourcing

Event sourcing stores state changes as an ordered history of events and derives current state by replaying them. Ordinary audit logging adds history beside current tables; it does not automatically make a system event-sourced. Events become long-lived data, so versioning and replay compatibility are substantial responsibilities.

For a stock item, events might be StockReceived(+10), StockReserved(-2), and ReservationReleased(+2). Replaying them gives an available balance. Correctness needs aggregate versions or another concurrency

mechanism so two writers cannot append conflicting histories. Snapshots can speed recovery but must correspond to a known event position.

Think about corrections, personal-data retention, and event evolution before adopting the pattern. A wrongly designed immutable history can preserve mistakes and sensitive information indefinitely. Use ordinary relational state plus audit/outbox tables when that meets the problem more simply.

19.4 Distributed systems and consistency

A distributed system has components communicating across a network. Messages can be delayed, duplicated, reordered, or lost; a process can fail after performing a side effect but before acknowledging it. Timeouts tell you the response did not arrive in time, not necessarily that the operation did not happen.

Strong consistency provides a defined up-to-date view under the system's model. Eventual consistency allows replicas or projections to converge after updates stop, subject to the replication design. Neither phrase replaces a business requirement. Payment confirmation needs an authoritative financial decision; a recommendation carousel may tolerate stale data.

The CAP theorem concerns a particular consistency property, linearizability, and availability of responses during network partitions. When a partition prevents coordination, a system cannot guarantee both of those properties for every request. It is not a slogan saying every database simply "chooses any two" in all circumstances. Ordinary latency, durability, and isolation choices involve additional tradeoffs.

19.5 Idempotency and sagas

An idempotent effect tolerates retry without duplicating the intended result. Keys need a scope, request fingerprint, retention policy, and durable outcome. Deleting deduplication records too soon can make a delayed retry create another order. A key generated anew on every retry defeats the design.

A saga coordinates a sequence of local transactions across services. Each step can have a compensating action. Reserving stock, collecting payment, and requesting shipment may require compensation if a later step fails. Compensation is not time travel: refunding a payment may take time and incur a different outcome from never charging it.

Model states such as reserving, reserved, charging, charged, shipment_requested, and compensation_pending. Persist progress and retry idempotently. Decide who coordinates and how stuck workflows are detected. A catch block that calls three undo functions without recording progress is not a robust saga.

19.6 Architecture Decision Records (ADRs)

An ADR records a significant design decision with context, alternatives, consequences, and a review trigger. For the shop, write: "Use a modular monolith because one team owns the product, transactions span inventory and orders, and current load does not justify distributed coordination." Record the downside: deployments scale together. Define a review trigger based on measured load or team boundaries.

Do not write an ADR as advertising. It should help a future maintainer understand why a reasonable alternative was rejected and when that reasoning may no longer apply.

19.7 Multi-tenant and multi-region systems

A tenant is a separate customer organization sharing a service. Multi-tenancy needs isolation in every query, unique constraint, cache key, file path, job, and audit record. A tenant ID accepted from a request is not proof of membership. Determine it from authenticated authorization context and validate access. Options include shared tables with tenant keys, separate schemas, or separate databases, each with operational tradeoffs.

Multi-region deployment places systems in more than one geographical region. Benefits can include latency and resilience; costs include replication lag, conflict handling, data-residency requirements, and more complex failover. Decide where authoritative writes occur and what happens during a partition before advertising active-active operation.

Exercise: explain how a customer in region B sees a paid order created in region A while replication is delayed. Then explain a failover during stock reservation. If you cannot state the correctness rule, adding another region has outrun the design.

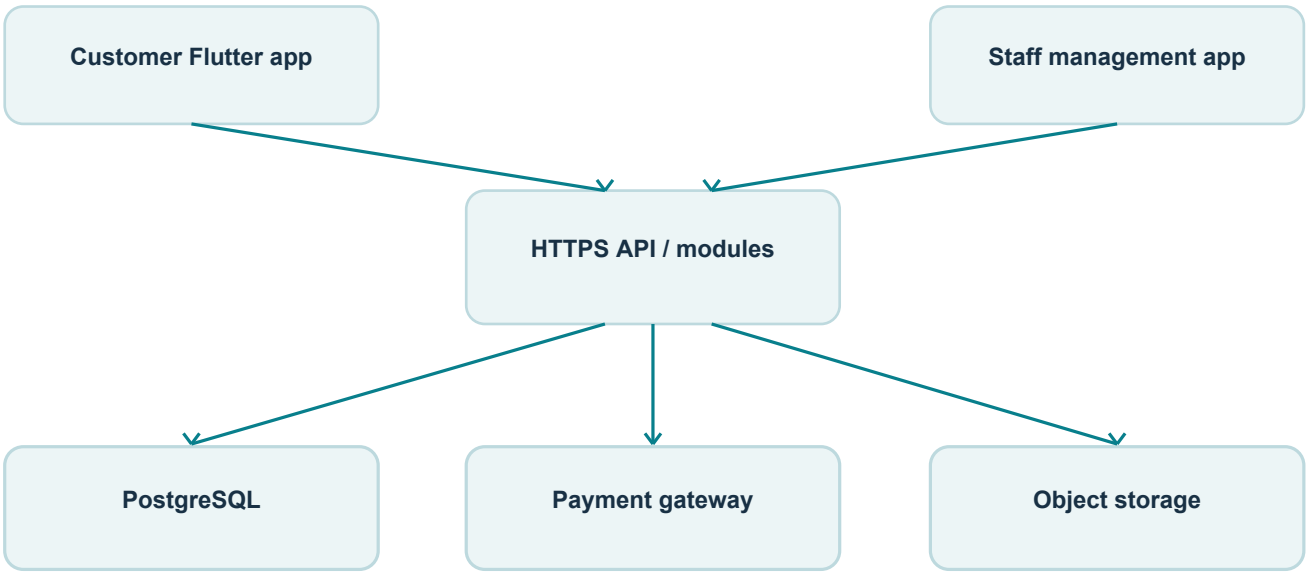
Level 20 | Final shirts platform: run, understand, extend

20.1 What this executable reference does

The reference connects a Flutter customer or management interface to FastAPI and a relational database. Customers can register, sign in, search seeded shirt variants, adjust a local cart, provide an address, create an idempotent order, inspect history, simulate payment, cancel eligible unpaid orders, and sign out. Staff can inspect orders, receive stock, and perform a demo full refund. The backend includes optional server-side Razorpay order creation and captured-event verification.

This is a runnable teaching baseline, not a claim that every commercial feature is finished. The notes explain the expansion to catalogue editing, address books, coupons, notifications, fulfillment, complete refunds, secure native persistence, production migrations, and operations. Those features need implementation and acceptance tests for your actual business. A production launch also requires real provider, device, concurrency, recovery, and security validation.

The final architecture corrects a common diagram mistake: the database is not downstream of the payment gateway, and Redis is not downstream of PostgreSQL in a universal request chain. The backend coordinates its database and external services as separate dependencies.



Monitoring, secrets, backups, and CI/CD support every deployed component.

Learning diagram - component relationships, not a deployment screenshot.

20.2 Folder structure and responsibilities

```
flutter_notes_source/
  requirements.txt
  requirements-tested.txt
  flutter_app/
    lib/main.dart
    lib/examples/
    test/widget_test.dart
    pubspec.yaml
    pubspec.lock
    web/
  flutter_examples/
  employee_api/main.py
  shop_api/
    db.py
    auth.py
    main.py
    payments.py
    promote_admin.py
    test_shop.py
  advanced/order_domain.py
  ops/Dockerfile
  ops/compose.yaml
  ops/ci.yml
```

The Flutter `main.dart` is the connected shop. `lib/examples/` contains independent earlier entrypoints. `flutter_examples/` preserves the annotated source listings used in the book. The backend is divided by responsibility, although its domain rules remain compact for teaching. The operations folder contains local container and CI examples. The tested requirements file records the environment used for this edition rather than a promise about all future dependency versions.

20.3 Exact local run sequence

Create a writable folder named flutter_notes_source. Reproduce the folder structure above and save the code listings into their named files. Use Python 3.12 for the recorded backend environment. The dependency files are printed below; no separate archive is needed.

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements-tested.txt
$env:JWT_SECRET = .venv\Scripts\python.exe -c "import secrets; print(secrets.token_urlsafe(48))"
$env:PAYMENT_MODE = 'demo'
cd shop_api
..\.venv\Scripts\python.exe db.py
..\.venv\Scripts\python.exe -m uvicorn main:app --reload
```

Line 1 creates the isolated environment. Line 2 installs the recorded dependencies. Line 3 generates a secret in this shell without writing it into source. Line 4 enables clearly labelled fake payments. Line 5 chooses the backend folder. Line 6 creates the local learning schema and seeds two variants. Line 7 starts the API. Keep this terminal running.

Before starting Flutter for the first time, create its generated platform files from the project root:

```
flutter create --project-name notes_lab flutter_app
cd flutter_app
flutter pub add http:1.6.0 shared_preferences:2.5.5 flutter_riverpod:3.3.2
```

Replace lib/main.dart with the connected-shop listing below. To run earlier lessons, create lib/examples and copy their full listings there under the printed filenames. Replace the generated widget test with the counter test shown in Level 13 after saving counter.dart under lib/examples. The package name notes_lab matches its import. Flutter generates the web wrapper, platform files, and lockfile.

In a second terminal, open `flutter_app`:

```
flutter pub get
flutter run -d chrome --web-port 5173
```

The first command resolves the committed package lock with a compatible Flutter SDK. The second launches the web client at the development origin allowed by the API. For an Android emulator, create/enable the Android target in your own project and set the API base using `--dart-define=API_BASE=http://10.0.2.2:8000`. Local cleartext HTTP is a development choice requiring appropriate platform configuration; release uses HTTPS. A real phone needs the host's reachable address and network setup.

After saving an earlier listing under lib/examples in this project, use `flutter run -d chrome -t lib/examples/calculator.dart`, replacing the target filename as desired. Only run the Employee client when its matching Employee API is listening. Both APIs default to port 8000, so stop one before starting the other or configure different ports and base URLs.

Dependency files to create

Save the following flexible dependency list as requirements.txt in the project root. It is used by the introductory setup and container example.

requirements.txt

```
fastapi>=0.115,<1
uvicorn>=0.30,<1
sqlalchemy>=2.0,<3
psycopg[binary]>=3.2,<4
pwdlib[argon2]>=0.2,<1
PyJWT>=2.9,<3
httpx>=0.27,<1
pytest>=8,<10
```

For reproducing the backend checks for this edition, save the next exact list as requirements-tested.txt. These versions were resolved and tested on Python 3.12. Review updates separately rather than silently changing a working learning environment.

requirements-tested.txt

```
annotated-doc==0.0.5
annotated-types==0.8.0
anyio==4.15.1
argon2-cffi-bindings==26.1.0
argon2-cffi==25.1.0
certifi==2026.7.22
cffi==2.1.1
click==8.5.0
colorama==0.4.6
fastapi==0.142.2
h11==0.16.0
httpcore==1.0.9
httpx==0.28.1
idna==3.20
iniconfig==2.3.0
opentelemetry-api==1.45.0
packaging==26.3
pluggy==1.6.0
psycopg-binary==3.3.6
psycopg==3.3.6
pwdlib==0.3.1
pycparser==3.0
pydantic==2.13.5
pydantic_core==2.46.5
Pygments==2.21.0
PyJWT==2.15.1
pytest==9.1.1
SQLAlchemy==2.1.3
starlette==1.7.0
typing-inspection==0.4.4
typing_extensions==4.16.0
tzdata==2026.5
uvicorn==0.54.0
```

20.4 Complete connected Flutter client

This listing is intentionally a compact reference you can trace end to end. It contains HTTP setup, login/session state, cart intent, retry keys, and both customer and management views. Later refactor it into typed models, repositories, and Riverpod controllers while preserving its API behavior and tests.

flutter_examples/connected_shop.dart | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

connected_shop.dart - lines 1-24

```
001 import 'dart:convert';
002 import 'dart:math';
003 import 'package:flutter/material.dart';
004 import 'package:http/http.dart' as http;
005 const base = String.fromEnvironment('API_BASE', defaultValue: 'http://127.0.0.1:8000');
006 const adminApp = bool.fromEnvironment('ADMIN_APP', defaultValue: false);
007 void main() => runApp(MaterialApp(theme: ThemeData(colorSchemeSeed: Colors.teal), home:
    const ConnectedShop()));
008 class ApiFailure implements Exception {
009   final int statusCode;
010   final String body;
011   ApiFailure(this.statusCode, this.body);
012   @override
013   String toString() => 'HTTP $statusCode: $body';
014 }
015 class ConnectedShop extends StatefulWidget {
016   const ConnectedShop({super.key});
017   @override
018   State<ConnectedShop> createState() => _ConnectedShopState();
019 }
020 class _ConnectedShopState extends State<ConnectedShop> {
021   final email = TextEditingController();
022   final password = TextEditingController();
023   final address = TextEditingController();
024   final search = TextEditingController();
```

Line-by-line explanation

1. JSON conversion for the API boundary.

2. Secure randomness for idempotency keys.

3. Import the named library or module so the following code can use its types and functions.

4. Import the named library or module so the following code can use its types and functions.

5. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

6. Build the management view separately with a compile-time flag.

7. Declare the method and its return type. Its parameters describe the information supplied by the caller.

8. Declare class ApiFailure implements Exception and begin the fields or methods that define this type.

9. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

10. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

11. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

12. Tell the Dart analyzer this declaration implements or replaces an inherited member.

13. Declare a typed field or value and initialize it from the expression on the right.

14. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

15. Declare class ConnectedShop extends StatefulWidget and begin the fields or methods that define this type.

16. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

17. Tell the Dart analyzer this declaration implements or replaces an inherited member.

18. Declare the method and its return type. Its parameters describe the information supplied by the caller.

19. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

20. Declare class _ConnectedShopState extends State<ConnectedShop> and begin the fields or methods that define this type.

21. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

22. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

23. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

24. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

connected_shop.dart - lines 25-48

```
025   final client = http.Client();
026   String? token;
027   String? refreshToken;
028   String? orderKey;
029   Map<String, Object>? pendingBody;
030   final cart = <int, int>{};
031   List<Map<String, dynamic>> products = [], orders = [];
032   bool busy = false;
033   String message = 'Sign up, then sign in. Demo payments move no money.';
034   String money(int paise) => 'INR ${(paise / 100).toStringAsFixed(2)}';
035   Future<dynamic> api(String method, String path, {Object? body, String? key, bool retry =
    true}) async {
036     final request = http.Request(method, Uri.parse('$base$path'));
037     request.headers['Content-Type'] = 'application/json';
038     if (token != null) request.headers['Authorization'] = 'Bearer $token';
039     if (key != null) request.headers['Idempotency-Key'] = key;
040     if (body != null) request.body = jsonEncode(body);
041     final stream = await client.send(request).timeout(const Duration(seconds: 15));
042     final response = await http.Response.fromStream(stream).timeout(const Duration(seconds:
     15));
043     if (response.statusCode == 401 && retry && refreshToken != null && path != '/refresh')
    {
044       final refreshed = await api('POST', '/refresh', body: {'refresh_token':
    refreshToken}, retry: false);
045       token = refreshed['access_token'] as String;
046       refreshToken = refreshed['refresh_token'] as String;
047       return api(method, path, body: body, key: key, retry: false);
048     }
```

Line-by-line explanation

25. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

26. Access token stays in memory in this web-friendly learning client.

27. Native persistence is taught separately; this demo signs in after restart.

28. Retain this exact key when retrying an uncertain submission.

29. Freeze the request so the same key never describes changed input.

30. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

31. Declare a typed field or value and initialize it from the expression on the right.

32. Declare a typed field or value and initialize it from the expression on the right.

33. Declare a typed field or value and initialize it from the expression on the right.

34. Declare a typed field or value and initialize it from the expression on the right.

35. Declare the method and its return type. Its parameters describe the information supplied by the caller.

36. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

37. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

38. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

39. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

40. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

41. Bound the wait for response headers too.

42. Bound reading the body.

43. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

44. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

45. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

46. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

47. Retry once with the same operation and idempotency key.

48. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

connected_shop.dart - lines 49-72

```
049     if (response.statusCode >= 400) throw ApiFailure(response.statusCode, response.body);
050     return response.body.isEmpty ? null : jsonDecode(response.body);
051   }
052   Future<void> action(Future<void> Function() work) async {
053     if (busy) return;
054     setState(() { busy = true; message = ''; });
055     try { await work(); } catch (error) { message = '$error'; }
056     if (mounted) setState(() => busy = false);
057   }
058   List<Map<String, dynamic>> rows(dynamic value) => (value as List).map((x) => Map<String,
    dynamic>.from(x as Map)).toList();
059   Future<void> reload() async {
060     products = rows(await api('GET',
    '/products?q=${Uri.encodeQueryComponent(search.text)}'));
061     orders = rows(await api('GET', adminApp ? '/admin/orders' : '/orders'));
062   }
063   Future<void> authenticate(bool signup) => action(() async {
064     final result = await api('POST', signup ? '/signup' : '/login',
065       body: {'email': email.text.trim(), 'password': password.text});
066     if (signup) { message = 'Account created. Sign in now.'; return; }
067     token = result['access_token'] as String;
068     refreshToken = result['refresh_token'] as String;
069     if (mounted) password.clear();
070     await reload();
071   });
072   Future<void> checkout() => action(() async {
```

Line-by-line explanation

49. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

50. Produce the stated result and finish this function call. In a build method the result is a widget description.

51. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

52. Declare the method and its return type. Its parameters describe the information supplied by the caller.

53. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

54. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

55. Begin an operation whose failures will be handled by the following exception handler.

56. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

57. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

58. Declare a typed field or value and initialize it from the expression on the right.

59. Declare the method and its return type. Its parameters describe the information supplied by the caller.

60. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

61. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

62. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

63. Declare the method and its return type. Its parameters describe the information supplied by the caller.

64. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

65. Supply this named property, field, or configuration entry with the value shown here.

66. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

67. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

68. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

69. Do not retain the password longer than needed.

70. Wait for this asynchronous operation before continuing the current function; other event-loop work can still run.

71. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

72. Declare the method and its return type. Its parameters describe the information supplied by the caller.

connected_shop.dart - lines 73-96

```
073     if (pendingBody == null) {
074       if (cart.isEmpty || address.text.trim().length < 10) throw Exception('Add a shirt and
     a complete address.');
075       orderKey = List.generate(24, (_) => Random.secure().nextInt(256)).map((n) =>
    n.toRadixString(16).padLeft(2, '0')).join();
076       pendingBody = {'address': address.text.trim(), 'lines': cart.entries.map((e) =>
    {'variant_id': e.key, 'quantity': e.value}).toList()};
077     }
078     dynamic result;
079     try {
080       result = await api('POST', '/orders', body: pendingBody, key: orderKey);
081     } on ApiFailure catch (error) {
082       if (error.statusCode == 422 || error.statusCode == 404 ||
083           (error.statusCode == 409 && error.body.contains('Insufficient stock'))) {
084         pendingBody = null; orderKey = null;
085       }
086       rethrow;
087     }
088     cart.clear(); pendingBody = null; orderKey = null;
089     message = 'Order ${result['id']} created. Choose demo payment below.';
090     await reload();
091   });
092   Future<void> logout() => action(() async {
093     await api('POST', '/logout');
094     token = null; refreshToken = null; cart.clear(); orders = []; products = [];
095     pendingBody = null; orderKey = null;
096     message = 'Signed out';
```

Line-by-line explanation

73. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

74. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

75. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

76. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

77. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

78. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

79. Begin an operation whose failures will be handled by the following exception handler.

80. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

81. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

82. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

83. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

84. These server responses prove this input was not committed, so editing is safe.

85. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

86. Unknown outcomes keep the frozen request and original retry key.

87. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

88. Clear only after the server confirms the result.

89. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

90. Wait for this asynchronous operation before continuing the current function; other event-loop work can still run.

91. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

92. Declare the method and its return type. Its parameters describe the information supplied by the caller.

93. Revoke the server session before clearing local state.

94. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

95. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

96. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

connected_shop.dart - lines 97-120

```
097   });
098   @override
099   void dispose() { email.dispose(); password.dispose(); address.dispose();
    search.dispose(); client.close(); super.dispose(); }
100   @override
101   Widget build(BuildContext context) => Scaffold(
102     appBar: AppBar(title: Text(adminApp ? 'Shirt Studio Management' : 'Shirt Studio'),
103       actions: [if (token != null) TextButton(onPressed: busy ? null : logout, child: const
     Text('Sign out'))]),
104     body: ListView(padding: const EdgeInsets.all(20), children: [
105       if (busy) const LinearProgressIndicator(),
106       if (message.isNotEmpty) Text(message),
107       if (token == null) ...[
108         TextField(controller: email, decoration: const InputDecoration(labelText:
    'Email')),
109         TextField(controller: password, obscureText: true, decoration: const
    InputDecoration(labelText: 'Password (12+ characters)')),
110         FilledButton(onPressed: busy ? null : () => authenticate(false), child: const
    Text('Sign in')),
111         TextButton(onPressed: busy ? null : () => authenticate(true), child: const
    Text('Create customer account')),
112       ] else ...[
113         TextField(controller: search, decoration: const InputDecoration(labelText: 'Search
    shirts')),
114         OutlinedButton(onPressed: busy ? null : () => action(reload), child: const
    Text('Search / reload')),
115         for (final p in products) Card(child: ListTile(
116           leading: const Icon(Icons.checkroom),
117           title: Text('${p['name']} / ${p['size']} / ${p['colour']}'),
118           subtitle: Text('${money(p['price_paise'] as int)} | Stock ${p['stock']}'),
119           trailing: adminApp
120             ? IconButton(icon: const Icon(Icons.inventory), tooltip: 'Receive one unit',
```

Line-by-line explanation

97. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

98. Tell the Dart analyzer this declaration implements or replaces an inherited member.

99. Declare the method and its return type. Its parameters describe the information supplied by the caller.

100. Tell the Dart analyzer this declaration implements or replaces an inherited member.

101. Declare the method and its return type. Its parameters describe the information supplied by the caller.

102. Supply this named property, field, or configuration entry with the value shown here.

103. Supply this named property, field, or configuration entry with the value shown here.

104. Supply this named property, field, or configuration entry with the value shown here.

105. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

106. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

107. Spread the login widgets into this child list.

108. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

109. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

110. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

111. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

112. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

113. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

114. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

115. Repeat the following body once for each value in the stated sequence.

116. Supply this named property, field, or configuration entry with the value shown here.

117. Supply this named property, field, or configuration entry with the value shown here.

118. Supply this named property, field, or configuration entry with the value shown here.

119. Supply this named property, field, or configuration entry with the value shown here.

120. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

connected_shop.dart - lines 121-144

```
121                 onPressed: busy ? null : () => action(() async { await api('PATCH', '/admin/stock/${p['id']}', body: {'delta': 1}); await reload(); }))
122             : IconButton(icon: const Icon(Icons.add_shopping_cart), tooltip: 'Add to cart',
123                 onPressed: busy || pendingBody != null ? null : () => setState(() { final id = p['id'] as int; cart[id] = (cart[id] ?? 0) + 1; }))))),
124         if (!adminApp) ...[
125           const Text('Cart (server recalculates the price)'),
126           for (final entry in cart.entries) ListTile(title: Text('Variant ${entry.key} x${entry.value}'),
127             trailing: IconButton(icon: const Icon(Icons.remove_circle_outline), onPressed: busy || pendingBody != null ? null : () => setState(() {
128               if (entry.value == 1) { cart.remove(entry.key); } else { cart[entry.key] = entry.value - 1; }
129             }))),
130           TextField(controller: address, enabled: pendingBody == null, decoration: const InputDecoration(labelText: 'Delivery address')),
131           FilledButton(onPressed: busy ? null : checkout, child: Text(pendingBody == null ? 'Create order' : 'Retry same order')),
132           if (pendingBody != null) const Text('Input is frozen until this attempt is resolved. Do not start another order after a timeout.'),
133         ],
134         const Text('Orders'),
135         for (final order in orders) Card(child: Padding(padding: const EdgeInsets.all(12), child: Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
136           Text('Order ${order['id']} | ${order['status']} | ${money(order['total_paise'] as int)}'),
137           if (!adminApp && order['status'] == 'awaiting_payment') Wrap(children: [
138             TextButton(onPressed: busy ? null : () => action(() async { await api('POST', '/orders/${order['id']}/simulate-payment'); await reload(); }), child: const Text('Simulate payment (no money)')),
139             TextButton(onPressed: busy ? null : () => action(() async { await api('POST', '/orders/${order['id']}/cancel'); await reload(); }), child: const Text('Cancel unpaid order')),
140           ]),
141           if (adminApp && order['status'] == 'paid') TextButton(onPressed: busy ? null : () => action(() async {
142             await api('POST', '/admin/orders/${order['id']}/demo-refund'); await reload();
143           }), child: const Text('Demo full refund')),
144         ]))),
```

Line-by-line explanation

121. Supply this named property, field, or configuration entry with the value shown here.

122. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

123. Supply this named property, field, or configuration entry with the value shown here.

124. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

125. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

126. Repeat the following body once for each value in the stated sequence.

127. Supply this named property, field, or configuration entry with the value shown here.

128. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

129. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

130. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

131. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

132. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

133. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

134. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

135. Repeat the following body once for each value in the stated sequence.

136. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

137. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

138. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

139. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

140. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

141. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

142. Wait for this asynchronous operation before continuing the current function; other event-loop work can still run.

143. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

144. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

connected_shop.dart - lines 145-148

```
145       ],
146     ]),
147   );
148 }
```

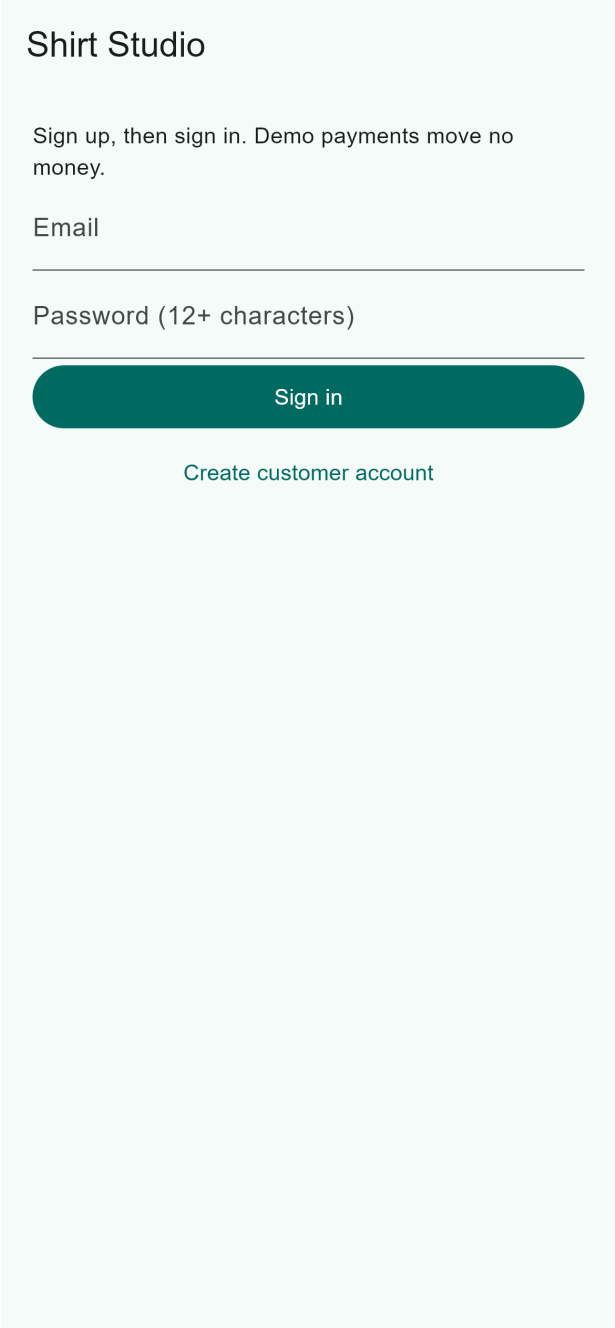
Line-by-line explanation

145. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

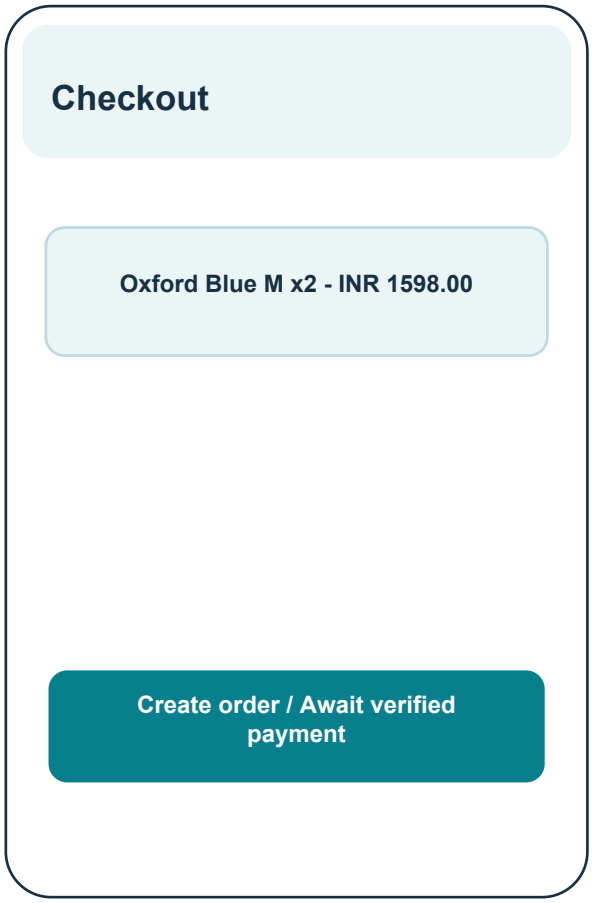
146. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

147. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

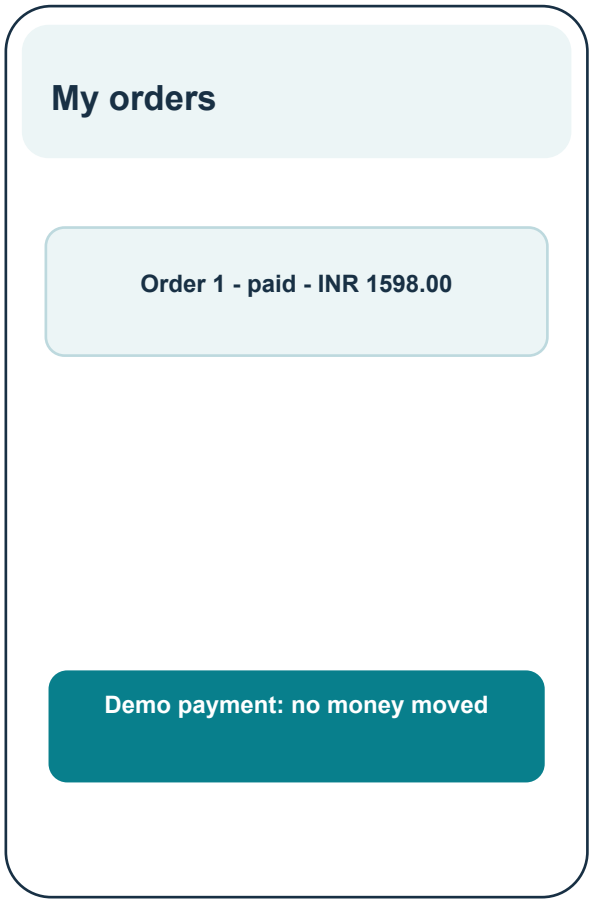
148. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.



Expected screen rendered from the Flutter example in a widget test. Fonts and platform details may differ on your device.



Expected-state illustration: shows the intended information and flow, not an exact device screenshot.



Expected-state illustration: shows the intended information and flow, not an exact device screenshot.

Important retry behavior. The client freezes the address and lines after beginning an order attempt and retains the idempotency key while the outcome is uncertain. This protects a retry in the same running session. The current draft is not persisted across process death. A production client needs durable pending-operation recovery or an API that looks up recent attempts before allowing a replacement. Definitive validation, missing-variant, and insufficient-stock responses reset the frozen request so the customer can edit. Timeouts and ambiguous conflicts preserve it; they do not automatically generate a new key.

Session behavior. The client retries once after refreshing a token. Its single busy operation prevents ordinary parallel refreshes in this small UI. A larger app with concurrent requests needs a shared refresh coordinator so all callers await one rotation. Failed refresh should lead to a dedicated signed-out state in the production UI. The example displays the failure rather than pretending the session is still usable.

20.5 Expected journey and verification

Step 1: create a customer with a unique email and a password of at least twelve characters. Expect "Account created. Sign in now." Step 2: sign in. Expect the Oxford M/Blue and Linen L/White variants. Step 3: add two Oxford units and enter a complete address. Step 4: create the order. Expect a server total of INR 1598.00 and awaiting_payment. Step 5: choose Simulate payment. Expect paid. This moves no money.

Step 6: sign out and sign back in. The order history should persist because it is stored in the database; the cart is local memory. Step 7: create another account. It should not see the first customer's orders. Step 8: inspect the API through `/docs` and try changing a requested total. The request must be rejected or the price recalculated; the client never owns pricing authority.

To try management, register a separate staff account, stop the customer browser run, promote that account locally using the supplied script, and run `flutter run -d chrome --web-port 5173 --dart-define=ADMIN_APP=true`. Sign in with the staff account. Expect orders and stock actions. Sign in with an ordinary account in the same build and expect authorization failures on admin endpoints. The server is the source of permission.

20.6 Turn the reference into your commercial project

Catalogue and images. Separate products from variants, add SKU uniqueness, protected create/edit/deactivate endpoints, image upload authorization, and accessible customer detail pages. Add pagination and filter contracts before growing the dataset.

Customer and addresses. Add verified email, password reset, profile update, address-book CRUD, and account lifecycle handling. Scope every address to its owner. Keep order snapshots independent of later edits.

Cart and pricing. Decide guest versus authenticated carts and merge behavior. Persist carts where the product requires it. Model taxes, shipping, coupons, rounding, and quote expiry on the server. Show changes before asking for payment.

Payments and refunds. Integrate actual native/web checkout for your chosen targets, test provider events, implement payment-attempt and refund tables, reconcile unknown outcomes, and support your cancellation/return policy. Never enable the demo-payment endpoint in a real merchant environment.

Fulfillment and notifications. Implement allowed warehouse transitions and shipment tracking. Write notification intents to an outbox and use idempotent workers. Store delivery outcomes and provide retry/operational tools.

Architecture and operations. Extract domain rules from HTTP handlers, add PostgreSQL concurrency tests, introduce migrations, provision staging, configure logs/metrics/alerts, test recovery, and release through a controlled pipeline. These are concrete work items with acceptance criteria, not merely topics to mark "read."

20.7 Practical project progression and completion gates

Project 1 is the counter. Gate: explain state and rebuilds. Project 2 is the calculator. Gate: validate input and handle division by zero. Project 3 is the task list. Gate: build dynamic rows and remove the correct item. Project 4 is notes. Gate: save, reload, and handle loading/error states.

Project 5 is the multi-screen local shopping UI. Gate: navigate to details and return a result. Project 6 is the local cart. Gate: add, decrement, remove, and derive totals. Project 7 connects Flutter to an API. Gate: trace a request and distinguish server rejection from network failure. Project 8 adds sign-in. Gate: explain tokens, expiry, refresh, and logout.

Project 9 is employee management. Gate: complete CRUD and search against PostgreSQL after the SQLite introduction. Projects 10-12 evolve shopping with cart, authentication, and backend orders. Gate: ownership and server pricing tests pass. Project 13 adds payment handling. Gate: duplicate/late events cannot duplicate effects. Project 14 is management. Gate: an unauthorized account cannot mutate inventory or refund.

The final project combines these capabilities and closes the listed production gaps. Finish one gate before moving to the next. You can revisit a previous layer when a later test reveals a weak assumption; that is normal engineering, not failed learning.

Level 21 | Machine learning and Flutter integration

21.1 What machine learning actually adds

1. Concept. Machine learning, or ML, means fitting a program's behavior using examples or observed data instead of writing every decision as a hand-made rule. A model is the fitted mathematical relationship. Training produces its parameters. Inference applies the fitted model to new input.

2. Why it exists. Some patterns are difficult to capture with a short list of rules. Thousands of shirt reviews use different words to express satisfaction. Product descriptions can be matched to a customer's search even when no exact product-name match exists.

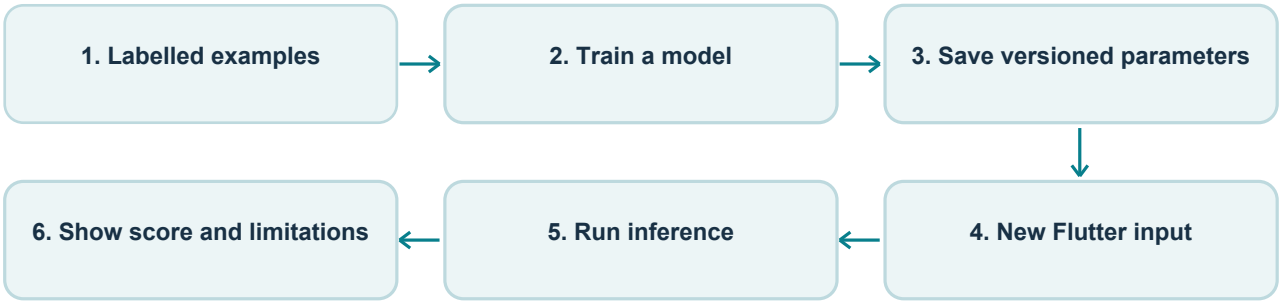
3. Why Flutter needs it. Flutter is the interface and application layer. It can send an input to a backend model, run a supported model on the device, and display an interpretable result. Flutter does not become a model-training framework merely because the app contains an "AI" button.

4. Analogy. A new shop assistant studies examples of positive and negative feedback. Later, the assistant guesses the tone of a new review. The guess can be wrong, especially when the wording differs from what was learned.

5. Small example. A model observes that soft and comfortable appear often in positive training examples, while rough and uncomfortable appear often in negative ones. It combines evidence from the words in a new review. A rule would instead explicitly say "if the review contains comfortable, label it positive." Learning word frequencies is different from hard-coding that rule.

6-7. Code and line explanations. The complete classifier, training script, API, and Flutter client appear below. Every file is included in this PDF so this chapter does not depend on a separate archive.

8. Visual.



9. Internally. Text becomes numbers, the model computes class scores, and application logic decides how to present or act on those scores. A learned prediction never grants permission, confirms a payment, or overrides a stock constraint.

10. Common mistakes. A model score is not a fact. A tiny successful demonstration is not evidence of production accuracy. Do not collect personal data merely because more data sounds useful. Start with a measurable task and a simple baseline.

11. Exercise. Identify one task where a rule is enough, one where learned prediction might help, and one where the system must use an authoritative database rule. Explain your choices.

12. Project usage. The shop can suggest similar shirts, group review themes, estimate demand, or assist staff. Pricing authority, authorization, refunds, and payment verification remain deterministic business controls.

21.2 The vocabulary you need before writing ML code

A sample is one example. A feature is an input measurement or representation, such as word counts, sleeve length, or product category. A label is the answer supplied for supervised training, such as positive/negative. A dataset is a collection of samples. A parameter is a value learned from data, such as a weight. A hyperparameter is a setting chosen outside the fitting process, such as a regularization strength or number of training epochs.

A feature vector is an ordered list of numbers. Every position must mean the same thing during training and inference. If position zero means cotton count during training but linen count during inference, valid-looking arithmetic produces nonsense. Preprocessing includes normalization, tokenization, missing-value handling, and encoding. Store or reproduce preprocessing with the model.

A training set fits parameters. A validation set helps choose settings and thresholds. A test set estimates performance after those choices are fixed. Repeatedly adjusting the model to improve the test score turns the test set into another validation set. Split by customer, product, or time where the real deployment requires independence, not merely by randomly shuffling almost identical rows.

21.3 Types of ML and when each is useful

Supervised classification predicts a category. A review classifier can predict positive or negative. Fraud screening is also classification, but its stakes, labels, costs, and review process are much more demanding than this toy lesson.

Supervised regression predicts a numeric value. A model might estimate packing time or next week's demand. Keep units explicit and measure numeric error. A regression prediction can be outside plausible bounds; the application needs an appropriate policy for invalid or uncertain outputs.

Unsupervised learning looks for structure without supplied target labels. Clustering can group similar descriptions or customer behavior. A cluster number does not automatically mean "loyal customer"; interpretation requires validation. Dimensionality reduction creates a smaller representation while preserving selected information.

Recommendation and ranking choose and order useful items. Content-based methods compare item attributes. Collaborative methods learn from interactions between users and items. A new user or item with few interactions is a cold-start case. Popularity, availability, and content similarity can provide fallbacks.

Deep learning uses neural networks with multiple processing layers. It is useful for many image, speech, and language tasks but adds requirements for data, compute, evaluation, and model distribution. Transfer learning adapts a model already trained on another task or dataset. Reinforcement learning learns policies from rewards through interaction; it is not a sensible first implementation for this small shop.

Generative AI produces text, images, or other content. A language model can draft a product description or answer questions, but generated text may be incorrect. Retrieval-augmented generation, or RAG, first fetches relevant records and supplies them as context. Retrieval reduces some grounding problems; it does not guarantee truthful output or authorize tools to change orders.

21.4 First numeric model: regression you can calculate

Objective: understand fitting a line before using a large library. We invent four examples where packing time increases with item count. The relationship is `predicted_minutes = slope * item_count + intercept`. Least squares chooses the line that minimizes the sum of squared vertical errors. This example has a perfect artificial pattern, so it teaches the arithmetic rather than realistic forecasting quality.

Save the following as regression.py and run `python regression.py`. Python's standard library is enough.

ml_service/regression.py | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

regression.py - lines 1-10

```
001 xs = [1, 2, 3, 4]
002 ys = [4, 6, 8, 10]
003 mean_x = sum(xs) / len(xs)
004 mean_y = sum(ys) / len(ys)
005 numerator = sum((x - mean_x) * (y - mean_y) for x, y in zip(xs, ys))
006 denominator = sum((x - mean_x) ** 2 for x in xs)
007 slope = numerator / denominator
008 intercept = mean_y - slope * mean_x
009 prediction = slope * 5 + intercept
010 print(f'Predicted packing time for 5 items: {prediction:.1f} minutes')
```

Line-by-line explanation

1. Invented numbers of items to pack.

2. Invented packing times in minutes, not measured business data.

3. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

4. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

5. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

6. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

7. Find the least-squares line's change per additional item.

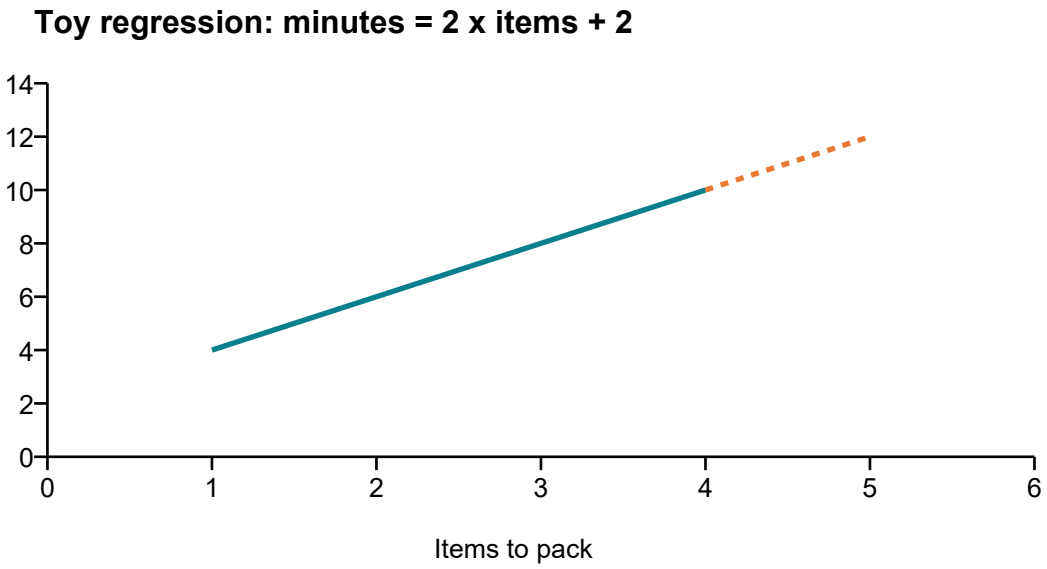
8. Find the predicted value at zero items.

9. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

10. Write this value to the console so the lesson result can be inspected.

Expected output:

```
Predicted packing time for 5 items: 12.0 minutes
```



Worked-example output image generated from the stated toy data.

The fitted slope is two minutes per extra item and the intercept is two minutes. Predicting five items extends beyond the four training inputs, which is called extrapolation. In real data, packaging complexity, staffing, and batch size could break this simple relationship. Try changing one observed time to 20 and inspect how the line changes. An outlier can strongly influence least squares.

21.5 Text features, bag of words, and Naive Bayes

Tokenization splits text into units. Our tokenizer keeps lowercase English alphabetic words. It deliberately does not handle all languages, emoji, negation, or word order. A bag-of-words representation counts words without preserving their order. Thus "not good" requires more thought than independently counting good and not.

Naive Bayes is a probabilistic classifier. It combines a class prior with word likelihoods under a simplifying conditional-independence assumption. A prior is the class frequency before considering the current words. A likelihood measures how compatible a word is with a class under the model. The independence assumption is often false for language, but the method is easy to understand and can be a useful baseline [M6].

Add-one smoothing adds a small count so an unseen word-class combination does not force a probability to zero. Logarithms turn a product of many small likelihoods into a sum of log values and avoid numerical underflow. Normalizing two class scores produces a number between zero and one. That number is not automatically calibrated: 0.94 does not establish that 94 percent of comparable real reviews are positive.

21.6 Complete trained review classifier

Create a new folder `ml_service`. Save model.py, train.py, recommend.py, and main.py from this chapter into that folder. The first two files need only standard Python. The API uses FastAPI and Uvicorn. Training is a separate command; do not retrain on every user request.

ml_service/model.py | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

model.py - lines 1-24

```
001 import math, re
002 from collections import Counter
003
004 def tokens(text):
005     return re.findall(r'[a-z]+', text.lower())
006
007 def fit(rows):
008     counts = {'positive': Counter(), 'negative': Counter()}
009     documents = {'positive': 0, 'negative': 0}
010     for text, label in rows:
011         counts[label].update(tokens(text))
012         documents[label] += 1
013     vocabulary = sorted(set(counts['positive']) | set(counts['negative']))
014     return {'version': 'review-nb-v1', 'counts': {k: dict(v) for k, v in counts.items()},
015             'documents': documents, 'vocabulary': vocabulary}
016
017 def predict(model, text):
018     vocabulary = set(model['vocabulary'])
019     words = [word for word in tokens(text) if word in vocabulary]
020     if not words:
021         return {'label': 'unknown', 'positive_score': 0.5, 'model_version':
    model['version']}
022     scores = {}
023     for label in ['positive', 'negative']:
024         prior = model['documents'][label] / sum(model['documents'].values())
```

Line-by-line explanation

1. Mathematical operations and a simple word tokenizer.

2. Import the named library or module so the following code can use its types and functions.

3. Blank line separating related pieces of code; it performs no operation.

4. Define this named Python operation and its parameters. The indented body runs when the function is called.

5. Convert English letters to lowercase word tokens for this learning model.

6. Blank line separating related pieces of code; it performs no operation.

7. Define this named Python operation and its parameters. The indented body runs when the function is called.

8. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

9. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

10. Repeat the following body once for each value in the stated sequence.

11. Learn how frequently each word occurs in each labelled class.

12. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

13. Build vocabulary from training data only.

14. Produce the stated result and finish this function call. In a build method the result is a widget description.

15. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

16. Blank line separating related pieces of code; it performs no operation.

17. Define this named Python operation and its parameters. The indented body runs when the function is called.

18. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

19. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

20. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

21. Produce the stated result and finish this function call. In a build method the result is a widget description.

22. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

23. Repeat the following body once for each value in the stated sequence.

24. Start with the observed class frequency.

model.py - lines 25-33

```
025         counts = model['counts'][label]
026         denominator = sum(counts.values()) + len(vocabulary)
027         scores[label] = math.log(prior)
028         for word in words:
029             scores[label] += math.log((counts.get(word, 0) + 1) / denominator)
030     difference = max(-60, min(60, scores['negative'] - scores['positive']))
031     positive = 1 / (1 + math.exp(difference))
032     label = 'positive' if positive >= 0.6 else 'negative' if positive <= 0.4 else
    'uncertain'
033     return {'label': label, 'positive_score': round(positive, 4), 'model_version':
    model['version']}
```

Line-by-line explanation

25. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

26. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

27. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

28. Repeat the following body once for each value in the stated sequence.

29. Add-one smoothing avoids zero likelihoods.

30. Bound exponent input for numerical stability.

31. Normalize the two log scores into a model score between zero and one.

32. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

33. Produce the stated result and finish this function call. In a build method the result is a widget description.

ml_service/train.py | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

train.py - lines 1-24

```
001 import json
002 from pathlib import Path
003 from model import fit, predict
004
005 training = [
006     ('soft cotton comfortable shirt', 'positive'),
007     ('excellent fit and quality', 'positive'),
008     ('love the comfortable fabric', 'positive'),
009     ('good quality and soft fabric', 'positive'),
010     ('great fit nice colour', 'positive'),
011     ('excellent comfortable shirt', 'positive'),
012     ('love this great shirt', 'positive'),
013     ('nice soft cotton quality', 'positive'),
014     ('rough fabric uncomfortable shirt', 'negative'),
015     ('poor fit and quality', 'negative'),
016     ('hate the uncomfortable fabric', 'negative'),
017     ('bad quality and rough fabric', 'negative'),
018     ('terrible fit faded colour', 'negative'),
019     ('poor uncomfortable shirt', 'negative'),
020     ('hate this terrible shirt', 'negative'),
021     ('faded rough cotton quality', 'negative'),
022 ]
023 held_out = [
024     ('excellent soft shirt', 'positive'),
```

Line-by-line explanation

1. Import the named library or module so the following code can use its types and functions.

2. Import the named library or module so the following code can use its types and functions.

3. Import the named library or module so the following code can use its types and functions.

4. Blank line separating related pieces of code; it performs no operation.

5. Invented teaching examples, not customer data or a representative training corpus.

6. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

7. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

8. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

9. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

10. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

11. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

12. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

13. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

14. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

15. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

16. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

17. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

18. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

19. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

20. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

21. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

22. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

23. Keep these out of vocabulary learning and word counts.

24. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

train.py - lines 25-39

```
025     ('love nice comfortable cotton', 'positive'),
026     ('bad rough shirt', 'negative'),
027     ('hate faded uncomfortable cotton', 'negative'),
028 ]
029 model = fit(training)
030 path = Path(__file__).with_name('model.json')
031 path.write_text(json.dumps(model, indent=2), encoding='utf-8')
032 correct = 0
033 for text, expected in held_out:
034     answer = predict(model, text)
035     correct += answer['label'] == expected
036     print(expected, '->', answer['label'], '|', text)
037 print(f'Toy holdout accuracy: {correct}/{len(held_out)}')
038 print(json.dumps(predict(model, 'soft comfortable shirt')))
039 print('Saved', path.name)
```

Line-by-line explanation

25. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

26. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

27. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

28. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

29. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

30. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

31. Save data-only parameters instead of executable serialized objects.

32. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

33. Repeat the following body once for each value in the stated sequence.

34. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

35. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

36. Write this value to the console so the lesson result can be inspected.

37. Write this value to the console so the lesson result can be inspected.

38. Write this value to the console so the lesson result can be inspected.

39. Write this value to the console so the lesson result can be inspected.

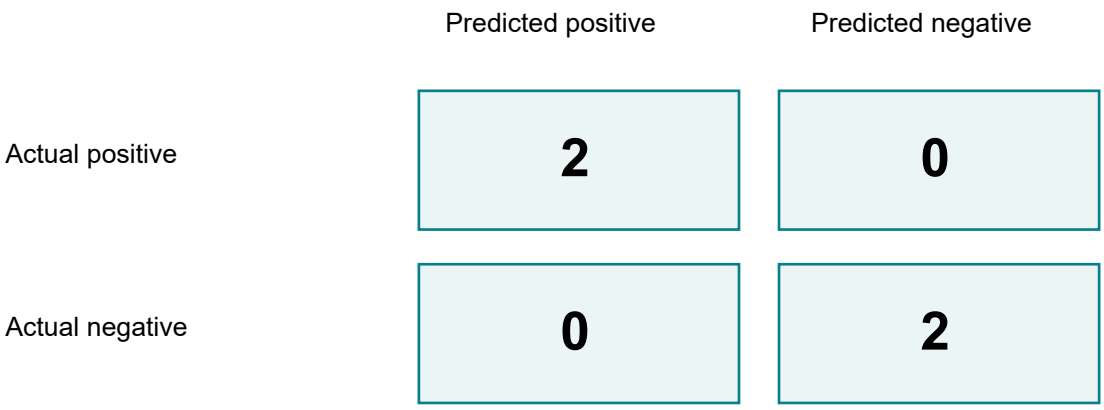
Run `python train.py` in ml_service. It creates model.json beside the script. This file contains vocabulary, counts, document totals, and a version. It is data, not executable Python. The training and held-out examples are intentionally invented and tiny. No real customer reviews were used.

Actual output from this edition's run:

```
positive -> positive | excellent soft shirt
positive -> positive | love nice comfortable cotton
negative -> negative | bad rough shirt
negative -> negative | hate faded uncomfortable cotton
Toy holdout accuracy: 4/4
{"label": "positive", "positive_score": 0.9412,
 "model_version": "review-nb-v1"}
Saved model.json
```

The JSON above is visually wrapped for the page. The program prints the object on one line. Four correct toy predictions show that the pipeline executes; they do not measure performance on realistic reviews. Use a larger representative dataset with independently assigned labels before evaluating usefulness.

Toy holdout confusion matrix (only four examples)



A functional demonstration, not evidence of real-world model quality.

Worked-example output image generated from the stated toy data.

Exercise. Predict `not comfortable`, `comfortable but poor stitching`, an unknown word, and a review in another language. Record failures without changing the test labels to fit the model. The unknown-token fallback is intentional. A production system may need an explicit unsupported-language or low-confidence response, but the exact policy must be validated.

21.7 Metrics, overfitting, and data leakage

Accuracy is the fraction of correct labels. If ninety-nine percent of examples belong to one class, always predicting that class can look accurate while being useless for the rare class. A confusion matrix shows actual versus predicted classes. For the positive class, precision asks how many predicted positives were correct; recall asks how many actual positives were found. F1 combines precision and recall as a harmonic mean. Choose metrics based on the costs of errors [M4].

For regression, mean absolute error averages absolute prediction error in the original units. Root mean squared error gives larger errors more weight. For recommendations, evaluate ranking quality and relevance, not sentiment accuracy. Offline metrics need later validation against user outcomes with carefully designed experiments.

Overfitting means learning accidental training details that do not generalize. Underfitting means the model is too limited or poorly fitted to capture useful structure. More training is not always better. Regularization constrains complexity. Cross-validation repeats training/evaluation across splits when appropriate, but it does not repair a leakage-prone split strategy.

Data leakage means information unavailable at prediction time influences training or evaluation. Learning a text vocabulary from the entire dataset before splitting can leak test information. A demand model must not use a future sales total as an input to predict that same period. Fit preprocessing only on training data and apply the learned transform to validation/test data [M3].

For the included classifier, the vocabulary is built only from the sixteen training examples. The four held-out examples never update counts. This avoids that particular leakage, but the artificial examples are still too simple to support a commercial quality claim. Evaluate difficult language, mixed sentiment, product differences, and time drift separately.

21.8 Complete content-based shirt recommendations

TF-IDF means term frequency times inverse document frequency. It gives a word weight based on its frequency in a description and how common it is across the catalogue. A word shared by every item may be less useful for distinguishing them. Cosine similarity compares vector directions after normalization; it is not a purchase probability [M1].

The next file ranks the two learning-catalogue variants against a text query. It has no user history and does not provide personalized collaborative recommendations. Its fitted catalogue statistics belong to the current catalogue. Recompute or update the index when product descriptions change. Production ranking must also filter inactive or unavailable items and return IDs whose details are fetched through the normal product API.

ml_service/recommend.py | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

recommend.py - lines 1-24

```
001 import math
002 from collections import Counter
003 from model import tokens
004
005 catalogue = [
006     {'id': 1, 'name': 'Oxford shirt', 'description': 'blue cotton formal office regular fit
     shirt'},
007     {'id': 2, 'name': 'Linen shirt', 'description': 'white linen casual summer breathable
    shirt'},
008 ]
009
010 def recommend(query, limit=2):
011     documents = [tokens(row['description']) for row in catalogue]
012     vocabulary = sorted(set(word for doc in documents for word in doc))
013     idf = {word: math.log((1 + len(documents)) / (1 + sum(word in doc for doc in
    documents))) + 1 for word in vocabulary}
014     def vector(text_tokens):
015         counts = Counter(text_tokens)
016         raw = [counts[word] * idf[word] for word in vocabulary]
017         length = math.sqrt(sum(value * value for value in raw))
018         return [value / length for value in raw] if length else raw
019     needle = vector(tokens(query))
020     ranked = []
021     for row, document in zip(catalogue, documents):
022         score = sum(a * b for a, b in zip(needle, vector(document)))
023         if score > 0:
024             ranked.append({'id': row['id'], 'name': row['name'], 'similarity': round(score,
     4)})
```

Line-by-line explanation

1. Import the named library or module so the following code can use its types and functions.

2. Import the named library or module so the following code can use its types and functions.

3. Import the named library or module so the following code can use its types and functions.

4. Blank line separating related pieces of code; it performs no operation.

5. Match these IDs to the two seeded variants in the shop example.

6. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

7. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

8. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

9. Blank line separating related pieces of code; it performs no operation.

10. Define this named Python operation and its parameters. The indented body runs when the function is called.

11. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

12. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

13. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

14. Define this named Python operation and its parameters. The indented body runs when the function is called.

15. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

16. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

17. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

18. Produce the stated result and finish this function call. In a build method the result is a widget description.

19. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

20. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

21. Repeat the following body once for each value in the stated sequence.

22. Dot product of unit vectors equals cosine similarity.

23. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

24. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

recommend.py - lines 25-28

```
025     return sorted(ranked, key=lambda row: (-row['similarity'], row['id']))[:limit]
026
027 if __name__ == '__main__':
028     print(recommend('linen summer'))
```

Line-by-line explanation

25. Produce the stated result and finish this function call. In a build method the result is a widget description.

26. Blank line separating related pieces of code; it performs no operation.

27. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

28. Write this value to the console so the lesson result can be inspected.

Run `python recommend.py`. Expected output for `linen summer`:

```
[{'id': 2, 'name': 'Linen shirt', 'similarity': 0.6027}]
```

The numeric score depends on this exact two-item catalogue and preprocessing. It should change when the catalogue changes. For an unknown query, the function returns an empty list instead of fabricating a meaningful match. Add a deterministic popular-items fallback if that matches your product requirements, and label it appropriately.

Exercise. Add a third description and predict which inverse-frequency weights change. Then separate exact filters such as size M from fuzzy similarity. A shirt with the wrong required size should not outrank a usable shirt merely because its description contains more query words.

21.9 Serving ML through FastAPI

The server loads the approved model at startup and exposes a bounded request schema. POST `/analyze-review` returns a label, positive-class score, and model version. GET `/recommend` returns ranked IDs and similarity scores. GET `/health` identifies the loaded version. For this local lab, the endpoints are unauthenticated and intended for development only. Integrating them into the commercial backend requires normal authorization where appropriate, rate limits, and privacy controls.

ml_service/main.py | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

main.py - lines 1-24

```
001 import json
002 from pathlib import Path
003 from contextlib import asynccontextmanager
004 from fastapi import FastAPI, Query
005 from fastapi.middleware.cors import CORSMiddleware
006 from pydantic import BaseModel, Field, ConfigDict
007 from model import predict
008 from recommend import recommend
009
010 @asynccontextmanager
011 async def lifespan(app):
012     app.state.model =
    json.loads(Path(__file__).with_name('model.json').read_text(encoding='utf-8'))
013     yield
014
015 app = FastAPI(title='ML learning API', lifespan=lifespan)
016 app.add_middleware(CORSMiddleware, allow_origins=['http://localhost:5173'],
017     allow_methods=['GET', 'POST'], allow_headers=['Content-Type'])
018
019 class ReviewIn(BaseModel):
020     model_config = ConfigDict(str_strip_whitespace=True, extra='forbid')
021     text: str = Field(min_length=1, max_length=500)
022
023 @app.post('/analyze-review')
024 def analyze(value: ReviewIn):
```

Line-by-line explanation

1. Import the named library or module so the following code can use its types and functions.

2. Import the named library or module so the following code can use its types and functions.

3. Import the named library or module so the following code can use its types and functions.

4. Import the named library or module so the following code can use its types and functions.

5. Import the named library or module so the following code can use its types and functions.

6. Import the named library or module so the following code can use its types and functions.

7. Import the named library or module so the following code can use its types and functions.

8. Import the named library or module so the following code can use its types and functions.

9. Blank line separating related pieces of code; it performs no operation.

10. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

11. Define this named Python operation and its parameters. The indented body runs when the function is called.

12. Load one approved artifact per server process.

13. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

14. Blank line separating related pieces of code; it performs no operation.

15. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

16. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

17. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

18. Blank line separating related pieces of code; it performs no operation.

19. Declare class ReviewIn and begin the fields or methods that define this type.

20. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

21. Bound input before tokenization and inference.

22. Blank line separating related pieces of code; it performs no operation.

23. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

24. Define this named Python operation and its parameters. The indented body runs when the function is called.

main.py - lines 25-33

```
025     return predict(app.state.model, value.text)
026
027 @app.get('/recommend')
028 def suggestions(query: str = Query(min_length=1, max_length=100)):
029     return {'items': recommend(query), 'method': 'tfidf-cosine-v1'}
030
031 @app.get('/health')
032 def health():
033     return {'ready': True, 'model_version': app.state.model['version']}
```

Line-by-line explanation

25. Produce the stated result and finish this function call. In a build method the result is a widget description.

26. Blank line separating related pieces of code; it performs no operation.

27. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

28. Define this named Python operation and its parameters. The indented body runs when the function is called.

29. Produce the stated result and finish this function call. In a build method the result is a widget description.

30. Blank line separating related pieces of code; it performs no operation.

31. Register or decorate the function below with the behavior named here, such as an HTTP route or a test fixture.

32. Define this named Python operation and its parameters. The indented body runs when the function is called.

33. Produce the stated result and finish this function call. In a build method the result is a widget description.

Create a virtual environment in the folder that contains ml_service. Install `fastapi`, `uvicorn`, `httpx`, and `pytest` with your environment's Python. Then run these commands from ml_service:

```
python train.py
python -m uvicorn main:app --reload --port 8001
```

Use the environment's Python executable if it is not activated. Training writes the data artifact. Uvicorn starts inference on port 8001, leaving port 8000 available for the shop. If startup reports model.json

missing, run the training command in the correct folder. Do not download and load arbitrary executable model files from an untrusted source.

For a large model, inference can consume substantial CPU, GPU memory, or accelerator time. Load it once per intended worker and plan memory use. Use bounded concurrency and request timeouts. Queue slow jobs and provide job status rather than keeping a mobile screen blocked for minutes. The small synchronous endpoints here are suitable only for this small computation.

21.10 Complete Flutter ML screen

Create a Flutter project, add `http` using `flutter pub add http`, and replace lib/main.dart with this listing. Run the server above and start Flutter web on port 5173. If the main shop is already using that browser port, stop that Flutter run or allow a different exact origin in the ML server's CORS configuration.

For an Android emulator, set `--dart-define=ML_API=http://10.0.2.2:8001` and configure local development networking as in the backend chapter. Use HTTPS for a released application. This Flutter code sends JSON and displays results; it does not train the model inside build.

flutter_examples/ml_lab.dart | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

ml_lab.dart - lines 1-24

```
001 import 'dart:convert';
002 import 'package:flutter/material.dart';
003 import 'package:http/http.dart' as http;
004 const mlBase = String.fromEnvironment('ML_API', defaultValue: 'http://127.0.0.1:8001');
005 void main() => runApp(MaterialApp(theme: ThemeData(colorSchemeSeed: Colors.teal), home:
    const MlLab()));
006 class MlLab extends StatefulWidget {
007   final http.Client? client;
008   const MlLab({super.key, this.client});
009   @override
010   State<MlLab> createState() => _MlLabState();
011 }
012 class _MlLabState extends State<MlLab> {
013   final review = TextEditingController(text: 'soft comfortable shirt');
014   final query = TextEditingController(text: 'linen summer');
015   late final http.Client client;
016   bool busy = false;
017   String result = 'No prediction yet';
018   String recommendations = 'No suggestions yet';
019   String error = '';
020   @override
021   void initState() { super.initState(); client = widget.client ?? http.Client(); }
022   Future<void> run(Future<void> Function() action) async {
023     if (busy) return;
024     setState(() { busy = true; error = ''; });
```

Line-by-line explanation

1. Import the named library or module so the following code can use its types and functions.

2. Import the named library or module so the following code can use its types and functions.

3. Import the named library or module so the following code can use its types and functions.

4. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

5. Declare the method and its return type. Its parameters describe the information supplied by the caller.

6. Declare class MlLab extends StatefulWidget and begin the fields or methods that define this type.

7. Allow a controlled HTTP replacement in tests.

8. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

9. Tell the Dart analyzer this declaration implements or replaces an inherited member.

10. Declare the method and its return type. Its parameters describe the information supplied by the caller.

11. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

12. Declare class _MlLabState extends State<MlLab> and begin the fields or methods that define this type.

13. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

14. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

15. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

16. Declare a typed field or value and initialize it from the expression on the right.

17. Declare a typed field or value and initialize it from the expression on the right.

18. Declare a typed field or value and initialize it from the expression on the right.

19. Declare a typed field or value and initialize it from the expression on the right.

20. Tell the Dart analyzer this declaration implements or replaces an inherited member.

21. Declare the method and its return type. Its parameters describe the information supplied by the caller.

22. Declare the method and its return type. Its parameters describe the information supplied by the caller.

23. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

24. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

ml_lab.dart - lines 25-48

```
025     try { await action(); } catch (_) { error = 'ML service unavailable. Try again; normal
    shopping can continue.'; }
026     if (mounted) setState(() => busy = false);
027   }
028   Future<void> analyze() => run(() async {
029     final response = await client.post(Uri.parse('$mlBase/analyze-review'),
030       headers: {'Content-Type':'application/json'}, body:
    jsonEncode({'text':review.text})).timeout(const Duration(seconds:10));
031     if (response.statusCode != 200) throw Exception('Request rejected');
032     final value = jsonDecode(response.body) as Map<String,dynamic>;
033     result = '${value['label']}\nPositive-class score: ${value['positive_score']}\nModel:
    ${value['model_version']}';
034   });
035   Future<void> suggest() => run(() async {
036     final uri = Uri.parse('$mlBase/recommend').replace(queryParameters:
    {'query':query.text});
037     final response = await client.get(uri).timeout(const Duration(seconds:10));
038     if (response.statusCode != 200) throw Exception('Request rejected');
039     final values = (jsonDecode(response.body) as Map<String,dynamic>)['items'] as List;
040     recommendations = values.isEmpty ? 'No matching shirts' : values.map((row) =>
    '${row['name']} (similarity ${row['similarity']})').join('\n');
041   });
042   @override
043   void dispose() { review.dispose(); query.dispose(); if (widget.client == null)
    client.close(); super.dispose(); }
044   @override
045   Widget build(BuildContext context) => Scaffold(
046     appBar: AppBar(title: const Text('ML workshop')),
047     body: ListView(padding: const EdgeInsets.all(20), children: [
048       const Text('Review classifier - tiny teaching model', style: TextStyle(fontSize:20)),
```

Line-by-line explanation

25. Begin an operation whose failures will be handled by the following exception handler.

26. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

27. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

28. Declare the method and its return type. Its parameters describe the information supplied by the caller.

29. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

30. Supply this named property, field, or configuration entry with the value shown here.

31. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

32. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

33. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

34. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

35. Declare the method and its return type. Its parameters describe the information supplied by the caller.

36. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

37. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

38. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

39. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

40. Assign or configure the named value using the expression on the right; any function calls in that expression run to obtain it.

41. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

42. Tell the Dart analyzer this declaration implements or replaces an inherited member.

43. Declare the method and its return type. Its parameters describe the information supplied by the caller.

44. Tell the Dart analyzer this declaration implements or replaces an inherited member.

45. Declare the method and its return type. Its parameters describe the information supplied by the caller.

46. Supply this named property, field, or configuration entry with the value shown here.

47. Supply this named property, field, or configuration entry with the value shown here.

48. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

ml_lab.dart - lines 49-62

```
049       TextField(controller:review, maxLength:500, decoration:const
    InputDecoration(labelText:'Review text')),
050       FilledButton(onPressed:busy?null:analyze, child:const Text('Analyze review')),
051       Text(result),
052       const Text('Scores are not proof that a review is true or false.'),
053       const SizedBox(height:24),
054       const Text('Content-based shirt suggestions', style:TextStyle(fontSize:20)),
055       TextField(controller:query, decoration:const InputDecoration(labelText:'Describe a
    shirt')),
056       OutlinedButton(onPressed:busy?null:suggest, child:const Text('Suggest shirts')),
057       Text(recommendations),
058       if(busy) const LinearProgressIndicator(),
059       if(error.isNotEmpty) Text(error),
060     ]),
061   );
062 }
```

Line-by-line explanation

49. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

50. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

51. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

52. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

53. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

54. Declare the named Dart value. final prevents reassignment; const additionally requires a constant expression.

55. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

56. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

57. Continue the surrounding call or expression with these explicit arguments. Read it together with the opening line and its annotated purpose.

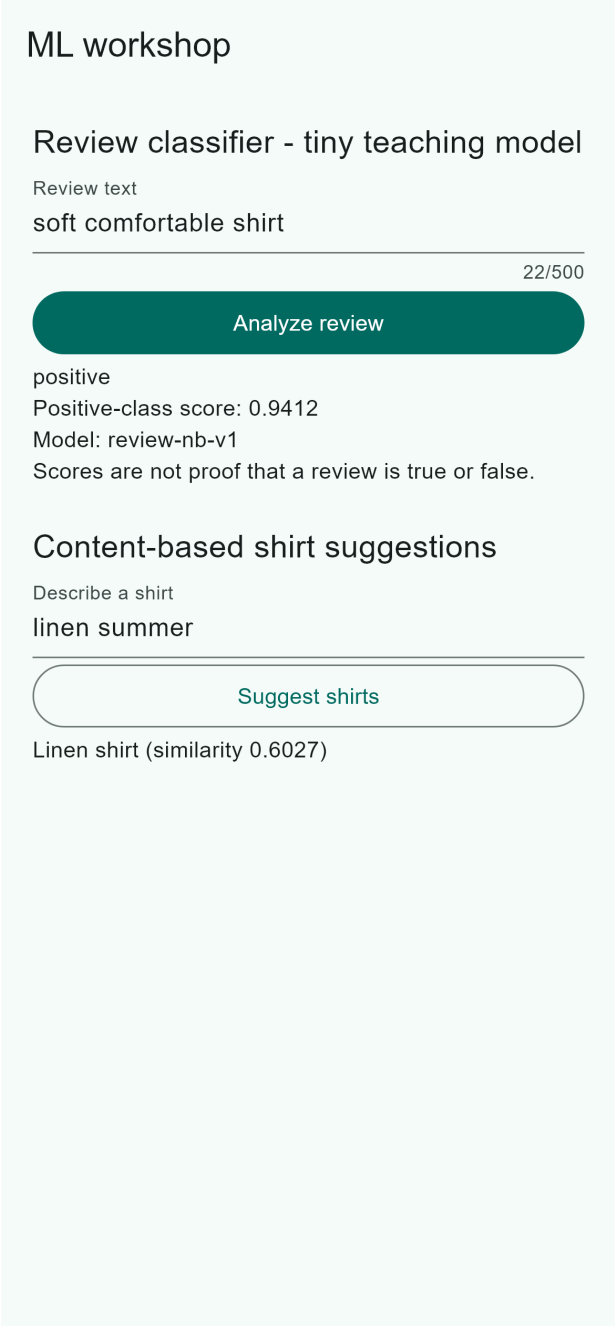
58. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

59. Evaluate this condition; run the following branch only when it is true. This guards the operation described by the expression.

60. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

61. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.

62. Close the innermost open block, argument list, or collection; the punctuation completes that surrounding expression.



Flutter widget-test rendering with the recorded ML API outputs supplied by a controlled HTTP client. The backend predictions were tested separately.

The expected screen displays the model's actual recorded responses for `soft comfortable shirt` and `linen summer`. The image is rendered from Flutter with those responses supplied by a controlled test client. Separate API tests check that the backend produces the expected classes and ranking. The image is not evidence that a live cloud deployment was exercised.

Tap Analyze review to send a POST with a JSON body. The button is disabled during the request. A successful response updates the result and version. Tap Suggest shirts to send a GET with an encoded query. A timeout shows a failure message without breaking ordinary shopping. To use suggestions in the shop, fetch the returned product IDs through your repository and reuse your existing product cards.

Do not interpret "positive" as permission to publish a review or "negative" as a reason to hide it. Sentiment is an uncertain classification of wording, not a truth or abuse detector. Human review and a clearly stated policy are needed for consequential moderation decisions.

21.11 Testing the ML integration

Save the following test file beside main.py. Run train.py once to create the artifact, then `python -m pytest -q` in that folder. Tests check positive, negative, unknown, invalid input, query ranking, and no-match behavior. They are functional integration tests, not a comprehensive model-quality evaluation.

ml_service/test_ml.py | complete source listing

Line numbers are for explanation, not part of the program. Join visually wrapped continuations to the preceding numbered line when copying; preserve indentation at each new line number.

test_ml.py - lines 1-17

```
001 import json
002 from fastapi.testclient import TestClient
003 from main import app
004 from model import predict
005
006 def test_predictions_and_unknown_input():
007     with TestClient(app) as client:
008         assert client.post('/analyze-review', json={'text':'soft comfortable
    shirt'}).json()['label']=='positive'
009         assert client.post('/analyze-review', json={'text':'rough uncomfortable
    shirt'}).json()['label']=='negative'
010         assert client.post('/analyze-review',
    json={'text':'xyzzy'}).json()['label']=='unknown'
011         assert client.post('/analyze-review', json={'text':' '}).status_code==422
012
013 def test_retrieval_ranking_and_empty_match():
014     with TestClient(app) as client:
015         assert client.get('/recommend',params={'query':'linen
    summer'}).json()['items'][0]['id']==2
016         assert client.get('/recommend',params={'query':'cotton
    office'}).json()['items'][0]['id']==1
017         assert client.get('/recommend',params={'query':'xyzzy'}).json()['items']==[]
```

Line-by-line explanation

1. Import the named library or module so the following code can use its types and functions.

2. Import the named library or module so the following code can use its types and functions.

3. Import the named library or module so the following code can use its types and functions.

4. Import the named library or module so the following code can use its types and functions.

5. Blank line separating related pieces of code; it performs no operation.

6. Define this named Python operation and its parameters. The indented body runs when the function is called.

7. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

8. Check the expected result. A mismatch fails the test and identifies a broken contract.

9. Check the expected result. A mismatch fails the test and identifies a broken contract.

10. Check the expected result. A mismatch fails the test and identifies a broken contract.

11. Check the expected result. A mismatch fails the test and identifies a broken contract.

12. Blank line separating related pieces of code; it performs no operation.

13. Define this named Python operation and its parameters. The indented body runs when the function is called.

14. Enter a managed resource scope. The connection or transaction is cleaned up when the block exits.

15. Check the expected result. A mismatch fails the test and identifies a broken contract.

16. Check the expected result. A mismatch fails the test and identifies a broken contract.

17. Check the expected result. A mismatch fails the test and identifies a broken contract.

Separate four test questions: Does preprocessing reproduce training behavior? Does inference return the expected contract? Does the model perform adequately on representative unseen data? Does the app handle latency and failure? A correct HTTP 200 response answers only part of the problem.

Keep a small fixed regression suite of known examples and a larger independently maintained evaluation dataset. Store the dataset version, code version, model version, metric definitions, and thresholds with every release. Test a rollback to the previous model, not merely the deployment of the new one.

21.12 On-device ML versus server inference

On-device inference can work offline and keep some raw input on the device. It can reduce network latency, but uses device memory, battery, and compute. Model size, supported operations, acceleration, and platform integration constrain what can run. A model included in an app can be inspected; do not treat its embedded parameters or keys as secret.

Server inference centralizes updates and allows larger models and controlled hardware. It needs network access, secure transport, request budgets, service availability, and a justified data-processing policy. A hybrid design might run a small local classifier and use the server for optional richer analysis.

LiteRT is Google's on-device inference framework. Its supported model formats, runtimes, and platform APIs must match the exported model [M5]. A Flutter integration can use a maintained compatible plugin or a platform channel to native code. A platform channel is a bridge between Dart and platform-specific code. Do not assume every TensorFlow, PyTorch, or scikit-learn model can be copied into a Flutter assets folder and executed unchanged.

The on-device sequence is export a supported model, bundle or securely download it, load an interpreter/session, prepare input tensors, run inference, interpret output tensors, and release resources. A tensor is a multidimensional array of numbers. Image preprocessing must reproduce training: dimensions, crop/resize policy, colour order, scale, and datatype. A mismatch such as RGB versus BGR can produce plausible-looking but wrong results.

For a shirt-image classifier, first test exported-model outputs against reference Python outputs on the same images. Then test representative phones for latency, memory, battery, and thermal behavior. Quantization reduces numeric precision and can shrink/accelerate a model, but evaluate the quality change. The book's executable ML project uses server inference; this on-device section is an integration design, not an untested plugin snippet presented as working code.

21.13 Embeddings, visual search, and generative assistants

An embedding is a numerical representation learned so selected similarities become easier to measure. Text embeddings can support semantic search; image embeddings can help find visually similar shirts. An embedding does not automatically encode the exact business constraints you need, such as size availability or price range. Apply those constraints separately.

For visual search, the app obtains an image with appropriate permission, the server or device computes an embedding, a vector index retrieves candidates, and the product service supplies current catalogue details. Evaluate image rights, personal content in photographs, upload size, retention, and deletion. Do not store every camera image indefinitely for unspecified future training.

For a shop assistant, retrieve current product and policy facts from authoritative sources. Generated answers should identify uncertainty and use structured tools for actual order lookup. Treat retrieved documents and customer messages as data, not privileged instructions. A model must not issue refunds merely because a prompt asks it to ignore permission checks. Keep authorization in deterministic server code and require the appropriate user action for consequential operations.

21.14 Production ML: drift, privacy, feedback, and rollout

Data drift means the input distribution changes, such as new languages or new product styles. Concept drift means the relationship between inputs and target changes. Monitor feature distributions, unknown-input rates, latency, error rate, and delayed quality labels. A stable uptime chart does not prove model quality is stable.

Collect only the data needed for the stated purpose and permitted by your product's policy and applicable requirements. Remove unnecessary personal identifiers from training data and logs. Define retention, access, and deletion processes. Feedback is useful but can be noisy or malicious; do not automatically retrain on every user's correction without review.

Use shadow evaluation or a limited rollout before replacing a model for everyone. Compare against the existing baseline and define rollback triggers. Test recommendations for relevance, catalogue coverage, and harmful feedback loops rather than optimizing only immediate clicks. Repeatedly showing the same popular shirts can distort the interaction data used for the next training cycle.

Final ML exercise. Add an optional "Similar shirts" section to the product page using the recommendation endpoint. Keep the ordinary catalogue usable if inference fails. Show current product price and stock from the product service, not the ML response. Write tests proving that an unavailable ML service cannot block checkout and that a suggested ID cannot bypass normal ownership or product validation.

ML reference reading

[M1] Text features and TF-IDF: https://scikit-learn.org/stable/modules/feature_extraction.html [M2] Linear models and logistic regression: https://scikit-learn.org/stable/modules/linear_model.html [M3] Leakage and preprocessing pitfalls: https://scikit-learn.org/stable/common_pitfalls.html [M4] Evaluation metrics: https://scikit-learn.org/stable/modules/model_evaluation.html [M5] LiteRT on-device inference: https://developers.google.com/edge/litert [M6] Naive Bayes and smoothing: https://scikit-learn.org/stable/modules/naive_bayes.html These references explain established methods and current integration options. The standard-library teaching implementations in this chapter are original small examples, not copied library source.

Worked exercises and debugging notebook

Common errors and how to reason about them

Undefined name or missing import. Read the exact identifier and file location. Check spelling, case, scope, and imports. Do not install a new package when the value is simply declared inside another function.

Type mismatch. State the type you have and the type the operation expects. JSON `dynamic`, nullable map lookups, and text from controllers are common boundaries. Convert and validate once at that boundary.

setState after dispose. An asynchronous result arrived after the screen left the tree. Cancel work when appropriate and check mounted before UI updates. Do not catch and ignore every framework exception.

Vertical viewport given unbounded height. A scrolling area lacks a finite height in its layout context. In a Column, Expanded can give it the remaining bounded space. In another scrollable, choose one owner of scrolling or a sliver-based composition instead of blindly setting shrinkWrap everywhere.

HTTP 401, 403, 409, or 422. Respectively inspect session validity, permission, current business state, and input validation details. These are not interchangeable "network errors."

Connection refused. Check that the correct process is listening on the expected host and port. **CORS failure.** Check the browser origin against the backend allowlist. **TLS failure.** Check certificate and hostname configuration; do not turn verification off in a release.

Database unique conflict. Decide whether the request is a duplicate, a retry, or invalid new data. Return an appropriate result. **Deadlock or serialization failure.** Retry the whole safe transaction using the same operation identity, with a bounded policy.

Order created but client timed out. Query or retry the same idempotent operation. **Payment setup unknown.** Reconcile provider records before attempting another gateway order. **Webhook duplicated.** Return a successful acknowledgement after verifying that the earlier effect was applied safely.

Sample exercise answers

For quantity four at 79,900 paise, the line total is 319,600 paise, displayed as INR 3196.00. A negative quantity is invalid even if the text successfully parses as an integer. A missing map key and a present zero-stock key are different states; decide how each should appear.

A reset counter sets the authoritative count to zero inside setState; it does not create another screen or alter the Text widget directly. A completed task should be a field on a Task model with stable identity, not a Boolean list that can drift away from the task list after deletion.

A warehouse permission test should log in a warehouse account, adjust permitted inventory, and assert that a refund endpoint returns 403. A customer editing a role field must not become an administrator. Database constraints complement these authorization checks but do not replace them.

A retry test must check the order ID and confirm stock decreased only once. Test refund records and inventory separately: reversing a payment does not prove returned goods were received.

Reference glossary

API: a defined way for software components to communicate. SDK: tools and libraries for development. UI: the interface a person uses. Widget: a Flutter UI description. State: changing information that affects behavior or display. Repository: an application-facing boundary for data access. DTO: a data-transfer shape used at a boundary.

HTTP: request-response communication protocol. JSON: structured text data format. Endpoint: a method and path handled by an API. Backend: trusted server-side application logic. Database: organized persistent data. Schema: the structure and rules of stored data. Migration: a versioned schema/data change.

Authentication: establishing identity. Authorization: deciding permitted actions. JWT: signed claims container. Hash: one-way digest; password hashing requires an appropriate slow salted algorithm. Secret: confidential credential or signing material. RBAC: permission grouping by roles.

Transaction: an atomic group of database changes. Constraint: a database-enforced rule. Index: a structure supporting selected lookups. Idempotency: repeated application does not duplicate the intended effect. Webhook: a service-originated HTTP event notification. Reconciliation: resolving differences between authoritative records.

CI: automated checks on changes. CD: release preparation or deployment automation, according to context. Container: isolated packaged process environment. CDN: distributed content delivery/cache service. Observability: the ability to understand system behavior through evidence such as logs, metrics, and traces.

DDD: domain-driven design. CQRS: separating command and query responsibilities. Saga: durable coordination of local transactions and compensations. Tenant: an isolated customer organization in a shared service. SLO: measurable service target. RPO: acceptable recovery data loss. RTO: acceptable recovery duration.

Sources, versions, and verification

The explanations and teaching programs are original worked material. The following official references were consulted for current setup and API guidance on 4 October 2026. Package APIs and deployment requirements change; use the recorded environment for reproducing this edition and review current migration guidance before upgrading.

[S1] Flutter installation: https://docs.flutter.dev/install

[S2] Dart language reference: https://dart.dev/language

[S3] Flutter navigation: https://docs.flutter.dev/ui/navigation

[S4] Shared preferences package: https://pub.dev/packages/shared_preferences

[S5] Riverpod getting started and migration: https://riverpod.dev/docs/introduction/getting_started

[S6] FastAPI password hashing and JWT guide: https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/

[S7] Flutter secure storage: https://pub.dev/packages/flutter_secure_storage

[S8] Razorpay Flutter integration: https://razorpay.com/docs/payments/payment-gateway/flutter-integration/standard/integration-steps/

[S9] Razorpay webhook validation: https://razorpay.com/docs/webhooks/validate-test/

[S10] PostgreSQL constraints: https://www.postgresql.org/docs/current/ddl-constraints.html

[S11] PostgreSQL transactions: https://www.postgresql.org/docs/current/tutorial-transactions.html

[S12] PostgreSQL backup and restore: https://www.postgresql.org/docs/current/backup.html

[S13] OWASP REST security: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html

[S14] Flutter testing: https://docs.flutter.dev/testing/overview

[S15] Flutter performance: https://docs.flutter.dev/perf/best-practices

[S16] Docker containers: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

[S17] GitHub Actions: https://docs.github.com/en/actions/get-started/understand-github-actions

[S18] Flutter Android release: https://docs.flutter.dev/deployment/android

[S19] Flutter iOS release: https://docs.flutter.dev/deployment/ios

[S20] Flutter web release: https://docs.flutter.dev/deployment/web

[S21] SQLAlchemy: https://docs.sqlalchemy.org/en/20/orm/quickstart.html

Executed environment: Flutter 3.41.7 with Dart 3.11.5 on Windows. The resolved Flutter packages include http 1.6.0, shared_preferences 2.5.5, and flutter_riverpod 3.3.2. This is the tested environment for the main examples, not a claim that these are the newest releases.

All 15 standalone Dart console listings executed successfully. Final Flutter static analysis reported no issues across the main examples, inline Flutter listings, widget workshop, and asynchronous examples. The counter interaction and form-validation/confirmation tests passed. Widget tests also exercised calculator input/division and task entry, and rendered counter, calculator, tasks, shopping, and sign-in screens. The printed UI pictures use those renders where labelled; other pictures are explanatory illustrations. The advanced Python domain example produced the expected paid state and deduplicated total.

Five shop-backend tests and one employee CRUD test passed using Python 3.12 and a disposable SQLite database. The shop tests cover request idempotency, authoritative prices, stock rollback, order ownership, denied admin access, refresh rotation, logout revocation, cancellation release, rejected price injection, and signed payment-event validation/deduplication. The test runtime emitted a Starlette notice that its httpx TestClient compatibility path is deprecated; the recorded environment still completed the tests. The PDF prints the exact tested backend package versions in Level 20.

Not executed here: PostgreSQL concurrency/load tests, Docker/CI workflows, Android/iOS release builds, real payment-account calls, real webhook delivery, native Razorpay checkout, real refunds, cloud deployment, and disaster-recovery drills. The native payment lab and operations files are instructional integrations that require your own environment and credentials. The commercial expansion sections explicitly describe work beyond the implemented teaching baseline.

The ML training and recommendation programs ran successfully. On the four invented holdout reviews, all four predictions matched their toy labels; this is not a real-world quality estimate. Two ML API tests passed, covering sentiment classes, unknown text, rejected blank input, recommendation ranking, and no-match behavior. The Flutter ML screen was rendered with a controlled HTTP client supplying the measured response examples.

The PDF was checked by text extraction for all original 21 levels plus the ML chapter and by rendered-page review for readability, code wrapping, diagrams, and pagination. When copying code from this PDF, omit printed line numbers and join visual continuation lines; retain the original indentation shown at each new numbered line.